Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Contents

1 Scheduling Part 1 **5**

1.1 Course information . . . 5
Groups. . . . 5
Assignment and grading. . . . 5
Accounts. . . . 5
Organisation. . . . 6

1.2 Where we are . . . 6
Approach of the course. . . . 6

1.3 Unit 1 - Process management: the ideas . . . 7

1.3.1 Program, process and task . . . 7
Process vs task. . . . 7

1.3.2 One CPU, several tasks . . . 7

1.3.3 Task states . . . 7

1.3.4 What does a task look like? . . . 9

1.3.5 The Task Control Block (TCB) . . . 10

1.3.6 When does the scheduler run? . . . 11

1.3.7 Preemptive vs cooperative . . . 12

1.3.8 Exercise: follow two tasks . . . 13
Setting. . . . 13
Method. . . . 13
Solution. . . . 13

1.4 Scheduling criteria . . . 14

1.5 Scheduling algorithms . . . 15

1.5.1 Exercise conventions . . . 15
Tie rules. . . . 16
Exam. . . . 16

1.5.2 First-Come, First-Served (FCFS) . . . 16
Example 1: a long task first. . . . 17
Example 2: the long task last. . . . 17
Exercise: FCFS. . . . 17

1.5.3 Round Robin (RR) . . . 18
Worked example, $q = 2$. . . . 19

1.5.4 Exercise: round robin, $q = 2$ . . . 19

1.5.5 Exercise: change the quantum . . . 21
$q = 1$. . . . 21
$q = 4$. . . . 21

1.5.6 Quantum size and context-switch overhead . . . 22
Exercise: overhead. . . . 23

1.5.7 Priority-based scheduling . . . 23
Worked example. . . . 24
1.5.8 Starvation and aging . . . 24
Exercise: priority and starvation. . . . 24
1.6 Unit 1: summary . . . 26
Are these algorithms used in practice? . . . 26
1.7 Unità 2 - Context switch and round robin in the scratch-OS . . . 26
Link with the project. . . . 26
The code of this week. . . . 27
1.7.1 The folder `week-05/base` . . . 28
1.7.2 Running the two tasks . . . 29
1.7.3 What must be saved to pause a task . . . 30
Exception entry: what the CPU saves, and what it does not. . . . 31
1.7.4 A switch from A to B: the two stacks . . . 31
Compared with week 4. . . . 32
1.7.5 Inspect: `struct tcb` in `kernel/task.h` . . . 32
Why must `sp` be the first field? . . . 32
The task table. . . . 32
1.7.6 Who does the switch? PendSV . . . 32
How to pend PendSV. . . . 33
Priorities and tail-chaining. . . . 33
1.7.7 The switch, step by step: `PendSV_Handler` in `kernel/pendsv.s` 34
Why may the handler call C without saving `r0`–`r3`? . . 35
Why is the entry `lr` never saved? . . . 36
Other remarks. . . . 36
1.7.8 Inspect: `task_switch_context`, the round-robin circle . . . 36
A's slice ends while B is BLOCKED: who runs? . . . 36
A and B both BLOCKED: who runs? . . . 36
1.7.9 Try it: one switch in gdb . . . 37
Continue: A saved, B chosen. . . . 38
1.7.10 The first run of a task: a fake frame . . . 39
Inspect: `task_init_stack` in `kernel/task.c`. . . . 39
1.7.11 One MPU window, moved at every switch . . . 40
Try it: a rogue task after a switch. . . . 40
1.7.12 SysTick, the time slice and `task_increment_tick` . . . 41
Why `(int32_t)(tick_count - wake_tick) >= 0` and not `tick_count >= wake_tick`? . . . 43
Lesson: an OS is many small parts working together. . 43
1.7.13 Try it: change the quantum . . . 43
1.7.14 Task states in the code . . . 44
1.7.15 Giving the CPU away: yield, delay, delay until, exit . . . 44
Why two delays? . . . 44
Exit. . . . 45
Inspect: `task_block_until` and the delay services. . . 45
1.7.16 Try it: no preemption, no yield . . . 46
1.7.17 The idle task . . . 46
Try it: delays and the idle task. . . . 47

1.7.18 Seeing the schedule: a trace . . . 48
1.7.19 Writing the trace: record, buffer and recording . . . 49
Checking the records in gdb. . . . 50
1.7.20 Writing the trace: reading and printing the records . . . 51
1.7.21 Writing the trace: the Gantt chart . . . 53
1.7.22 Try it: the trace of the other runs . . . 55
1.8 Unit 2: summary . . . 56
1.9 Unità 3 - Real-time basics . . . 56
1.9.1 Real-time systems . . . 56
Example: braking. . . . 57
1.9.2 Hard, firm and soft deadlines . . . 57
1.9.3 Exercise: hard, firm or soft? . . . 57
Solution. . . . 58
1.9.4 Real-time is not the same as fast . . . 58
Example. . . . 59
1.9.5 Periodic, sporadic and aperiodic tasks . . . 59
Examples from an engine controller. . . . 60
1.9.6 The task model . . . 61
1.9.7 Timing parameters and notation . . . 61
1.9.8 Utilisation . . . 62
In real life: what if $U > 1$? . . . 63
1.9.9 Hyperperiod . . . 63
Two ways to check schedulability. . . . 63
1.9.10 Why the worst-case execution time? . . . 64
Estimating the WCET is hard. . . . 64
1.9.11 Exercise: notation . . . 64
Solution. . . . 65
1.10 Timeline scheduling . . . 65
1.10.1 The idea and the rules . . . 66
1.10.2 Timeline: worked example . . . 66
Why it is not optimal. . . . 67
1.10.3 How a cyclic executive is implemented . . . 67
1.10.4 Timeline scheduling: pros and cons . . . 69
1.10.5 Non-harmonic periods . . . 69
Example. . . . 69
1.10.6 Exercise: timeline scheduling . . . 70
1.10.7 Exercise: a frame too small . . . 70
1.11 A tool to check your schedules: schedsim . . . 72
1.11.1 Try it: explain, solve and check a timeline . . . 72
1.11.2 The answer format . . . 73
Why use the tool. . . . 74
1.12 Unit 3: summary . . . 74
1.13 A prova di coglione . . . 75
1.13.1 Unità 1 . . . 75
1.13.2 Unità 2 . . . 78
1.13.3 Unità 3 . . . 85

Chapter 1

Scheduling Part 1

1.1 Course information

Groups. The professor checked the spreadsheet of the groups uploaded by the deadline (last Friday). Student ID colours: **white**, enrolled, everything fine; **yellow**, in the list but with the *career not active*: no problem for the project, but it must become active before the exam, or the exam cannot be recorded; **red**, *not eligible* (e.g. attended the course for the first time two years ago). A one-person group is not acceptable: the project is practically impossible to complete alone. Students not yet in the spreadsheet must email the professor **by the end of the day** of the lecture, with their student ID and the group they want to join.

Assignment and grading. The assignment is published, and correctly registered groups can start: a **loader** for the scratch-OS, so that programs are loaded at run time instead of being statically compiled into the OS image. Groups of **4 or fewer** have the basic requirements, groups of **5 or 6** also 3–4 additional ones; otherwise the projects are identical. The project is not complex, but the requirements document is long, since it tries to anticipate every question: read it carefully (about 5–6 hours) *before* asking. Each requirement is worth the points stated in the assignment: the basic ones up to **14 points** for every group; the extra ones add nothing, but **missing** them **subtracts** points, so the basic requirements weigh the same in both versions.

Accounts. When the groups are frozen (after the late registrations), each group gets an account and a repository on the lab's Git server, with the credentials sent by email. Students without a student ID yet can share a teammate's account (the password is random, so it can be shared safely). Nobody is evaluated on the number of contributions per account. *Update:* the spreadsheet is now **frozen**: the accounts exist and the project can start; whoever emailed and got no answer should write again. Check your team composition: whoever registered with an old or temporary ID and now has a new one must email it, since each account is named after the current student ID and uses its student email. The Politecnico network was temporarily unreachable from outside, teaching-material servers included (reachable through the VPN); the material is published only on the professor's website, built on Git, not on the teaching portal.

Organisation. From this week programming gets lighter and theory grows. This week and the next are on **scheduling**: the only implementation is the round-robin scheduler of the scratch-OS, in the second part of this lecture (Unit 2), the last difficult programming part of the course. No new code comes in the next lecture or next week, which leaves about two weeks to study it; the rest is theory and exercises on paper. One last week on the scratch-OS then adds a small synchronisation system call, and the second half of the course uses **FreeRTOS** from the *user's* point of view: knowing how an OS works inside, we write application software on top of a real OS, very similar to the very limited scratch-OS but with much more, at a slightly higher level (with a simple exam exercise on it too). The labs start to include exam-like exercises on the first half of the course: next week's is on scheduling (some implementation, plus exam-style exercises in the optional part), the following one only exam-style exercises.

1.2 Where we are

The scratch-OS has system calls, privileged and unprivileged software, memory management and protection: since week 4 each task has its own RAM partition, and the MPU stops a task from touching memory that is not its own, so the kernel is safe from its tasks and the tasks from each other. But tasks still run *one after the other*: B starts only when A has called `exit`, and while A waits one second between two lines the CPU is simply wasted. This week the kernel shares the CPU, and A and B run *at the same time*. The units, in the same order in slides and notes:

- **Unit 1, Process management: the ideas** (no code): tasks and their states, the task control block, when the scheduler runs, and three algorithms: first-come first-served, round robin and priority scheduling, compared on turnaround, waiting and response time with schedules solved on paper.

- **Unit 2, Context switch and round robin**: the preemptive round-robin scheduler prepared in the scratch-OS (task control block, context switch in the `PendSV` exception, SysTick-driven time slice, system calls that give the CPU away, idle task), observed with gdb, a different time slice and demo builds; then a hand-written **trace** that records every task switch and prints the schedule as a Gantt chart, like those drawn on paper in Unit 1.

- **Unit 3, Real-time basics**: tasks that must finish **on time**: hard, firm and soft deadlines; periodic, sporadic and aperiodic tasks; the notation (release, period, deadline, response time, lateness); utilisation and hyperperiod; **timeline scheduling** (the cyclic executive), built by hand and checked with the course tool `schedsim`.

Approach of the course. There are many algorithms to study, but the *mechanism* that runs any of them is always the same. So the scratch-OS implements **only one** (round robin): changing algorithm changes only a *high-level* part of the OS, while the low-level mechanisms, above all the **context switch**, stay the same. An algorithm is a few lines of C, and implementing all of them would teach little; the context switch is much harder, and different on every machine. The implementation

effort goes there; the other algorithms are studied in theory (and can be implemented as a personal exercise).

1.3 Unit 1 - Process management: the ideas

1.3.1 Program, process and task

A **program** is code and data in memory, the result of compiling (e.g. the ELF file of our projects): by itself it does nothing. A **process** is a *program in execution*: copied from FLASH to memory, with its kernel data structures created, and so on. It is the code (**text**) plus everything that changes while it runs: **data** (global and static variables), **stack** (local variables, arguments, return addresses) and **CPU state** (the PC, pointing at the next instruction, the SP and all the other registers). It runs **sequentially**: never two of its own instructions at the same time.

Process vs task. Synonyms: general-purpose OSs (e.g. Linux books) talk about *process management*, embedded OSs (e.g. FreeRTOS) about *task management*. An embedded **task** has its own stack and CPU state, but usually *no* private address space, open files or parent process: all tasks are linked into **one** program with the kernel. Week 4's A and B are tasks in exactly this sense: own code, stack and partition, in one executable.

1.3.2 One CPU, several tasks

A core executes one instruction stream at a time, so the tasks must share it: the kernel **interleaves** them (A, then B, then A again). Truly simultaneous execution is impossible on one core, but with short turns both seem to run together to a human; this is the simplest solution, and ours. Stopping A and starting B means saving A's whole CPU state (its *context*) and loading B's: a **context switch**. Done correctly, A cannot tell it was stopped: it resumes at the same instruction with the same registers. The **scheduler** is the kernel part that decides which task runs next, following a **scheduling algorithm** (**policy**): e.g. each task runs 10 ms, then gives the CPU to another; or each task runs to its end, then the next. Bachelor OS courses present the context switch as simple and abstract, but in practice it is complex because **machine-dependent**: it requires knowing exceptions, how a program is interrupted, how to move between privileged and unprivileged mode, and many other processor details. We implement it for the Cortex-M3; on an Intel processor it would be a completely different story.

The rest of the unit answers: what does the kernel know about each task? *When* does the scheduler decide, and *how*?

1.3.3 Task states

Only the kernel moves a task between **execution states**, which determine how the scheduler works. Even very simple OSs have **four** (complex ones such as Linux have more).

- **RUNNING**: the task holds the CPU. On one core *at most one* task is RUNNING; every other started task is in another state.

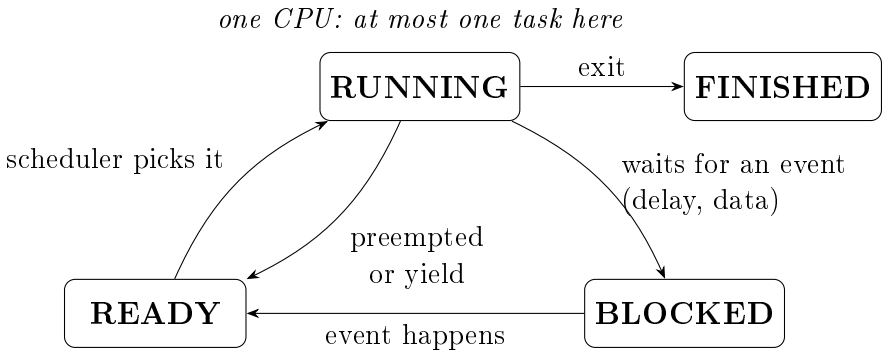


Figure 1.1: Task states and transitions.

- **READY**: in memory with everything it needs except the CPU. The READY tasks are the "bucket" from which the scheduler takes the next task: the policy, implemented by an algorithm, decides which.
- **BLOCKED** (**waiting**): the task cannot run until an *event* (a delay ends, data arrives, a resource is freed), so the CPU would be useless to it. E.g. during the scratch-OS's delay (sleep) the kernel parks the task here, which tells the scheduler not to give it the CPU. Whenever it gets control, the OS checks whether each blocked task's reason is gone (a task that sleeps 10 ms must run again after 10 ms). Any system call that behaviourally requires waiting blocks, not only delays (from a question in class): in Linux, reading or writing a file takes milliseconds while the CPU works at the micro/nanosecond scale, so a process calling `read` or `write` is BLOCKED until the disk (or other device) completes, usually signalling it with an interrupt, and then READY again. "Blocked" is a behavioural notion, not a hardware one.
- **FINISHED**: the task called `exit` (in the scratch-OS, the system call that tells the kernel the task is over) and never runs again. The OS knows the task is completed and can free the dynamic structures allocated for it; the scratch-OS has none, so for us this state does nothing more.

Transitions:

- **READY** → **RUNNING**: the scheduler picks the task.
- **RUNNING** → **READY**: the kernel takes the CPU away (**preemption**) or the task gives it up (**yield**); either way it could still run.
- **RUNNING** → **BLOCKED**: the task asks to wait (a delay, data not yet available).
- **BLOCKED** → **READY**: the event happens. **Never straight to RUNNING**, in any OS: another task has the CPU, so the woken task waits its turn like every READY task, and the policy decides.
- **RUNNING** → **FINISHED**: the task calls `exit`.

With N tasks on one CPU, at any moment exactly 1 is RUNNING, M are BLOCKED and $N - M - 1$ READY (FINISHED ones aside). General-purpose

systems also have a **NEW** state, for a process being created and not ready yet; our kernel creates all tasks at boot, before any runs, so it needs none. A scheduler is thus a *policy*: a simple algorithm (a function) that knows all tasks and their states, uses this diagram to decide where each task goes, and picks the READY task that becomes RUNNING.

1.3.4 What does a task look like?

Scheduling needs assumptions on how tasks are written. A general-purpose OS lets a process be structured freely (as the tasks of the course project must be), but in the course OS a task has one of two structures: not just a simplification, but the assumption of *every* embedded OS. (Most function names of this week come from FreeRTOS, which allows both, so they will look familiar later.) An embedded task is a C function that is:

- **run to completion** (non-periodic task): setup, then the work, then exit to FINISHED (like week 4's task A); or
- an **endless loop** (periodic task[1]): setup, then forever wait for an event, do the work, wait again (like week 4's task B). It never ends; this structure is very important, especially in real-time embedded systems.

```
/* Run to completion, like week 4's task A */
void task_a( void )
{
    setup();
    do_work();
    /* then return: the kernel calls exit for it */
}

/* Endless loop, like week 4's task B */
void task_b( void )
{
    setup();                    /* once, before the loop */
    for ( ;; )
    {
        wait_for_event();       /* e.g. "one second has passed": BLOCKED un
        do_work();
    }
}
```

Most embedded tasks are endless loops that never reach FINISHED, e.g. monitoring a sensor, read as long as the device is on: at each new reading the task wakes up, processes it and waits for the next. Waiting for an event needs OS support (very simple in our case), and it is what makes sharing work: a BLOCKED task leaves the CPU to the others, while a task that never waits (looping on busy work) keeps it as long as the kernel lets it.

[1]Here "periodic" is used informally, for a task that repeats its work forever. Unit 3 gives the precise definition of periodic, sporadic and aperiodic tasks.

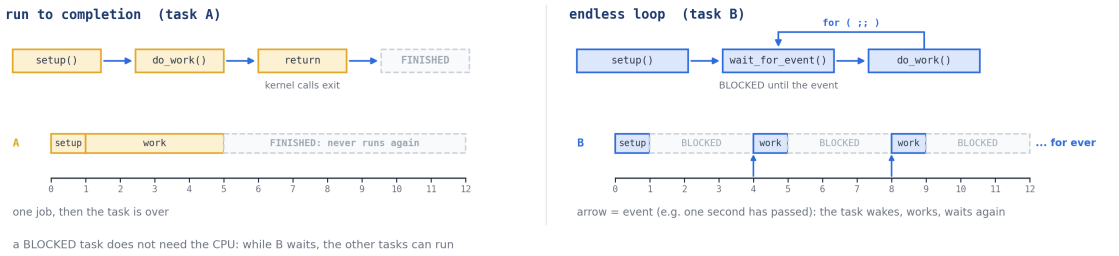


Figure 1.2: The two structures of an embedded task. A run-to-completion task does its job once and then is FINISHED for ever; an endless-loop task waits for an event (BLOCKED), works, and waits again, never ending. While it is BLOCKED, the CPU is free for the other tasks.

1.3.5 The Task Control Block (TCB)

With a scheduler, four run-time states and two task structures, the kernel needs a **data structure** per task, above all to save the full CPU context in memory when the task stops. Week 4's `struct task`, made to run two tasks, is extended to hold more: the **TCB** (**process control block**, PCB, in general-purpose systems), one record per task, so that the kernel can stop it and restart it later. It holds:

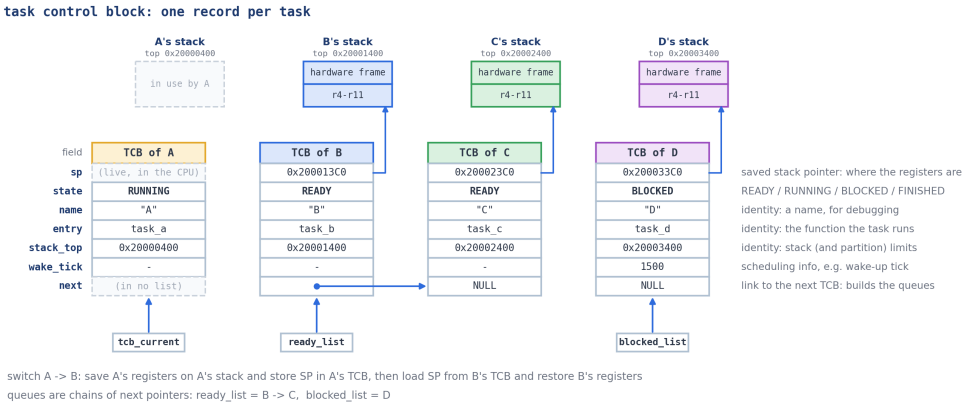


Figure 1.3: A generic TCB, one per task. In a stopped task, `sp` points at the lowest saved word (`r4`) on the task's own stack: the 16 saved words lie just below the stack top, so `0x20001400` − 64 = `0x200013C0`. In the running task, `sp` is not meaningful, because its registers are live in the CPU. The `next` links chain the TCBs into queues: `ready_list` = B → C, while D, BLOCKED until tick 1500, is in `blocked_list`.

- the **saved stack pointer**: where the task's registers were saved (pushed on its own stack) when it stopped; already in week 4's structure;
- the **state**: READY, RUNNING, BLOCKED or FINISHED; new, not in last week's code;
- the **identity**: name or number, entry function, stack and partition limits (mostly already in week 4's `struct task`). A name is not mandatory, but every OS has one: when debugging, a mnemonic name is much easier than a number;

- **scheduling information**: whatever the algorithm needs, e.g. a priority, or the wake-up time of a delayed task;
- a **link to the next TCB**, to build queues (e.g. of READY tasks).

A desktop OS's PCB holds much more: open files, memory maps, CPU time used, parent and children (Linux: `struct task_struct`). In TCB terms, a context switch saves the CPU state, stores the SP in the old task's TCB, takes the SP from the new task's TCB and restores its CPU state. The TCBs live in a **static** structure such as an **array** (simple, but the set of tasks is fixed by its size) or a **dynamic** one such as a **linked list** (tasks added and removed at run time, each TCB pointing to the next); Linux organises almost every data structure with more than one element as a *doubly* linked list (pointers to next and previous).

1.3.6 When does the scheduler run?

A very important point. Take two tasks and a scheduler that can interrupt one and run the other (whatever the policy). It runs at boot, to start the first task; afterwards, specific **events** activate it.

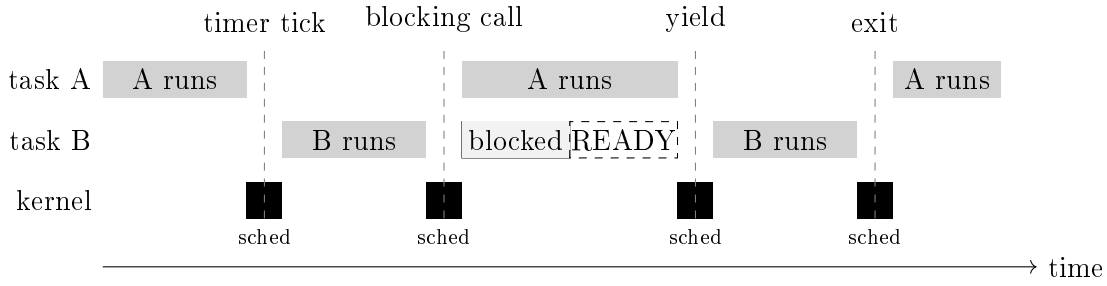


Figure 1.4: The four events that activate the scheduler: A's time slice ends (A: RUNNING → READY); B calls `delay` (B: RUNNING → BLOCKED); A gives up the CPU (A: RUNNING → READY); B finishes (B: RUNNING → FINISHED).

The scheduler is not a task: it is a kernel function, run only when an event calls it. The four **activation points**:

1. **Timer tick**: a periodic hardware timer interrupt (**SysTick** on Cortex-M), in general **high-priority** (on Cortex-M a *lower* number means a *higher* priority).[2] Every x milliseconds, whatever runs is interrupted and the CPU enters the OS's interrupt service routine. This is how the OS gets time for its own work on a single core busy with programs (from a question in class): most of its work reacts to the tick, whose handler is a sequence of function calls, each an OS service; running the scheduler, to check whether another task should run, is one of them.
   - Not mandatory, but the tick is *the* place for a **preemptive** scheduler: even a task that will not release the CPU must give control to the OS (the "brutal" event: the task can do nothing about it).

[2] In the scratch-OS of Unit 2, SysTick shares with PendSV the least urgent priority value used by the kernel, `0xE0`: the reason is explained there.

- The **time slice** (**quantum**) is the longest a task may run before the scheduler is called; when it ends, the task can be preempted (RUNNING → READY).
- The tick also checks the BLOCKED tasks: one whose delay has ended goes BLOCKED → READY.
- It is the most complex activation point to implement: it requires knowing the hardware.

2. **Blocking call**: the running task makes a system call that waits (a delay, data not yet available, I/O to complete): RUNNING → BLOCKED, so another task must be chosen.

3. **Yield**: the running task *voluntarily* gives up the CPU, although it could continue (RUNNING → READY). Many OSs have a `yield` system call, in practice "run the scheduler again", e.g. after a task changed something in the system and wants the scheduler to reassess the best task to run.

4. **Exit**: the running task finishes (RUNNING → FINISHED), and another must be chosen.

Only the tick comes from outside: the other three are **system calls** of the running task. In the figure, B's delay ends while A runs: B is READY again but does **not** run, and the end of the delay does not by itself call the scheduler. A holds the only CPU, so the OS cannot take control before the next activation point (here A's yield; with a preemptive kernel, at the latest the next tick), and only then is it decided whether B runs.

1.3.7 Preemptive vs cooperative

A **preemptive** scheduler can take the CPU from a running task *without its cooperation*, at any instruction, at a tick (or another interrupt), e.g. with the scheduler called in the timer tick. A **cooperative** one leaves it to the tasks to give control back: a task keeps the CPU until it blocks, yields or exits, and no tick switches tasks; e.g. no scheduler in the tick, but a `yield` system call that every task calls at some point. Week 4's scratch-OS is *not* preemptive: the kernel regains control only at `exit`; SysTick is there, but its handler does nothing for scheduling, so A runs to its end, then B. **Interrupts are not preemption**: in a non-preemptive OS an interrupt (an external event) can still interrupt a task, but the scheduling question is whether the *OS* can stop a task that has the CPU and give the CPU to another task; there, it cannot.

- Cooperative is **simpler**: a task is never stopped in the middle of its work, so tasks interfere less.
- But it is **dangerous**: one task looping without blocking or yielding (e.g. stuck in an infinite loop) freezes the whole system, and a long task delays every urgent one.
- Preemptive keeps the system **responsive**, but a task can be stopped between any two instructions: **shared data needs protection**.

Likewise for algorithms: **non-preemptive** ones never take the CPU from a running task, **preemptive** ones do.

1.3.8 Exercise: follow two tasks

No algorithm and no computation: for each event, decide **which task states change** and **whether the scheduler runs**.

Setting. A preemptive kernel gives every task a fixed time slice and runs tasks A and B, both READY at boot. A task that becomes READY waits until the running task's time slice ends. Events, in order:

1. the scheduler picks A;
2. A calls `delay(10 ticks)`;
3. a tick ends B's time slice;
4. A's 10 ticks have passed (the tick that detects it does not end B's slice);
5. B's time slice ends;
6. A calls `exit`.

After each event, write the state of A and B and the transitions it caused.

Method. For every event, ask three questions:

1. **What happens to the running task?** `delay`: RUNNING → BLOCKED. `exit`: RUNNING → FINISHED. End of slice: RUNNING → READY, but *only* if another task is READY.
2. **Does a task wake up?** The end of a delay only moves it BLOCKED → READY: it does not call the scheduler.
3. **Does the scheduler run, and whom does it pick?** After a `delay`, an `exit` or the end of a slice, it picks a READY task (on a tie, the lowest identifier). If no other task is READY, the running task continues.

Solution. The exercise gives no times. The figure assumes a 6-tick slice and 2 ticks of work before each call; only the order of the events matters.

1. At boot A and B tie, so the **lowest identifier** runs (Section 1.5.1). A: READY → RUNNING.
2. `delay` blocks A (RUNNING → BLOCKED), and the only READY task, B, starts (READY → RUNNING).
3. **No transition**. A is BLOCKED, so B gets a new slice and goes on. Preemption needs another READY task, and the CPU does not idle waiting for A (implementing this correctly is not as easy as it sounds: see Unit 2).

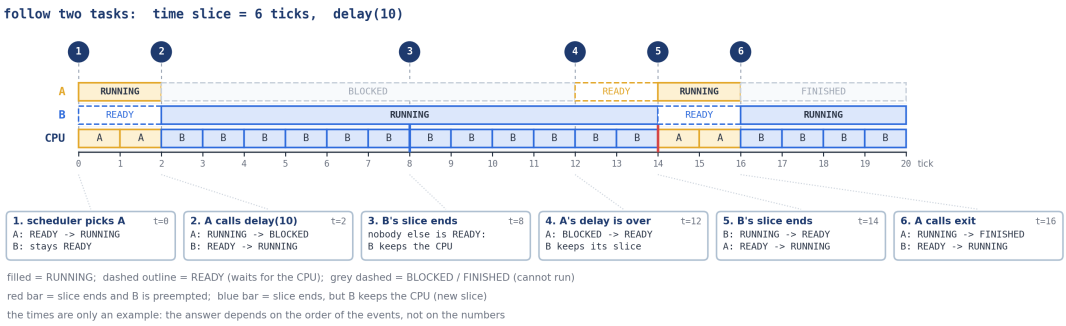


Figure 1.5: The six events on a time axis, with the transitions of each event. At event 3 nobody else is READY, so B keeps the CPU (blue bar). At event 4 A only becomes READY. At event 5 B is preempted (red bar).

4. A goes BLOCKED → READY, **not** RUNNING. B keeps its slice to the end, even though it has already had two slices in a row.

5. Now A is READY, so B is preempted (RUNNING → READY) and A runs (READY → RUNNING).

6. A goes RUNNING → FINISHED, and B goes READY → RUNNING.

The two traps: the end of a slice is not always a preemption (event 3), and READY is not RUNNING (event 4). If A and B were both BLOCKED, nothing could run, yet the CPU must execute something: Unit 2's answer is the *idle task*.

1.4 Scheduling criteria

Policies are compared with **metrics**, which **conflict** (no algorithm is the best at all of them), so algorithms are usually compared on **averages** over all tasks. A task **arrives** (becomes READY) at a, needs C units of CPU, **starts** for the first time at s and **finishes** at f.

- **CPU utilisation** (maximise): the fraction of time the CPU actually runs tasks; time in the scheduler, in other OS code or idle lowers it. The ideal 100% is never reached: the closer, the better.

- **Throughput** (maximise): tasks finished per unit of time; the goal when finishing many tasks globally matters, even if one task may take long.

- **Turnaround time** (minimise): turnaround $= f - a$, from the request to the end (e.g. from the double click on a program's icon to the end of its execution): the single task's viewpoint, in a sense the opposite of throughput.

- **Waiting time** (minimise): waiting $= f - a - C$, the turnaround minus the time actually run: the time spent READY (an arrived task that is not running is waiting). E.g. a task that needs 1 ms of CPU but ends 10 ms after arriving has waited 9 ms, which is not good.

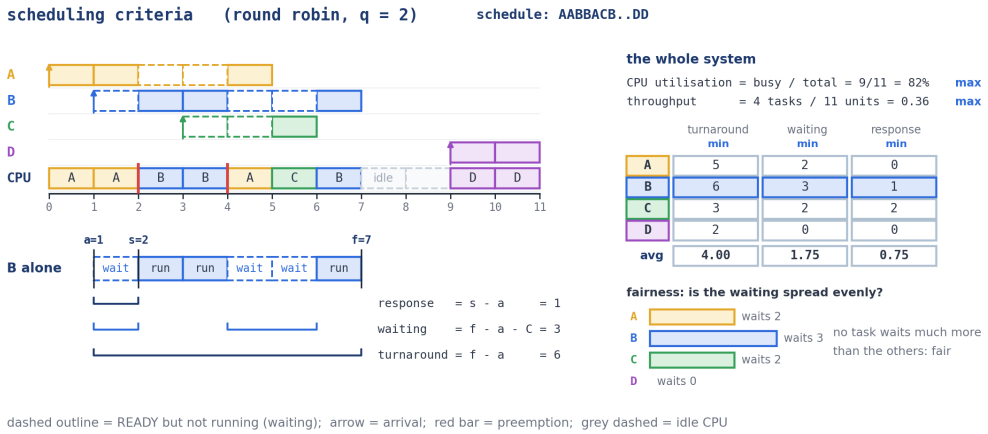


Figure 1.6: The scheduling criteria on one schedule (round robin, $q = 2$; A: $a = 0, C = 3$; B: $a = 1, C = 3$; C: $a = 3, C = 1$; D: $a = 9, C = 2$). Left: task B alone, from arrival a to first start s to finish f. Its response is $s - a$, its turnaround is $f - a$, and its waiting time is the time spent READY, i.e. $f - a - C$. Right: the whole system. Utilisation and throughput come from the CPU row (2 idle slots out of 11). The per-task metrics are compared through their averages. Fairness asks whether the waiting is spread evenly among the tasks.

- **Response time** (minimise): response $= s - a$, from arrival to the first time on the CPU. Very important in general-purpose OSs with a graphical interface: a long response time is what the user perceives as "lagging". **Careful**: real-time systems (Unit 3) call $f - a$ the response time.

- **Fairness**: each task's share of the CPU: a fair scheduler gives every task a similar amount, and no task waits forever.

1.5 Scheduling algorithms

This unit covers **non-real-time** scheduling; real-time scheduling starts in Unit 3 and continues next week.

1.5.1 Exercise conventions

Scheduling basically means drawing charts; the notation and rules below are those of class **and of the exam**.

- A task is one piece of work: it arrives at a, needs C units of CPU, then finishes. Time is **discrete**, in whole **time units** (milliseconds or microseconds: it does not matter).

- A **Gantt chart** has one row per task and one slot per unit, with `#` where the task runs. The slides use them, but drawing them at the exam takes too long, so there the schedule is a **string**, one letter per slot (`AABBC`: A in slots 0 and 1, B in 2 and 3, C in 4), with a dot `.` for an idle CPU.

- Slot t covers the time from t to $t+1$: a task running in slots 0 to 5 finishes at **6**, not 5 (the most common mistake in these exercises).

- The cost of a context switch is ignored (until the overhead discussion later in this unit).

Tie rules. Two tasks are often equally eligible; these conventions make every solution **deterministic** (no ambiguity from different assumptions), in all exercises **and in the exam**:

- On a **tie** (equally good for the algorithm: same arrival, same priority, ...) the **running task keeps the CPU**, because a context switch has a cost.

- If no tied task is running, the **lowest identifier** runs first (A before B): irrelevant to the scheduling, only a convention that gives every exercise exactly *one* solution (a real scheduler could break ties in any way, even randomly).

- In priority scheduling a **higher number means a higher priority** (some OSs use the opposite convention, so it must be fixed).

- In round robin, a task that **arrives at the instant a quantum expires** joins the queue **before** the preempted task (e.g. $t = 2$ in the round-robin worked example).

The exam text repeats the rules, so there is no need to memorise them all; but when a solution is unclear, the answer is always in the rules.

Exam. At least one exam exercise is on scheduling. The exercises come in a **text file**, with the answers in a fixed format and schedules in string notation, because a program reads the file and checks the answers (format and tool, `schedsim`: Unit 3); there is also space for the calculations, so that the professor can see whether a wrong answer was only a calculation error. A typical question asks for the schedule, or for a value such as the average waiting time. These exercises are very simple but must be practised (the slides contain many solved ones): exam time is short, there is more than one exercise, and 20 minutes on one of them is too much (it happens even to students who studied).

1.5.2 First-Come, First-Served (FCFS)

The simplest policy, the one of week 4: the tasks run one after the other. READY tasks wait in a **queue** in arrival order; each time it runs, the scheduler takes the head and runs it until it finishes (or blocks). FCFS is **non-preemptive** (cooperative: a later task never interrupts the running one), very simple (one queue, no timer) and fair in one sense (tasks are served in the order they came). **Big problem**: the waiting times depend heavily on the arrival order, and the same tasks in another order can wait very differently.

Example 1: a long task first. A typical exercise gives each task's arrival time and the units it needs; in FCFS the arrival time decides. A arrives at 0 and runs first; B, READY at 1, waits for A to complete; then B runs, then C. Schedule: `AAAAAABBC`.

task	a	C	start	finish	turnaround	waiting	response
A	0	6	0	6	6	0	0
B	1	2	6	8	7	5	5
C	2	1	8	9	7	6	6
				average	6.67	3.67	3.67

```
time  0        9
     |........|
A      ######...
B      ......##.
C      ........#
CPU    AAAAAABBC
```

A finds the CPU free and runs from 0 to 6: turnaround $6-0=6$, waiting $6-0-6=0$, response $0-0=0$. B: $8-1=7$, $7-2=5$, $6-1=5$. C: $9-2=7$, $7-1=6$, $8-2=6$. Averages: turnaround $(6+7+7)/3=6.67$, waiting and response $(0+5+6)/3=3.67$. The two short tasks wait 5 and 6 units behind the long one, to run for only 2 and 1.

Example 2: the long task last. Same tasks, but the long one arrives last (C: $a=0, C=1$; B: $a=1, C=2$; A: $a=2, C=6$). Schedule: `CBBAAAAAA`.

task	a	C	start	finish	turnaround	waiting	response
C	0	1	0	1	1	0	0
B	1	2	1	3	2	0	0
A	2	6	3	9	7	1	1
				average	3.33	0.33	0.33

```
time  0        9
     |........|
A      ...######
B      .##......
C      #........
CPU    CBBAAAAAA
```

Same work and same end (9), but the average waiting time drops from 3.67 to 0.33. Short tasks stuck behind a long one: the **convoy effect**, a really bad behaviour and the reason why general-purpose systems do not use FCFS; but FCFS is simple, which is why week 4 used it.

Exercise: FCFS. Four tasks: A ($a=0, C=4$), B ($a=2, C=3$), C ($a=3, C=1$), D ($a=9, C=2$). Draw the Gantt chart and the schedule string; compute turnaround, waiting and response time of each task and their averages; compute CPU utilisation and throughput.

Solution. Schedule: `AAAABBBC.DD`.

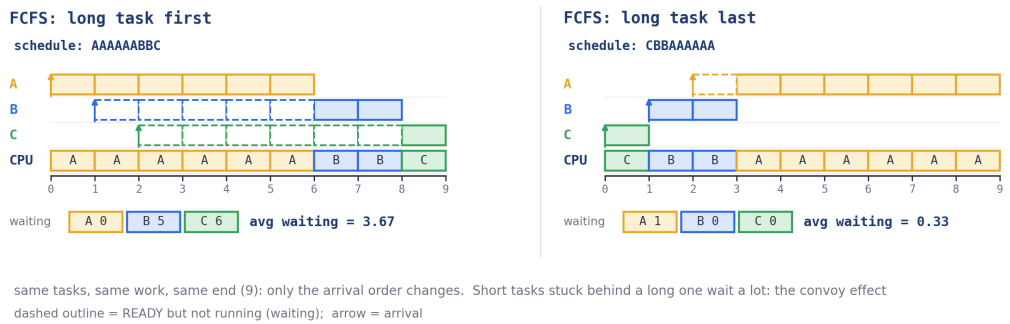


Figure 1.7: The convoy effect in FCFS. The same three tasks in two arrival orders: same work and same end, but the average waiting time drops from 3.67 to 0.33 when the long task arrives last.

task	a	C	start	finish	turnaround	waiting	response
A	0	4	0	4	4	0	0
B	2	3	4	7	5	2	2
C	3	1	7	8	5	4	4
D	9	2	9	11	2	0	0
				average	4.00	1.50	1.50

```
time  0          10
      |.........|.
A       ####.......
B       ....###....
C       .......#...
D       .........##
CPU     AAAABBBC.DD
```

Slot 8 is idle: C has finished and D arrives only at 9. Utilisation = 10 busy slots / 11 $\approx$ 91%; throughput = 4 tasks in 11 units. In FCFS, **waiting time** = **response time**: once started, a task never waits again.

1.5.3 Round Robin (RR)

The policy of the scratch-OS. It fixes the convoy effect by giving every task a turn of limited length, the **time slice** (quantum q, its important parameter), so a long task cannot keep the CPU while short ones wait. It is **preemptive**: it runs in the system tick, at fixed time slices, and the tick takes the CPU away. All tasks are equal: none is more important than another.

- READY tasks wait in a **FIFO queue**; a task that arrives (or wakes up) joins it at the **tail**.
- The head runs for at most one quantum (one or more ticks). If the quantum ends first, the task is **preempted** to the tail and the new head runs (A's quantum expires: A goes to the tail, B, the head, runs), circularly.
- A task that finishes or blocks before its quantum ends is replaced at once by the next head, with a fresh quantum.

- With n READY tasks, a task waits at most $(n-1) \times q$ before running again: the **response time is bounded**.

Worked example, $q = 2$**.** Schedule: `AABBCAADDBA`.

task	a	C	start	finish	turnaround	waiting	response
A	0	5	0	11	11	6	0
B	1	3	2	10	9	6	1
C	2	1	4	5	3	2	2
D	4	2	7	9	5	3	3
				average	7.00	4.25	1.50

```
time  0          10
     |.........|.
A      ##...##...#
B      ..##.....#.
C      ....#......
D      .......##..
CPU    AABBCAADDBA
```

The queue (head on the left) at each decision:

- $t = 0$: only A has arrived, and runs slots 0 and 1; B arrives at 1 and joins the queue.
- $t = 2$: A's quantum expires as C arrives, with B already waiting: both go to the tail, C first (tie rule). Queue `B C A`: B runs.
- $t = 4$: B's quantum expires as D arrives: D joins before B. Queue `C A D B`: C runs.
- $t = 5$: C needed only 1 unit, so the next head runs at once: queue `A D B`, A runs slots 5 and 6.
- $t = 7$: A goes to the tail: `D B A`. D runs slots 7 and 8 and finishes.
- $t = 9$: `B A`. B runs its last unit (slot 9); A runs its last one (slot 10) and finishes at 11.

The numbers (turnaround, waiting, response): A $11-0=11$, $11-5=6$, 0; B $10-1=9$, $9-3=6$, $2-1=1$; C $5-2=3$, 2, 2; D $9-4=5$, 3, 3.

1.5.4 Exercise: round robin, $q = 2$

Four tasks: A ($a=0, C=4$), B ($a=0, C=3$), C ($a=1, C=2$), D ($a=3, C=1$). Which of A and B runs first, and why? Write the queue after every scheduling decision, draw the Gantt chart and the schedule string, and compute turnaround, waiting and response times with their averages.

Solution. Schedule: `AABBCCAADB`.

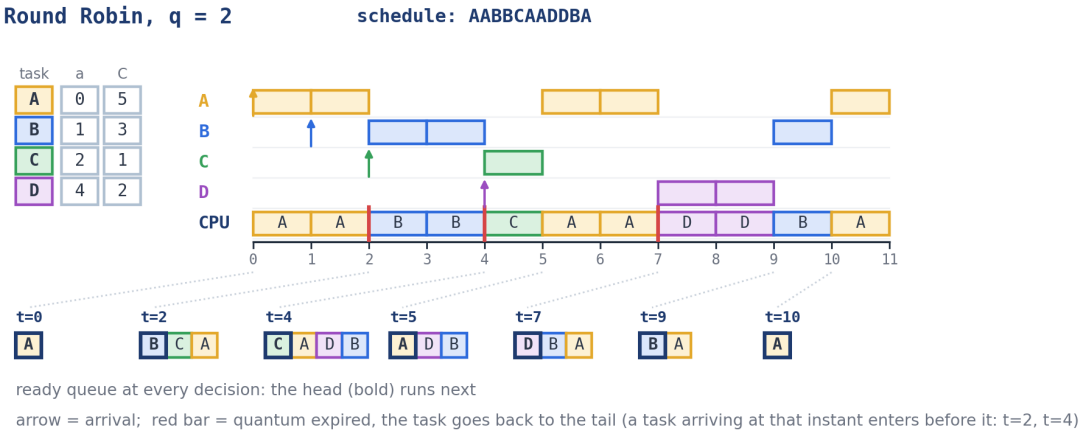


Figure 1.8: Round robin worked example ($q = 2$) with the ready queue at every scheduling decision. At $t = 2$ and $t = 4$ a task arrives exactly when a quantum expires, so it joins the queue before the preempted task.

task	a	C	start	finish	turnaround	waiting	response
A	0	4	0	8	8	4	0
B	0	3	2	10	10	7	2
C	1	2	4	6	5	3	3
D	3	1	8	9	6	5	5
				average	7.25	4.75	2.50

```
time  0          10
     |.........|
A      ##....##..
B      ..##.....#
C      ....##....
D      ........#.
CPU    AABBCCAADB
```

Queue: $t = 0$ `A B`; $t = 2$ `B C A`; $t = 4$ `C A D B`; $t = 6$ `A D B`; $t = 8$ `D B`; $t = 9$ `B`.

- **A runs first**: A and B tie on arrival and nothing runs yet, so the lowest identifier wins.
- $t = 2$: C (arrived at 1) is already behind B when A is preempted and goes to the tail.
- $t = 4$: D (arrived at 3) is *already* queued when B is preempted: the "same instant" rule is not needed.
- $t = 6$ and $t = 8$: C and A finish exactly at the end of a quantum, so they do not go back to the queue.
- $t = 9$: D needed 1 unit and finished; B runs its last unit.

1.5.5 Exercise: change the quantum

Same four tasks (A 0/4, B 0/3, C 1/2, D 3/1), scheduled with $q = 1$ and $q = 4$. Draw both Gantt charts, compute the averages, and count the **context switches** (the times the running task changes from one slot to the next).

$q = 1$. Schedule: `ABCABDCABA`.

task	a	C	start	finish	turnaround	waiting	response
A	0	4	0	10	10	6	0
B	0	3	1	9	9	6	1
C	1	2	2	7	6	4	1
D	3	1	5	6	3	2	2
				average	7.00	4.50	1.00

```
time  0         10
     |.........|
A      #..#...#.#
B      .#..#...#.
C      ..#...#...
D      .....#....
CPU    ABCABDCABA
```

The running task changes after every slot: 9 context switches in 10 units. Every task starts quickly (average response 1.00).

$q = 4$. Schedule: `AAAABBBCCD`.

task	a	C	start	finish	turnaround	waiting	response
A	0	4	0	4	4	0	0
B	0	3	4	7	7	4	4
C	1	2	7	9	8	6	6
D	3	1	9	10	7	6	6
				average	6.50	4.00	4.00

```
time  0         10
     |.........|
A      ####......
B      ....###...
C      .......##.
D      .........#
CPU    AAAABBBCCD
```

No task needs more than 4 units, so no quantum ever expires: this is exactly FCFS. Only 3 context switches.

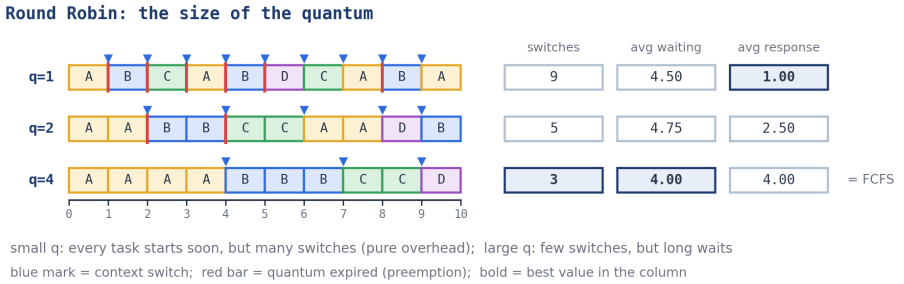


Figure 1.9: The same four tasks with three quanta. A small q gives the best response but 9 context switches. A large q gives few switches and the best waiting time, but it is exactly FCFS. No quantum is best for every metric.

1.5.6 Quantum size and context-switch overhead

quantum	switches	avg turnaround	avg waiting	avg response
$q = 1$	9	7.00	4.50	1.00
$q = 2$	5	7.25	4.75	2.50
$q = 4$	3	6.50	4.00	4.00

$q = 1$ has the best average response but the most switches; $q = 4$ the fewest switches and the best average waiting and turnaround, but it is exactly FCFS, with the worst response. No quantum is best for everything: a **small** one gives every task the CPU soon (good response time) at the cost of many context switches; a **large** one has few switches, but a task may wait long for its turn (with q longer than every task, round robin becomes FCFS). Typical choice: a few milliseconds, much longer than one context switch, but short enough for interactive tasks to feel immediate.

The algorithms seen so far are high-level policies, some more efficient than others; what really takes time in practice is the **context switch**, which does no useful work: **pure overhead**. It costs saving and loading the registers, running the scheduler and, on our kernel, reprogramming the MPU, plus hidden costs: the caches and the TLB (week 4) hold the other task's data, so the new task runs slower at first. If a switch takes s and every quantum q ends with one (all tasks always READY, as in round robin), each cycle lasts $q + s$: q of real work, s lost. The overhead is the ratio between switch time and total time:

$$\text{overhead} = \frac{s}{q + s}$$

Examples:

$$\begin{array}{llll} s = 10\,\mu s, & q = 1\,ms & \to\ 10/1010 & \approx 0.99\% \\ s = 10\,\mu s, & q = 100\,\mu s & \to\ 10/110 & \approx 9.1\% \\ s = 10\,\mu s, & q = 10\,\mu s & \to\ 10/20 & = 50\% \end{array}$$

So round robin's only parameter, the quantum, can completely destroy the efficiency of the system: it must be much larger than s. Yet a very large quantum, cheap in overhead, makes every task wait long for the CPU: counting the switches, with n tasks a task may wait $(n-1) \times (q+s)$ for its turn. It is always a **trade-off**: gaining on one side means paying on the other.

Exercise: overhead. A context switch takes $s = 20\,\mu s$; all tasks are always READY.

1. Compute the overhead for $q = 80\,\mu s$, $q = 1\,ms$ and $q = 10\,ms$.

2. What is the smallest quantum that keeps the overhead at or below 1%?

3. Five tasks share the CPU with $q = 10\,ms$. How long can a task wait, at most, between two of its quanta?

Solution.

1. $q = 80\,\mu s \to 20/100 = 20\%$; $q = 1\,ms \to 20/1020 \approx 1.96\%$; $q = 10\,ms \to 20/10020 \approx 0.20\%$.

2. $\dfrac{20}{q+20} \leq 0.01 \;\Rightarrow\; q + 20 \geq 2000 \;\Rightarrow\; q \geq 1980\,\mu s$ (about 2 ms).

3. The other 4 tasks run first: $4 \times (10\,ms + 20\,\mu s) = 40.08\,ms$.

Going from 1 ms to 10 ms saves less than 2% of the CPU, but makes the worst wait ten times longer (about 4 ms $\to$ 40 ms). This is the trade-off in numbers: overhead pushes q up, response time pushes it down.

1.5.7 Priority-based scheduling

Other OSs, and importantly **FreeRTOS**, use a **priority-based** scheduler: in round robin all tasks are equal, but reading a sensor may matter more than blinking an LED.

- Each task gets a **priority**, a number (course rule: higher number = higher priority). The kernel keeps **one ready list per priority** instead of a single one, and always runs the READY task with the **highest priority**.

- **Preemptive**: a task with a strictly higher priority that becomes READY takes the CPU at once; one with the same priority does not (tie rule: the running task keeps the CPU).

- The running task also loses the CPU when it blocks or exits; then the highest-priority READY task runs.

- **Static** priorities are fixed when the task is created; **dynamic** ones are changed by the kernel at run time (e.g. aging, below).

There are many refinements, but always the task with the highest priority runs. The main FreeRTOS scheduler works exactly this way, with round robin among READY tasks of the same priority: a combination of the two schedulers.

Worked example. The exercises now give each task's arrival time, execution time and priority. Schedule: `ABBCCDAAA`.

task	a	C	prio	start	finish	turnaround	waiting	response
A	0	4	1	0	9	9	5	0
B	1	2	3	1	3	2	0	0
C	2	2	3	3	5	3	1	1
D	3	1	2	5	6	3	2	2
					average	4.25	2.00	0.75

```
time  0        9
    |........|
A     #.....###
B     .##......
C     ...##....
D     .....#...
CPU   ABBCCDAAA
```

- $t = 0$: only A is there, so it runs (it needs 4 units, but its priority is only 1).
- $t = 1$: B (priority 3) arrives and beats A (1): A is preempted, B runs.
- $t = 2$: C (3) ties with B, which is running: B keeps the CPU.
- $t = 3$: B finishes and C runs; D (2) arrives, but is lower than C.
- $t = 5$: C finishes; D (2) beats A (1) and runs.
- $t = 6$: only A is left: its last 3 units end at 9.

The priorities drive the schedule: A runs only when nobody else is READY.

1.5.8 Starvation and aging

A low-priority task runs *only* when no higher-priority task is READY: if higher-priority work keeps arriving, it may never get the CPU. This is **starvation**, impossible in FCFS (every task eventually reaches the head of the queue) and in round robin (every task gets a quantum in each round), but possible with priorities. **Aging**, based on dynamic priority, fixes it and makes the scheduler fairer: the longer a task waits READY, the more its priority grows, until it is the highest and runs; then its priority goes back to its initial value. In a real-time system starvation is often a **design error**: the high-priority tasks must leave enough CPU time to the others (Unit 3 shows how to count the CPU time tasks need).

Exercise: priority and starvation. Six tasks, preemptive priority scheduling (higher number = higher priority): A ($a = 0, C = 3$, prio 3), L ($a = 0, C = 2$, prio 1), B ($a = 2, C = 3$, prio 2), C ($a = 5, C = 2$, prio 3), D ($a = 7, C = 3$, prio 2), E ($a = 10, C = 2$, prio 3). Draw the Gantt chart and the schedule string; who runs at $t = 7$, and why? Compute each task's waiting time and the average waiting and turnaround time. What happens to L if a new task of priority 2 or 3 keeps arriving every 3 units, forever?

Solution. Schedule: `AAABBCCBDDEEDLL`.

```
time  0          10
     |.........|....
A      ###............
B      ...##..#.......
C      .....##........
D      ........##..#..
E      ..........##...
L      ............##
CPU    AAABBCCBDDEEDLL
```

task	A	L	B	C	D	E	average
waiting	0	13	3	0	3	0	$19/6 \approx 3.17$
turnaround	3	15	6	2	6	2	$34/6 \approx 5.67$

Step by step:

- $t = 0$: A (3) and L (1) arrive; A runs, slots 0 to 2.
- $t = 3$: A has finished. B (2, arrived at 2) beats L (1): B runs.
- $t = 5$: C (3) arrives and preempts B (2). C runs slots 5 and 6.
- $t = 7$: C has finished. B (2, with 1 unit left) and D (2, just arrived) are tied on priority 2, and neither is running, so the lowest identifier, **B**, runs. B finishes at 8.
- $t = 8$: D runs. At $t = 10$, E (3) arrives and preempts D: E runs slots 10 and 11; then D finishes its last unit in slot 12.
- $t = 13$: only L is left. It runs slots 13 and 14 and finishes at 15.

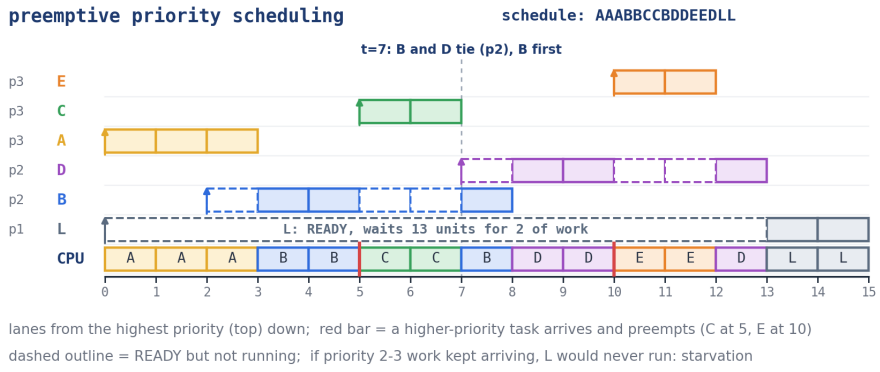


Figure 1.10: Preemptive priority scheduling, with lanes ordered from the highest priority down. C and E preempt lower-priority tasks as soon as they arrive. At $t = 7$, B and D tie on priority 2, so the lowest identifier runs. L, with the lowest priority, waits 13 units for 2 units of work: with a steady flow of higher-priority work it would starve.

L arrives at 0 but runs only at 13: 13 units of waiting for 2 of work. If higher-priority tasks kept arriving forever, L would **never** run (starvation); with aging, its priority would grow while it waits, until it beats the others and runs.

1.6 Unit 1: summary

- A task is a program in execution with its own stack and CPU state; the kernel keeps a TCB per task (saved SP, state, scheduling information).
- A task is READY, RUNNING, BLOCKED or FINISHED; the kernel moves it at a tick, a blocking call, a yield or an exit. Real OSs have more states (Linux has seven), but the idea is the same.
- Preemptive schedulers can take the CPU away, cooperative ones wait for the task to give it up; no scheduler of a real OS used today is non-preemptive.
- FCFS is simple but suffers the convoy effect; round robin bounds the response time, and its quantum trades response time against the switch overhead $s/(q+s)$.
- Priority scheduling serves important tasks first but can starve the others; aging is a fix.
- The tie rules (running task keeps the CPU, then lowest identifier; higher number = higher priority; in RR an arrival goes before the preempted task) hold in all exercises and in the exam.

Are these algorithms used in practice? They are the simple textbook algorithms, but not far from reality: the Linux scheduler uses more complex rules, not so different from these. **FCFS** is still used in some small and mid-size devices; **priority** scheduling is everywhere: every car, train and airplane, and real-time systems in general; all the other systems run **round robin**, with many add-ons for efficiency. Behind any of them, the really complex mechanism is the context switch, implemented on the Cortex-M3 in Unit 2.

1.7 Unità 2 - Context switch and round robin in the scratch-OS

This unit adds a **round-robin scheduler** with a real **context switch** to the scratch-OS. Tasks A and B, fixed in number, do not change; but instead of week 4's deterministic order (B started when A called `exit`, the only scheduling decision), they take turns as in Unit 1: a quantum is defined, and each task runs one time slice before the other gets the CPU.

Link with the project. This code is the **base of the course project**: the "project base" module at the top of the project page is essentially this unit's end point, plus a few mechanisms already implemented so that the project need not write them. The project keeps this scheduler and memory management, and changes only how a task starts: tasks become **program files stored in FLASH** instead of C functions linked into the image, so when a task is started (and only then) the OS locates its program in FLASH and copies it to memory; from there on it works as now. So it is really important to study this week's code.

The code of this week. `week-05/` has two sibling folders, `base/` and `solution/`, each with its own `Makefile` and building on its own; the base's `README.md` explains how it was prepared from week 4's finished code, what is given and what is written in class.

- `base/` is week 4's solution plus a **complete**, ready preemptive round-robin scheduler: it builds and runs from the first minute, A and B take turns, and an idle task runs when neither can. The OS is now too complex to write everything in class, so everything long to type and better read than written is in place; the slides follow one path through it (how a switch works, how the next task is chosen, what the tick does) in the "Inspect" slides, observed with `gdb` and the Makefile options `QUANTUM` and `DEMO`.

- The base cannot show **who used the CPU when**: its `kernel/trace.c` is an empty skeleton, and `task_switch_context` has no trace call. `solution/`, commented line by line, adds the **trace**, which records every switch and prints the schedule as a Gantt chart: shown running in class, written by you at home in the base following these notes (record type, ring buffer, recording and printing functions, Gantt chart, one call in `task_switch_context`), until your `kernel/trace.c` equals the solution's. If notes and code ever disagree, the code is the reference.

- The notes cover more than the slides: a second gdb session, the rogue and cooperative demos, the delay services and the idle task in code, and the trace step by step.

- Paths are relative to `week-05/solution/` unless stated otherwise. QEMU now runs with its serial port on the terminal, so it is stopped with **Ctrl-C**.

Files changed since week 4:

- `kernel/task.h`, `kernel/task.c`: `struct tcb`, `enum task_state`, `QUANTUM`, `tasks`, `tcb_current`, `tick_count` and the scheduler functions `task_init_stack`, `task_start_first`, `task_request_switch`, `task_increment_tick`, `task_block_until`, `task_finish_current`, `task_switch_context`;

- `kernel/pendsv.s`: `PendSV_Handler`, the context switch;

- `drivers/systick.c`: `SysTick_Handler` calls `task_increment_tick`, so the tick now drives the scheduler; `boot/startup.s`: PendSV and SysTick at priority `0xE0`;

- new system calls (yield, delay, delay until, tick count, trace dump): stubs `task_yield`, `task_delay`, `task_delay_until`, `task_tick_count`, `trace_dump` and service numbers 6 to 10 in `lib/syscalls.h` and `lib/syscalls.s`; services `sys_yield_impl`, `sys_delay_impl`, `sys_delay_until_impl`, `sys_exit_impl`, `sys_tick_count_impl`, `sys_trace_dump_impl` in `kernel/syscall.c`;

- `src/idle.c`, `scripts/mps2_m3.ld`: `idle_task` and its partition; `lib/printf.c`: `spin_ticks`, the tasks' busy work;

- `src/main.c`, `src/task_a.c`, `src/task_b.c`: the start of the tasks, the tasks and their `coop` and `idle` demo blocks; `Makefile`: `QUANTUM` and the two new demos;

- `kernel/trace.h`, `kernel/trace.c` (complete only in the solution): `trace_record_switch`, `trace_print` and their helpers.

command	what changes	what to observe
`make run`	default, quantum 5	A and B alternate every 5 ticks; B alone after 32
`make QUANTUM=2 run`	2-tick slice	one step per turn, A ends at 40, 21 switches
`make DEMO=rogue run`	B writes a kernel variable	MemManage after a switch: isolation holds
`make DEMO=coop run`	`QUANTUM=0`, no yield	B never runs: starvation
`make DEMO=idle run`	delays	A drifts, B does not, idle fills the gaps
`make debug` + gdb	breakpoints in PendSV	hardware frame, `sp`, MPU registers

1.7.1 The folder `week-05/base`

This is the code we start from: the scheduler is already in place, and only the trace is left to write. Files in bold are new or changed since week 4.

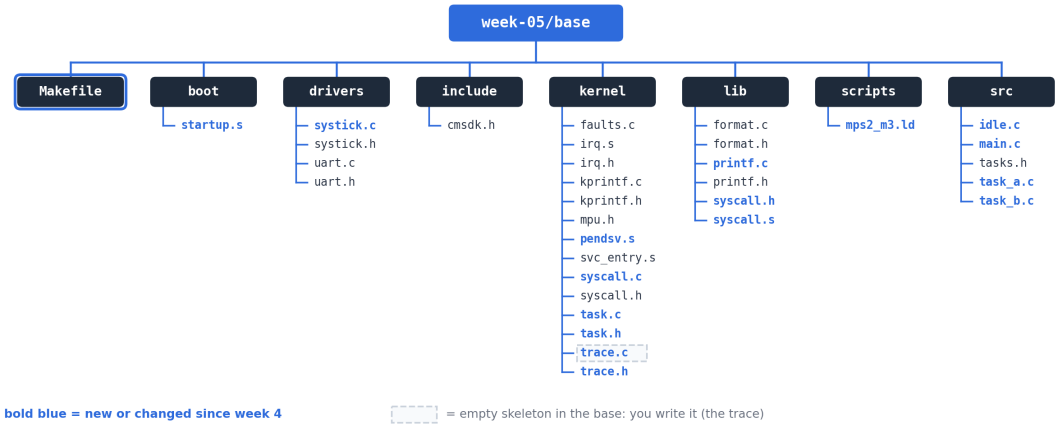


Figure 1.11: The folder `week-05/base`. Files new or changed since week 4 are in bold blue; `kernel/trace.c` is an empty skeleton, written in class and at home.

file	what it is
`Makefile`	build, `make run`, `make debug`; options `QUANTUM` and `DEMO` (`rogue`, `coop`, `idle`), each in its own build folder
`boot/`	
`startup.s`	vector table and `Reset_Handler`: sets the exception priorities (faults `0x00`, SVCall `0x80`, SysTick and PendSV `0xE0`), then `main`
`drivers/`	
`systick.c`, `systick.h`	the 10 ms tick; `SysTick_Handler` now also calls `task_increment_tick`
`uart.c`, `uart.h`	serial port: the text printed on the terminal (interrupt at `0xC0`)
`include/`	
`cmsdk.h`	addresses and registers of the board's hardware (SCB, SysTick, UART, MPU)
`kernel/`	
`faults.c`	fault handlers: MemManage prints `CFSR` and the address, then stops the system
`irq.s`, `irq.h`	low-level interrupt routines in assembly, and their declarations
`kprintf.c`, `kprintf.h`	the kernel's privileged `printf`, used inside handlers (e.g. `trace_print`)
`mpu.h`	MPU interface: regions and `mpu_set_task_region`
`pendsv.s`	`PendSV_Handler`: the context switch, 13 instructions
`svc_entry.s`	SVCall entry: takes the caller's frame and passes it to the C dispatcher
`syscall.c`, `syscall.h`	kernel side of the system calls: the `sys_*_impl` services (yield, delay, delay until, exit, tick count, trace dump)
`task.c`, `task.h`	the scheduler: `struct tcb`, `tasks[]`, `tcb_current`, `tick_count`, `QUANTUM` and the `task_*` functions
`trace.c`, `trace.h`	the trace: ring buffer, recording, printing, Gantt chart (`trace.c` is a skeleton: to write)
`lib/` (task side, unprivileged)	
`format.c`, `format.h`	formatting of numbers and strings for `printf`
`printf.c`, `printf.h`	the tasks' `printf` (through a system call) and `spin_ticks`, the busy work
`syscall.h`, `syscall.s`	service numbers and `svc` stubs: `task_yield`, `task_delay`, `task_delay_until`, `task_tick_count`, `trace_dump`, exit
`scripts/`	
`mps2_m3.ld`	linker script: FLASH, three 4 KB partitions (A, B, idle), kernel RAM
`src/` (the application)	
`idle.c`	`idle_task`: `wfi` forever
`main.c`	starts SysTick, then the tasks (`svc #5`)
`tasks.h`	declarations of the task functions
`task_a.c`, `task_b.c`	the two tasks: busy steps, plus the demo blocks

1.7.2 Running the two tasks

Each task does **busy work** in **steps** of 2 ticks (a tick is 10 ms, as in week 3) and prints the tick number after each step: A does 10 steps, B 15. The busy work, `spin_ticks` (`lib/printf.c`), reads the tick count with the system call `task_tick_count` and loops until the given ticks have passed, never blocking or yielding: it keeps the

CPU until the kernel takes it away. In the normal build, `task_a` (`src/task_a.c`) and `task_b` (`src/task_b.c`) are a `for` loop of `spin_ticks( WORK_TICKS )` and `printf`.

```
cd week-05/base
make run
```

`make run` builds, prints the QEMU command line and starts QEMU, which keeps running after `all tasks finished` (because of the idle task, see later): quit with `Ctrl-C`.

```
scratch-os: kernel started
scheduler: round robin, quantum 5
A: hello from task A
A: step 1 (tick 2)
A: step 2 (tick 4)
B: hello from task B
B: step 1 (tick 7)
B: step 2 (tick 9)
A: step 3 (tick 10)
A: step 4 (tick 12)
A: step 5 (tick 14)
B: step 3 (tick 15)
B: step 4 (tick 17)
B: step 5 (tick 19)
A: step 6 (tick 20)
A: step 7 (tick 22)
A: step 8 (tick 24)
B: step 6 (tick 25)
B: step 7 (tick 27)
B: step 8 (tick 29)
A: step 9 (tick 30)
A: step 10 (tick 32)
task A exited (tick 32)
B: step 9 (tick 32)
B: step 10 (tick 34)
B: step 11 (tick 36)
B: step 12 (tick 38)
B: step 13 (tick 40)
B: step 14 (tick 42)
B: step 15 (tick 44)
task B exited (tick 44)
all tasks finished
```

A and B alternate every **5 ticks**, the quantum (A 0–5, B 5–10, A 10–15, ...), each stopped mid-step: A at 5 during step 3 (started at 4, after steps 1 and 2 ended at 2 and 4), B at 10 during its step 3 (steps 1 and 2 ended at 7 and 9). Back at 10, A prints `A: step 3 (tick 10)` **at once**: the busy work counts **clock** ticks, not its own CPU time, and since step 3 started at 4, more than 2 ticks have passed, so the loop in `spin_ticks` ends at its first test. At 32 A finishes its 10th step and returns (`task A exited (tick 32)`); B, the only application task left, runs alone until 44. This is the round-robin behaviour we want.

1.7.3 What must be saved to pause a task

Round robin first needs the **context switch**: save the running task, restore another; low-level again. To resume A as if nothing happened, the kernel keeps everything A's next instruction may use:

- **the registers** `r0`–`r12`, SP, `lr`, `pc` (where A stopped) and `xPSR` (e.g. the flags of A's last comparison), all overwritten by the next task. The SP is special: a task runs on **PSP** (week 4), so if the others are pushed on A's own stack, remembering A's PSP is enough to find them;
- **the stack** itself needs nothing: in a correctly designed OS every task has its own (A's in A's partition, B's in B's), and the MPU protects it, so no other task can corrupt it while A is paused;

- **the memory rights**: the MPU must be reprogrammed, since region 1, which gives A its partition, must now describe B's.

Exception entry: what the CPU saves, and what it does not. Everything in an OS is started by an exception: whatever the reason to schedule (SysTick, a system call, ...), the CPU ends up in a handler. On exception entry (week 3, exploited in week 4) the CPU pushes the 8-word **hardware frame** (`r0`–`r3`, `r12`, `lr`, `pc`, `xPSR`) on the stack in use, and pops it on return; for an interrupted task that stack is PSP, so the frame lands on the task's own stack, in its partition: the hardware does half of the switch. It does **not** save `r4`–`r11`: the C calling rules (the ARM procedure call standard) make every function give them back unchanged to its caller, so a C handler saves and restores those it uses, which is enough for an ordinary interrupt. A switch, however, returns into *another* task, which expects its own `r4`–`r11`: the kernel must save the old ones and load the new ones **by software**. It pushes them on the old task's stack just below the hardware frame, so the task's whole CPU state is on its stack, and its saved SP points at the lowest saved word.

1.7.4 A switch from A to B: the two stacks

Each task's stack holds the hardware frame (saved by the CPU) and the registers saved by the kernel; each task's TCB holds its stack pointer.

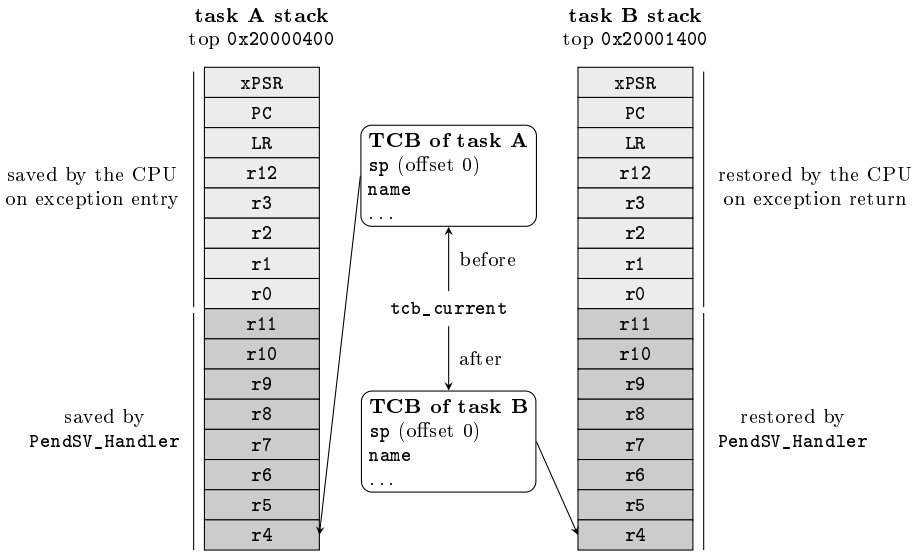


Figure 1.12: A switch from A to B. Each TCB's `sp` points at the lowest saved word (`r4`) of its task's stack. `tcb_current` points at the TCB of the task that runs: the switch saves A, moves `tcb_current` to B, and restores B. At the end, PSP points at B's hardware frame.

what	saved by	where	restored
`r0`–`r3`, `r12`, `lr`, `pc`, `xPSR`	the CPU, on exception entry	task's stack (PSP)	the CPU
`r4`–`r11`	PendSV (`stmdb`)	task's stack, below the frame	PendSV
stack pointer (PSP)	PendSV (`str`)	TCB, field `sp`	PendSV
memory rights	not saved: recomputed	TCB partition limits	`task_sw`

Compared with week 4. Very similar, with more to save and restore: last week only "launched" tasks (one ran, then the other, and the decision was always "the next task"), so registers were never restored and `r4`–`r11` never saved. Now every switch *saves and restores*, a real scheduler decides, and switches happen at *several moments* (tick, yield, delay, exit), which needs a smart solution so that the switch code is not copied or called in many places: PendSV.

1.7.5 Inspect: `struct tcb` in `kernel/task.h`

Week 4's `struct task` became the **task control block**, `struct tcb`, renamed for consistency with other OSs (task or process control block, TCB or PCB); `struct tcb`, `tcb_current` and `tick_count` follow common RTOS naming.

```
struct tcb {
    uint32_t *sp;                /* saved stack pointer, while the task is
    const char *name;            /* short name for messages, e.g. "A" */
    void ( *entry )( void );     /* the function the task runs */
    uint32_t *stack_top;         /* first address above the task's stack *
    uint32_t part_start;         /* lowest address of the partition */
    uint32_t part_end;           /* first address after the partition */
    uint32_t *data_start;        /* the task's variables in RAM: start...
    uint32_t *data_end;          /* ...and end (exclusive) */
    const uint32_t *data_load;   /* their initial values, stored in FLASH
    enum task_state state;       /* READY, RUNNING, BLOCKED or FINISHED */
    uint32_t wake_tick;          /* if BLOCKED: the tick at which it is RE
};
```

New since week 4: `sp`, `state` (an `enum task_state` value, in `kernel/task.h`: `TASK_READY`, `TASK_RUNNING`, `TASK_BLOCKED`, `TASK_FINISHED`) and `wake_tick`. The rest is week 4's: name, entry function, stack top and partition limits (all from the linker script), and the location of the task's variables and of their initial values.

Why must `sp` be the first field? Assembly knows nothing of C structs: with `sp` at offset 0, its address is the TCB's own, so with `r2 = tcb_current` a plain `ldr r0, [r2]` reads it and `str r0, [r2]` writes it. With `name` first, the switch would take the name pointer for the stack pointer, load a FLASH address (where the string `"B"` is stored) as B's stack and crash, while the C side would compile without any warning: hence the comment in `kernel/task.h`, `sp` "MUST stay the first field".

The task table. `tasks[]` (`kernel/task.c`) holds three TCBs, always in this order: A, B and the **idle task**. `NUM_TASKS` (`kernel/task.h`) is 3; in `kernel/task.c`, `NUM_APP_TASKS` is `NUM_TASKS - 1`, the **application** tasks A and B, and `IDLE_TCB` is `&tasks[NUM_TASKS - 1]`, the last entry. The table is no longer `const` as in week 4: `sp`, `state` and `wake_tick` change at run time.

1.7.6 Who does the switch? PendSV

An **exception** does the switch: the kind of solution computer architects come up with, which "normal" people find completely crazy. Every switch starts with an

exception (SysTick, a system call, ...); the naive approach appends the switch code to every handler that may need it, so the same code sits in several places (tick and system calls), which a programmer should avoid. Instead, such a handler raises another, **pendable** exception, which runs only after the current one has been served: the two handlers are **chained**, every handler stays clean, and the switch is written once, in the handler of **PendSV** ("pendable service call"), a system exception like SVCall and SysTick. Examples: for **round robin at the tick** (or every few ticks), the SysTick handler, after its many duties, would return to the user program, but since it pended PendSV the CPU goes to the PendSV handler, which switches and returns to the new task; **a system call that needs a switch**, e.g. `exit` (the scheduler must run when a program ends), is an SVC whose handler pends PendSV, which runs when the SVC handler finishes, and the CPU then returns to the next task.

How to pend PendSV. Write 1 to bit `PENDSVSET` (bit 28) of the **ICSR** (Interrupt Control and State Register, `0xE000ED04`), as `task_request_switch` in `kernel/task.c` does with `SCB->ICSR = SCB_ICSR_PENDSVSET;`. Writing 0 to the other bits does nothing, so a plain store is enough (no read-modify-write). Every place that needs a switch only *asks* for it ("another exception is chained after me"), and one handler *does* it; two requests together (a tick and a yield) cost one switch, since the pending bit is set once.

Priorities and tail-chaining. A pending exception starts only if it is more urgent than the running code (week 3: a lower number is more urgent). `Reset_Handler` (`boot/startup.s`) sets the faults to `0x00`, SVCall to `0x80`, SysTick and PendSV to `0xE0`, the least urgent value this kernel uses; the UART interrupt is at `0xC0`. So PendSV preempts *no* handler (SysTick is equal, the others more urgent): it runs only when no handler runs, just before the CPU would return to the task. When a handler ends with another exception pending, the CPU goes straight to the next handler without returning to the task: **tail-chaining**. A PendSV pended in `SysTick_Handler` thus runs right after it, never inside it; equal priority also means that SysTick cannot preempt PendSV, so a tick never cuts a switch in half. PendSV therefore always interrupts a *task*: PSP is the running task's stack, and the exception return goes back to thread mode, where the next task must start. Switching inside SysTick would be unsafe too: in most systems the tick is more urgent than some device handlers, and a switch while it interrupts one of them would make that handler return into the wrong task.

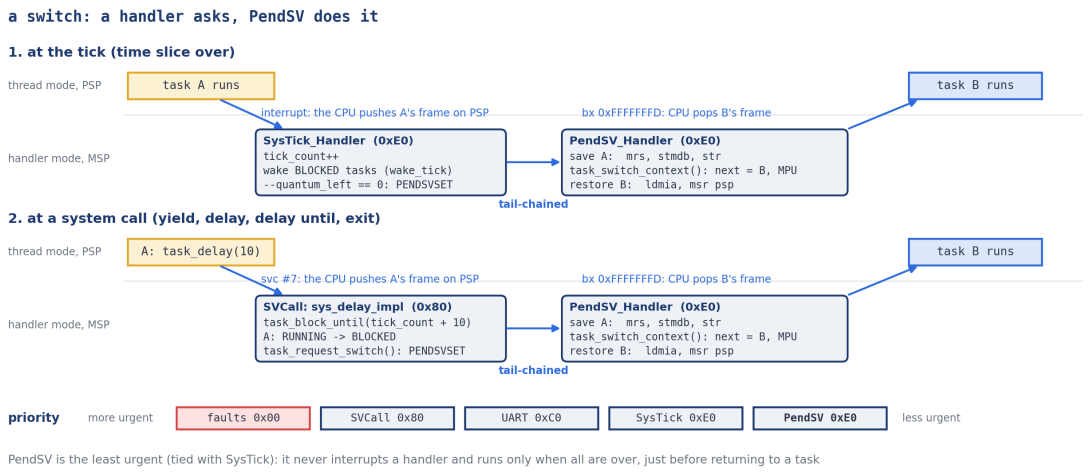


Figure 1.13: The two ways to a switch. At the tick, SysTick counts the slice and, when it is over, pends PendSV. At a system call, the SVCall service (here `task_delay`) blocks the task and pends PendSV. In both cases PendSV, the least urgent exception, runs tail-chained after the handler: it saves A, chooses B and returns into B.

1.7.7 The switch, step by step: `PendSV_Handler` in `kernel/pendsv.s`

A and B run unprivileged, on PSP. A runs; at a tick, `SysTick_Handler` (`0xE0`) increments `tick_count`, wakes the delayed tasks and decrements `quantum_left`, setting `ICSR.PENDSVSET` when it reaches 0 (the SVCs of `task_yield`, `task_delay` and `exit` set it too). Then PendSV does the switch: low-level, so in assembly, but short, 13 instructions (here with their C equivalents).

```
PendSV_Handler:
    mrs   r0, psp              /* sp = PSP: the old task's stack, below its
    stmdb r0!, {r4-r11}        /* sp -= 8; sp[0..7] = r4..r11 */
    ldr   r1, =tcb_current     /* r1 = &tcb_current */
    ldr   r2, [r1]             /* r2 = tcb_current */
    str   r0, [r2]             /* tcb_current->sp = sp (sp is at offset 0)
    bl    task_switch_context  /* task_switch_context(); changes tcb_curre
    ldr   r1, =tcb_current     /* r1 = &tcb_current */
    ldr   r2, [r1]             /* r2 = tcb_current (the new task) */
    ldr   r0, [r2]             /* sp = tcb_current->sp */
    ldmia r0!, {r4-r11}        /* r4..r11 = sp[0..7]; sp += 8 */
    msr   psp, r0              /* PSP = sp: now it points at the hardware f
    ldr   r0, =0xFFFFFFFD      /* r0 = EXC_RETURN: thread mode, PSP */
    bx    r0                   /* return_from_exception(0xFFFFFFFD) */
```

1. **Entry**: SysTick returns and PendSV is tail-chained. A's hardware frame, pushed on A's stack when SysTick started, is not popped and pushed again: it stays where it is, and only the other registers must be saved.

2. **Save A** (the first five instructions). The handler runs on **MSP**, so `mrs r0, psp` reads A's stack pointer explicitly. To be fast, `stmdb r0!, {r4-r11}` ("store

multiple, decrement before": registers on the right, destination first) pushes `r4–r11` with one instruction into the 8 words below `r0`, `r4` lowest, and `!` writes the new, lower address back into `r0`: a push, but on PSP instead of the handler's own stack. Then `r0` is stored at offset 0 of `tcb_current`, i.e. in `tcb_current->sp`, as in week 4. The two halves combine: part of the state saved by the hardware on entry, the rest pushed by software, the stack pointer kept in the TCB.

3. **Choose**: `bl task_switch_context` calls the C function that marks A READY, picks the next READY task in round-robin order (B), sets `tcb_current = B` and programs MPU region 1 for B's partition.

4. **Restore B**, the exact opposite: reload `tcb_current`, read B's `sp`, pop `r4–r11` with `ldmia` ("load multiple, increment after", the reverse of `stmdb`), and `msr psp, r0` points PSP at what is left: B's hardware frame.

5. **Return**: `0xFFFFFFFD` is the `EXC_RETURN` value the link register must hold, on return from an exception, to go back to thread mode (user space) using PSP (week 4). `bx r0` returns: the hardware pops B's frame, and B continues at the instruction where it was stopped.



Figure 1.14: The first switch (tick 5), with the addresses of the gdb session. 1: PSP points at A's hardware frame, while `r4–r11` are still in the CPU. 2: `stmdb` pushes them below the frame, `A.sp` = `0x20000398`, and `task_switch_context` moves `tcb_current` (and the MPU) to B. 3: `ldmia` pops B's `r4–r11`, PSP = `0x200013E0`, and the exception return pops B's frame.

PendSV starts in A and ends in B; A notices nothing, since at its next turn steps 4 and 5 give it back exactly the registers it had.

Why may the handler call C without saving `r0–r3`? A C function receives its arguments in `r0`, `r1`, `r2`, ..., returns its result in `r0`, and may change `r0–r3`, `r12`

and `lr`. Nothing of the old task is lost: those registers are in its hardware frame, `r4`–`r11` were just pushed (and the C function gives them back unchanged anyway, as the calling rules require), and right after the call all of the new task's registers are restored. And since the handler pushes nothing on MSP, the main stack keeps the **8-byte alignment** given by the hardware on exception entry, which the C calling rules require when calling a function. **Warning** (especially for the project): whenever assembly calls C, check very carefully that no register contents are destroyed; the compiler adds instructions to follow the calling convention, and forgetting that they exist leads to strange behaviours.

Why is the entry `lr` never saved? On exception entry `lr` holds an `EXC_RETURN` value, not a return address. Every task runs in thread mode on PSP, so the right value is always `0xFFFFFFFD`, loaded as a constant at the end; the `lr` received (destroyed by `bl`) is not needed, and the task's own `lr` is in its hardware frame, popped by the hardware.

Other remarks. `tcb_current` is read twice, because `task_switch_context()` changes it from the old task to the new one. `CONTROL` is never touched: `nPRIV`, set when the first task started (week 4), is never cleared, so every task, B included, runs unprivileged. Interrupts stay enabled: a UART interrupt (`0xC0`, more urgent) may interrupt PendSV, but never touches PSP or the TCBs; SysTick (same priority) cannot, and SVCall cannot happen while a handler runs.

1.7.8 Inspect: `task_switch_context`, the round-robin circle

Besides step 3 above, it gives the chosen task a fresh slice (`quantum_left = QUANTUM`). `(rr_last + k) % NUM_APP_TASKS` visits the application tasks in table order, from the one after the last that ran, so A and B alternate: round robin *without a separate queue*, since the table itself is the circle. Not the best possible implementation (it relies on a static array, as there is no dynamic memory), but simple. FINISHED and BLOCKED tasks are never READY, so they are skipped; a preempted task, READY again, goes to the end of the circle. With dynamic memory, a linked list of READY tasks would make round robin even easier (no scan: take the head of the list), but our OS has no `malloc`.

A's slice ends while B is BLOCKED: who runs? A is first set READY; the scan starts after `rr_last = 0` (A): `k = 1` tries B (BLOCKED), `k = 2` A itself (READY), so A runs again with a new slice (Unit 1's "with no other READY task, the running task keeps the CPU"). Not optimised, though: the tick came, PendSV was chained and A saved before the scheduler finds that A must run again, so A is restored: the **cost of a switch is paid anyway**, although the running task does not change. Optimisations could avoid it (e.g. checking whether a switch is really needed before pending PendSV).

A and B both BLOCKED: who runs? No task is READY, so `next` stays `IDLE_TCB`: the **idle task** runs. `rr_last` does not change, so afterwards the circle goes on where it stopped.

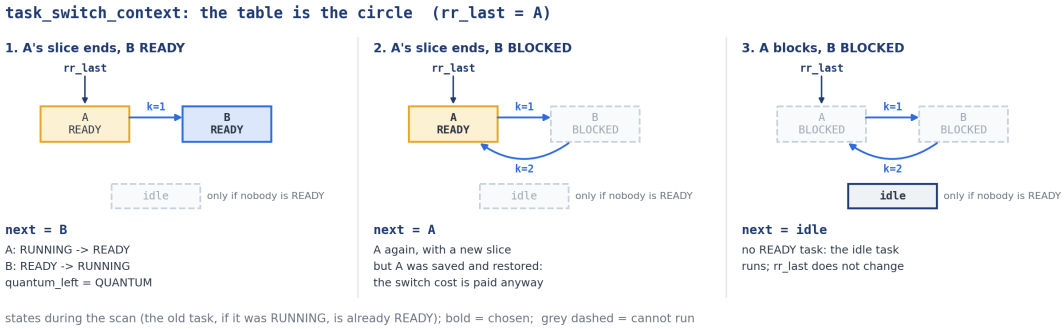


Figure 1.15: The three cases of `task_switch_context`. The scan starts after `rr_last`: if B is READY it runs; if B is BLOCKED, A runs again, but the switch cost has already been paid; if nobody is READY, the idle task runs.

1.7.9 Try it: one switch in gdb

Run `make debug` in `week-05/base` and, in a second terminal in the same folder, `arm-none-eabi-gdb build/scratch-os.axf`, with breakpoints on the switch code:

```
(gdb) target remote :1234
(gdb) break PendSV_Handler
(gdb) break task_switch_context
(gdb) continue
(gdb) p tick_count
(gdb) p tcb_current->name
(gdb) p/x $psp
(gdb) x/8wx $psp                                  # A's hardware frame
(gdb) info symbol *(uint32_t *)($psp + 24)        # the saved pc: word 6
(gdb) info symbol *(uint32_t *)($psp + 20)        # the saved lr: word 5
```

Rule for gdb this week: read kernel variables only while the CPU is stopped in a **handler**, as here. Stopped inside a task (thread mode), the MPU checks gdb's reads too, with the task's rights: `p tick_count` fails with `Cannot access memory`, since `tick_count` is in kernel RAM, which an unprivileged task may not read. The output:

```
Breakpoint 1, PendSV_Handler () at kernel/pendsv.s
$1 = 5
$2 = 0x11fc "A"
$3 = 0x200003b8
0x200003b8: 0x00000004 0x00000013 0x00000002 0x00000000
0x200003c8: 0x00000000 0x00001005 0x00001040 0x21000000
task_tick_count + 2 in section .text
spin_ticks + 21 in section .text
```

- The first switch is at tick 5, the end of A's first slice. The breakpoint is at the first instruction of `PendSV_Handler`: nothing is saved yet, and `tcb_current` is still A.

- PSP = `0x200003b8`, inside A's stack (`0x20000000`–`0x20000400`); the 8 words are the frame pushed on SysTick entry: `r0`–`r3`, `r12`, then `lr = 0x1005`, `pc = 0x1040`, `xPSR = 0x21000000`.
- The saved `pc` is in `task_tick_count`: A was stopped in its busy work, which calls that stub in a loop, and the saved `lr` returns into `spin_ticks`. `xPSR` has the T bit (`0x01000000`) and the C flag (`0x20000000`) of A's last comparison.
- With a compiler version other than the course one the addresses may differ slightly; the meaning is the same.

Continue: A saved, B chosen. Same session; `0xE000ED9C` is the MPU's `MPU_RBAR` (week 4), the next word `MPU_RASR`.

```
(gdb) continue                          # stops in task_switch_context
(gdb) p tcb_current->sp
(gdb) x/8wx tcb_current->sp             # A's r4..r11
(gdb) x/2wx 0xE000ED9C                  # MPU region 1 now
(gdb) finish                            # run to the end of the function
(gdb) p tcb_current->name
(gdb) p tcb_current->sp
(gdb) x/16wx tcb_current->sp            # B's stack
(gdb) x/2wx 0xE000ED9C                  # MPU region 1 after the choice
```

B has never run: what do you expect in its 16 words? The output:

```
Breakpoint 2, task_switch_context () at kernel/task.c
$4 = (uint32_t *) 0x20000398
0x20000398: 0x00000002 0x00000000 0x00000000 0x200003d8
0x200003a8: 0x00000000 0x00000000 0x00000000 0x00000000
0xe000ed9c: 0x20000001 0x13060017
PendSV_Handler () at kernel/pendsv.s
$5 = 0x1200 "B"
$6 = (uint32_t *) 0x200013c0
0x200013c0: 0x00000000 0x00000000 0x00000000 0x00000000
0x200013d0: 0x00000000 0x00000000 0x00000000 0x00000000
0x200013e0: 0x00000000 0x00000000 0x00000000 0x00000000
0x200013f0: 0x00000000 0x00001043 0x000010c4 0x01000000
0xe000ed9c: 0x20001001 0x13060017
```

- A's saved `sp` is `0x20000398` = `0x200003b8` − 32: `stmdb` pushed A's `r4`–`r11` (8 registers, 32 bytes) below the hardware frame, with their values at the moment A was stopped.
- `finish` runs to the end of `task_switch_context` and stops back in `PendSV_Handler`, just after the `bl`: `tcb_current` is now B.
- B's 16 words: 8 zeros for `r4`–`r11`, then a hardware frame with `r0`–`r3`, `r12` = 0, `lr = 0x1043` (`task_exit`, with the Thumb bit set), `pc = 0x10c4` (`task_b`) and `xPSR = 0x01000000` (T bit only). B never ran, so nobody ever saved it: it is a **fake** frame (next subsection).

- `MPU_RBAR` went from `0x20000001` (A's partition, region 1) to `0x20001001` (B's partition, region 1): the choice moved the MPU window.

Quit gdb with `quit` (answer `y`), and QEMU with `Ctrl-C` in its terminal.

1.7.10 The first run of a task: a fake frame

In week 4 a task's first run was a **fake exception**: `task_launch` wrote an 8-word hardware frame on its stack and returned into it, as if from an exception. This now proves useful, since every run of a task is assumed to come from an exception, even the very first; but a first run must now look like a **full context switch**, because PendSV switches to B, which never ran and was never saved, exactly as it switches back to A (load `sp`, pop 8 words, set PSP, return). So the fake frame must be *bigger*:

- At boot, `task_init_stack` builds on **every** task's stack what PendSV would have left had the task been switched out just before its first instruction: the 8-word hardware frame **plus** 8 words for `r4`–`r11` below it, with `t->sp` at the lowest word. The first switch to a task is then identical to every later one (PendSV believes the task was running, finds everything correctly in memory, and the task starts): no special case for a new task.
- Only the very first task, A, is entered differently: `main`'s `svc #5` (`SYS_START_TASKS`) ends in `task_start_first` (`kernel/task.c`), which sets PSP to `sp + 8`, i.e. to the hardware frame, skipping the 8 zeros for `r4`–`r11` because the exception return pops only the hardware frame; then it sets nPRIV and returns from SVCall with `0xFFFFFFFD`, as in week 4.

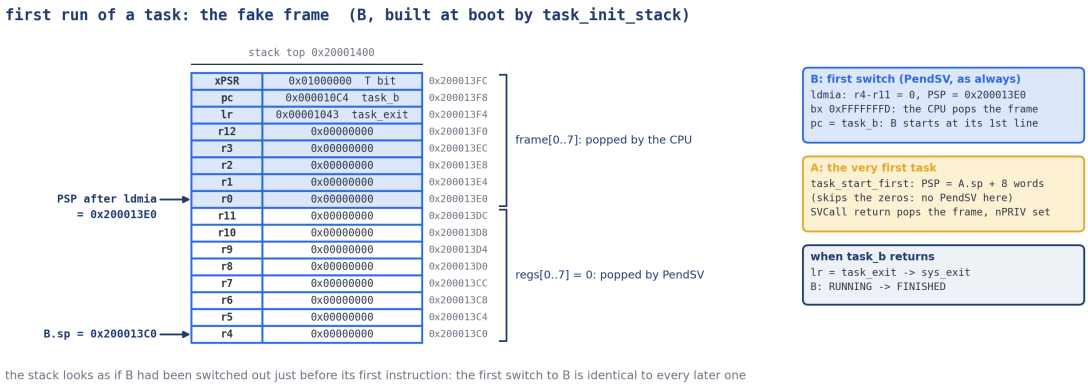


Figure 1.16: B's stack as `task_init_stack` leaves it at boot: a hardware frame (`pc` = `task_b`, `lr` = `task_exit`, T bit set) plus 8 zero words for `r4`–`r11`, with `B.sp` at the lowest word. B is entered by PendSV like every switch; only A, the very first task, is entered by `task_start_first`, which skips the zeros.

Inspect: `task_init_stack` **in** `kernel/task.c`. `task_start_first` calls it once per task (A, B and idle) before the first task starts, together with week 4's `partition_init` and with setting the state to READY. The 16 words for task B (stack top `0x20001400`) are the ones gdb showed:

Words	Addresses (task B)	Register	Value written
`regs[0..7]`	`0x200013c0`–`0x200013df`	`r4...r11`	0 (PendSV pops them)
`frame[0..4]`	`0x200013e0`–`0x200013f3`	`r0`, `r1`, `r2`, `r3`, `r12`	0
`frame[5]`	`0x200013f4`	`lr`	`task_exit`
`frame[6]`	`0x200013f8`	`pc`	`t->entry & ~1u`: `task_b`
`frame[7]`	`0x200013fc`	`xPSR`	`0x01000000` (`XPSR_THUMB`)

The heart of the function:

```
uint32_t *frame = t->stack_top - 8;    /* the hardware frame: 8 words
...                                    /* frame[0..7] as in the table
uint32_t *regs = frame - 8;            /* 8 more words for r4..r11 */
for ( int i = 0; i < 8; i++ )
    regs[i] = 0;                       /* r4 + i = 0 */
t->sp = regs;                          /* the saved stack pointer */
```

- B's saved `sp` is `0x200013c0`: 16 words are 64 bytes (`0x40`), and `0x20001400` − `0x40` = `0x200013c0`.
- `r4`–`r11` may start as 0: a C function never reads them before writing them (it only saves and restores them for its caller), so any value works for a task's first instruction.
- `lr`, `pc` and T bit work as in week 4's `task_launch`: the `pc` is the entry function with bit 0 cleared, since the popped `pc` must be a real instruction address; `xPSR` needs the T (Thumb) bit, or the first instruction faults; `lr` is `task_exit` (`lib/syscalls.s`), which calls `sys_exit` when `task_b` returns: returning from a task's function means exit.

1.7.11 One MPU window, moved at every switch

Already implemented in week 4. On a single core one task runs at a time, so the MPU needs only **one task region**, which the kernel keeps moving onto the running task: at the end of every scheduling decision, `task_switch_context()` takes the next task's partition limits from its TCB and calls `mpu_set_task_region()` with the partition's start and size. RAM holds three 4KB partitions (A at `0x20000000`, B at `0x20001000`, idle at `0x20002000`), with the kernel's RAM above; region 0 is FLASH, read-only, executable and shared by all.

Try it: a rogue task after a switch. Does week 4's isolation survive the switches? With `DEMO=rogue`, B stores `0x12345678` into the kernel's `uptime_ms`, as in week 4, but only after A's first slice, i.e. **after a context switch**.

```
make DEMO=rogue run
```

The demo builds into `build/demo-rogue/`. Output (quit QEMU with `Ctrl-C`):

```
scratch-os: kernel started
scheduler: round robin, quantum 5 ticks
A: hello from task A
```

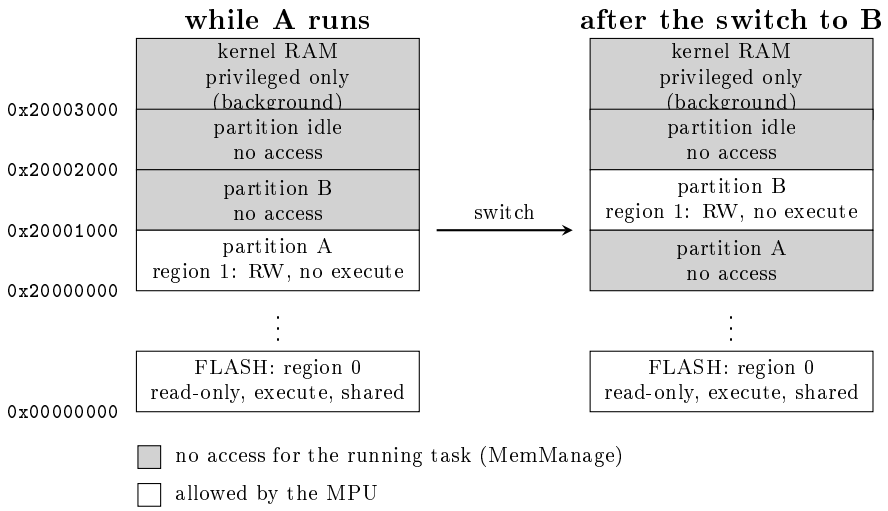


Figure 1.17: `task_switch_context()` calls `mpu_set_task_region()` for the new task: `MPU_RBAR` goes from `0x20000001` to `0x20001001`.

```
A: step 1 (tick 2)
A: step 2 (tick 4)
B: hello from task B
B: writing to the kernel's uptime counter
MemManage: CFSR=0x82
address 0x20003094
cause: data access
```

- `0x20003094` is `uptime_ms` (`drivers/systick.c`), in kernel RAM above the three partitions (a little higher in the solution, whose trace buffer also lives in kernel RAM). `CFSR = 0x82`: as in week 4, a data access violation (`DACCVIOL`) with a valid fault address (`MMARVALID`).
- **Which MPU region decides on B's store?** Region 1, set by the switch at tick 5 to B's partition, `0x20001000`–`0x20002000`. Kernel RAM is reachable only through the privileged background region, and B is unprivileged: the store faults, **exactly as in week 4**. A's partition is off limits to B too: after the switch, no region describes it for B. `MemManage_Handler` (`kernel/faults.c`) then stops the whole system, as in week 4.

1.7.12 SysTick, the time slice and `task_increment_tick`

The **system tick** is the hardware interrupt, every **10 ms** here (week 3's SysTick); the **time slice** is round robin's quantum, implemented by **counting ticks**. The scheduler gets control only at a tick, so the quantum must be a **multiple of the tick**: one that is not (e.g. shorter than a tick) is meaningless, as the task is interrupted only every 10 ms anyway. The tick period is set by programming the hardware counter, so tick and scheduler must be designed together. Since this week, `SysTick_Handler` (`drivers/systick.c`) also calls `task_increment_tick()`, which drives the scheduler; like all kernel functions from now on, it works by updating the TCBs.

```
void task_increment_tick( void )
{
    tick_count++;
    if ( tcb_current == 0 ) return;                    /* no task started ye

    int woke = 0;
    for ( unsigned i = 0; i < NUM_APP_TASKS; i++ )
        if ( tasks[i].state == TASK_BLOCKED
             && ( int32_t ) ( tick_count - tasks[i].wake_tick ) >= 0 )
        {
            tasks[i].state = TASK_READY; woke = 1;   /* may run again */
        }

    if ( tcb_current == IDLE_TCB )                     /* idle: leave it on
    {
        if ( woke ) task_request_switch();
        return;
    }

#if QUANTUM > 0
    quantum_left--;
    if ( quantum_left == 0 ) task_request_switch(); /* SCB->ICSR = PENDSV
#endif
}
```

At every tick:

- `tick_count++`: the kernel's clock, in ticks since the scheduler started, read by tasks with the system call `task_tick_count()`. `tcb_current` is 0 until the first task starts (`main` starts SysTick before the tasks), and until then the function only counts.
- **Wake up**: the tick is usually where the kernel wakes blocked tasks. The delay (sleep) system call, modified, now sets the task BLOCKED (here delays are the only way to block, the task saying how many ticks it sleeps); at each tick, every BLOCKED application task whose `wake_tick` has come becomes READY (BLOCKED → READY). A task that sleeps 10 ms (one tick) is detected by the loop at the following tick.
- **The time slice**: the kernel does not schedule at every tick; with a 10 ms tick and a 100 ms quantum, it counts the ticks since the start of the current quantum and schedules only when it is over. `quantum_left`, the ticks left to the running task, starts at `QUANTUM` (5 by default, a `#define` in `kernel/task.h` that the Makefile can change); at 0, `task_request_switch()` pends PendSV, which runs the scheduler.
- `QUANTUM = 0` means "never switch on the tick": `#if QUANTUM > 0` removes the slice code completely, and tasks keep the CPU until they block, yield or exit, passing control to each other: Unit 1's **cooperative** scheduling.

A woken task does nothing at once: READY, it waits for the end of the running task's slice (Unit 1's rules); only the idle task gives way at once, because when

`woke` is set during idle the tick asks for a switch immediately. Nor is the quantum **aligned to a global grid** (from a question in class): if a task stops after 1 ms of a 100 ms quantum (it blocks, yields or exits), the next task does not wait for a "quantum boundary": it runs as soon as the CPU is free, with its quantum starting then (`quantum_left = QUANTUM` in `task_switch_context`).

Why `(int32_t)(tick_count - wake_tick) >= 0` and not `tick_count >= wake_tick`? After 2^{32} ticks (about 497 days at 10 ms) `tick_count` wraps around to 0, and a plain comparison fails: a wake-up tick just before the wrap would look far in the future. The unsigned difference read as signed stays correct across the wrap (for values less than 2^{31} ticks apart), like week 3's `now - start` test.

Lesson: an OS is many small parts working together. The code is simpler than it looks on slides: many small, very simple parts working together, and building an OS means spending a lot of time checking that they are linked properly: e.g. `task_request_switch` is called from the system-call exceptions but also from `task_increment_tick`, and every event that can happen must be kept under control. Every single bug can crash the system. Big bugs are not really a problem: the system crashes and they get fixed. The nightmare is small hidden bugs that show up only in rare corner cases: everything seems fine, then suddenly the OS crashes.

1.7.13 Try it: change the quantum

The quantum is a Makefile variable, easy to experiment with. With 2 ticks:

```
make QUANTUM=2 run
```

The build goes into its own folder, `build/quantum-2/`, never mixing with the default one. Output (quit QEMU with `Ctrl-C`):

```
scratch-os: kernel started
scheduler: round robin, quantum 2
A: hello from task A
B: hello from task B
A: step 1 (tick 4)
B: step 1 (tick 6)
A: step 2 (tick 8)
B: step 2 (tick 10)
A: step 3 (tick 12)
B: step 3 (tick 14)
A: step 4 (tick 16)
B: step 4 (tick 18)
A: step 5 (tick 20)
B: step 5 (tick 22)
A: step 6 (tick 24)
B: step 6 (tick 26)
A: step 7 (tick 28)
B: step 7 (tick 30)
A: step 8 (tick 32)
B: step 8 (tick 34)
A: step 9 (tick 36)
B: step 9 (tick 38)
A: step 10 (tick 40)
task A exited (tick 40)
B: step 10 (tick 40)
B: step 11 (tick 42)
B: step 12 (tick 44)
B: step 13 (tick 46)
B: step 14 (tick 48)
B: step 15 (tick 50)
task B exited (tick 50)
all tasks finished
```

Compared with the default quantum of 5:

- **One step per turn**: a 2-tick slice is one step. A's slice ends at tick 2, just as step 1 would end; back on the CPU at 4, the step is over and A prints it. So each line is printed at the *start* of a turn of its task.

- **A finishes at tick 40 (not 32), B at 50 (not 44)**: the busy work counts clock ticks, also while the task is switched out, and a step ending with its slice is printed only at the task's next turn, 2 ticks later, so while both tasks run every step costs 4 ticks of clock time.
- **21 switches instead of 7** (counted by the trace): Unit 1's overhead $s/(q+s)$ grows as q shrinks.

1.7.14 Task states in the code

Unit 1's states are `enum task_state` (`kernel/task.h`), and each transition is one line of `kernel/task.c`:

Transition	When	Where in the code
→ READY	at boot, for every task	`task_start_first`
READY → RUNNING	the task is chosen	`task_switch_context`
RUNNING → READY	preempted or yield	`task_switch_context`
RUNNING → BLOCKED	delay, delay until	`task_block_until`
BLOCKED → READY	its `wake_tick` has come	`task_increment_tick`
RUNNING → FINISHED	the task calls exit (or returns)	`task_finish_current`

READY → RUNNING and RUNNING → READY are both in `task_switch_context`, since with round robin a switch removes one task from the CPU and starts another. There is no BLOCKED → RUNNING (a woken task waits as READY), and exactly one task is RUNNING (`tcb_current`): the idle task, if no other can.

1.7.15 Giving the CPU away: yield, delay, delay until, exit

Three new system calls (declared in `lib/syscalls.h`) leave the CPU before the slice ends: each is an `svc` whose service (`kernel/syscall.c`) pends PendSV, which, less urgent (`0xE0`) than SVCall (`0x80`), waits and runs right after the `svc`, tail-chained.

- `task_yield()` (`svc #6`, `sys_yield_impl`): "let the other task run now"; the task stays READY and goes to the end of the circle, and the scheduler picks it again if no other task is READY.
- `task_delay(ticks)` (`svc #7`, `sys_delay_impl`): BLOCKED until `tick_count` reaches now + `ticks`.
- `task_delay_until(&last, period)` (`svc #8`, `sys_delay_until_impl`): BLOCKED until tick `last + period`, with `last` moved to that tick.

Why two delays? `task_delay` waits a number of ticks counted from the call, ignoring the time the task already spent (on its work, in the system call, waiting for the CPU): inconvenient for a **periodic task**. A task that must run every 2 seconds is an infinite loop of "some activity, then wait"; with `delay` it would have to compute how long its activity took and delay only for the rest, which is very difficult, and a loop "work, then `delay(10)`" has a period of work + 10 ticks + any wait for the CPU: it **drifts**, and is never exact. With `task_delay_until` the task tells the OS "this was the beginning of my periodic activation, and I want one every 2 seconds": if the work

took 1 second, the OS makes it wait just 1 more. Wake-ups stay at `last + period`, `last + 2·period`, ... however long the work took (as long as it is shorter than the period): **no drift**; `last` is set once, with `task_tick_count()`, before the loop. For the scheduler both calls just block the task, but they are implemented differently, and a periodic task (one that must run every T ticks) needs `task_delay_until`. Both exist in FreeRTOS, where programming is largely about using them: this is why they were implemented here.

Exit. `sys_exit_impl` calls `task_finish_current`, which marks the task FINISHED (RUNNING → FINISHED) and pends PendSV; the task is never chosen again. When only the idle task is left, `task_finish_current` prints `all tasks finished`; QEMU keeps running after that line because the idle task runs forever. Unlike week 4, `main` never gets the CPU back.

Inspect: `task_block_until` and the delay services. The two delay services share one helper, `task_block_until` (`kernel/task.c`):

```
void task_block_until( uint32_t wake_tick )
{
    tcb_current->wake_tick = wake_tick;          /* when to wake up */
    tcb_current->state = TASK_BLOCKED;           /* not READY until then *
    task_request_switch();                       /* give the CPU to anothe
}
```

The body of `sys_delay_impl` (`kernel/syscall.c`; `a0` is the number of ticks):

```
    if ( a0 == 0 )
        task_request_switch();                   /* delay(0) = yield */
    else
        task_block_until( tick_count + a0 );     /* READY again at tick no
```

and of `sys_delay_until_impl` (`a0` is the address of `last_wake`, `a1` the period, `t` is `tcb_current`):

```
    if ( !in_range( a0, sizeof( uint32_t ), t->part_start, t->part_end )
         || ( a0 & 3 ) != 0 )
        return ( uint32_t ) -1;
    uint32_t *last_wake = ( uint32_t * ) a0;     /* the number is really a
    uint32_t wake = *last_wake + a1;             /* the next wake-up tick
    *last_wake = wake;                           /* remembered for the nex
    if ( ( int32_t ) ( wake - tick_count ) > 0 )
        task_block_until( wake );
```

Both services first return -1 if no task has started yet.

- **Why the `in_range` check?** The kernel **writes** `*last_wake`, and it runs privileged, so the MPU would not stop it: without the check, a task could pass the address of a kernel variable and have the kernel overwrite it (the lesson of week 4's `sys_write` check). Only an address inside the caller's own partition, aligned to 4 like every `uint32_t`, is accepted.
- **What if the task calls `task_delay_until` after its wake-up tick?** If `wake` is not in the future, the test is false: the task does **not** block, but continues at

once, to catch up. `last` still moves by one period, so the later wake-ups stay on the same grid.

- **Where does the task go on afterwards?** PendSV switched away right after the `svc`, saving the `pc` that follows it: when chosen again, the task resumes there, returns into the stub (`task_delay` in `lib/syscalls.s`), and the stub returns to the caller.

1.7.16 Try it: no preemption, no yield

In a cooperative system, a task that never gives the CPU away freezes the others (Unit 1). `DEMO=coop` shows it: `QUANTUM` is 0, so the tick never switches (the `Makefile` forces it with `override`), and task A loops forever on its busy work, never yielding.

```
make DEMO=coop run
```

Let it run for a few seconds, then quit QEMU with `Ctrl-C`:

```
scratch-os: kernel started
scheduler: round robin, quantum 0 ticks
A: hello from task A
A: step 1 (tick 0)
A: step 2 (tick 2)
A: step 3 (tick 4)
...
A: step 182 (tick 362)
```

- **Task B never prints anything**, while A's step number grows as long as QEMU runs: B is READY all the time, but nobody takes the CPU from A, and A never gives it away. B **starves**.

- **The fix is a yield**: `task_yield();` at the end of A's loop body (`src/task_a.c`) makes A give the CPU away after each step. B then runs, but its own loop never yields, so now **A** waits until B has finished its 15 steps; with `task_yield();` in the loops of **both** tasks, they alternate one step each, as with a quantum of 2. In a cooperative system every task must play fair: one loop without a yield or a delay freezes all the others.

1.7.17 The idle task

When A and B are both BLOCKED (or FINISHED), the CPU runs the idle task, the last entry of `tasks[]`: Unit 1's question answered. It does nothing, and runs whenever there is nothing else to do: where week 4's task B ended with an infinite loop, now A and B both end, and then the idle task runs forever, as in real operating systems.

- It is a task like the others: unprivileged, with its own stack and its own 4 KB partition (`0x20002000`–`0x20003000`, placed by the linker script by its file name, `idle.o`), so the switch, the MPU and the stacks need no special case for it.

- It is never in the round-robin circle (`task_switch_context` picks it only when no application task is READY), and it has no time slice: with no task able to run, switching makes no sense, so `task_increment_tick` leaves it only when a task wakes up.
- **Why not a loop inside the kernel?** Kernel code runs in handlers, and a looping handler blocks every exception of the same or lower urgency: PendSV and SysTick (`0xE0`) could never run, so no tick would come, no task would wake, and no switch would happen.
- It waits with `wfi` ("wait for interrupt", allowed in unprivileged code): the core stops executing until the next interrupt, which saves energy.

The whole task is `idle_task` in `src/idle.c`:

```
void idle_task( void )
{
    for ( ;; )
    {
        __asm volatile ( "wfi" );        /* sleep until the next interrupt
    }
}
```

At the latest the next SysTick, 10 ms away, wakes the CPU from `wfi`: `task_increment_tick` runs, and if a delayed task is READY again it pends PendSV, so the exception return goes into that task instead of the idle task. Otherwise the handler returns into the idle task right after its `wfi`, and the loop sends it back to sleep until the next tick. The idle task never returns: an `exit` from it would leave the CPU with nothing to run.

Try it: delays and the idle task. With `make DEMO=idle run`, both tasks do 5 steps of 1 tick of work each: A prints, works 1 tick and calls `task_delay(10)`; B calls `task_delay_until(&last, 6)`, then prints and works 1 tick, with `last` starting at B's first tick. Output, in chronological order:

```
scratch-os: kernel started
scheduler: round robin, quantum 5 ticks
A: hello from task A
A: step 1 (tick 0)
B: hello from task B
B: step 1 (tick 7)
A: step 2 (tick 11)
B: step 2 (tick 13)
B: step 3 (tick 19)
A: step 3 (tick 22)
B: step 4 (tick 25)
B: step 5 (tick 31)
task B exited (tick 32)
A: step 4 (tick 33)
A: step 5 (tick 44)
task A exited (tick 55)
all tasks finished
```

- A drifts: steps at 0, 11, 22, 33, 44, a period of 11 = 1 tick of work + 10 of delay; it exits at 55, after its last delay.
- B does not: `last` starts at 1, and B wakes at 7, 13, 19, 25, 31, exactly every 6 ticks, despite its 1 tick of work.
- Most of the time both tasks are BLOCKED and the idle task runs, until the tick at which a task wakes up.

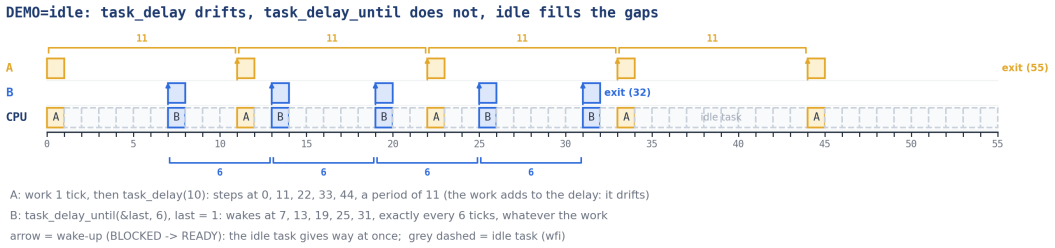


Figure 1.18: `DEMO=idle` tick by tick. A (`task_delay(10)`) runs every 11 ticks, because its work adds to the delay: it drifts. B (`task_delay_until`, period 6) wakes exactly every 6 ticks. In between, the idle task runs and gives way at once when a task wakes up.

1.7.18 Seeing the schedule: a trace

The output shows when each step *ended*, not who had the CPU at each tick. To check that many tasks run correctly, and to draw Unit 1's Gantt chart of a real run, the kernel must log its execution, **writing down every switch**. Logging inside an OS is hard: `printf` cannot simply be added anywhere, and where it can it is very slow, so in the scheduler, which must be fast, it would completely destroy the timing of the tasks. The trace is therefore "smart": events are logged in memory and printed later, when there is time. `week-05/solution/` has it; the next subsections write it in `week-05/base/`, at its `CLASS` comments. The design:

- **One record per switch**: the tick and the indexes in `tasks[]` of the task that stopped (`from`) and of the one that starts (`to`). Between two records the `to` of the first one owned the CPU, so the records are enough to rebuild the whole chart.
- **Recorded in** `task_switch_context`, the one place every switch passes through, and only when the task really changes ("A keeps the CPU" is not a switch).
- **Kept in RAM, printed later**: printing inside PendSV would slow every switch and change the very schedule we want to observe, while writing 8 bytes is fast.
- **A ring buffer** of 64 records: when full, the next record overwrites the **oldest**. A counter `count` of all the records ever written gives the next slot, `count % 64`, and tells how many are still stored.

- **Printed by a system call**: just before returning, task B calls `trace_dump()` (`svc #9`, already prepared: its service `sys_trace_dump_impl` in `kernel/syscall.c` calls `trace_print`). `trace_print` prints `TRACE BEGIN`, one line `T <tick> <from> <to>` per record (oldest first), `TRACE END <tick now>`, then a small Gantt chart; the course tool `schedsim` reads the same lines.

- The interface is prepared in `kernel/trace.h`: `trace_record_switch( uint8_t from, uint8_t to )` and `trace_print( void )`. You write `kernel/trace.c` at its `CLASS` comments, plus one call in `kernel/task.c`.

No lock protects the buffer, and none is needed: `trace_record_switch` runs inside PendSV and `trace_print` inside SVCall, which never interrupt each other in the middle of their work (PendSV runs only when SVCall is over).

1.7.19 Writing the trace: record, buffer and recording

In `week-05/base/kernel/trace.c`, the comment `/* CLASS: trace record type goes here */` becomes the record type:

```
/* One task switch. */
struct trace_record {
    uint32_t tick;      /* the tick at which the switch happened */
    uint8_t from;       /* index in the task table of the task that stopp
    uint8_t to;         /* index in the task table of the task that start
};
```

and `/* CLASS: ring buffer goes here */` the buffer:

```
/* How many records the buffer holds. */
#define TRACE_SIZE 64

/* The buffer is a ring: when it is full, a new record overwrites the old
static struct trace_record buffer[TRACE_SIZE];
/* How many records were written in all. The next one goes to slot count
static uint32_t count = 0;
```

`buffer` and `count` are `static`, visible only in `kernel/trace.c`: kernel variables, in kernel RAM, which no task can read or change. Inside `trace_record_switch`, the three placeholder lines (`(void) from;`, `(void) to;` and the `CLASS` comment) become:

```
    struct trace_record *r = &buffer[count % TRACE_SIZE];
/* the next slot */
    r->tick = tick_count;                   /* when */
    r->from = from;                         /* who stopped */
    r->to = to;                             /* who starts */
    count++;                                /* one more record */
```

The slot goes 0, 1, ..., 63, then 0 again: the ring. The function runs inside PendSV, in the middle of the switch, so `tick_count` is the tick that asked for it. At the end of `task_switch_context` (`kernel/task.c`), the comment `/* CLASS: trace hook goes here (task_switch_context) */` becomes:

```
/* Write the switch in the trace, but only if the task really changed
 * (a task that keeps the CPU is not a switch). The pointer differenc
 * is the index in the task table. */
if ( next != old )
{
    trace_record_switch( old - tasks, next - tasks );
}
```

`old - tasks` subtracts two pointers into the same array: in C the result counts **elements**, not bytes, so it is the index of `old` in `tasks[]` (0 for A, 1 for B, 2 for idle). After these edits, `make` must build with no warning.

Checking the records in gdb. Before any printing exists, gdb can show the first three records: `make debug`, then, in the second terminal, `arm-none-eabi-gdb build/scratch-os.axf`:

```
(gdb) target remote :1234
(gdb) break trace_record_switch
(gdb) continue                  # the 1st switch
(gdb) continue                  # the 2nd
(gdb) continue                  # the 3rd
(gdb) finish                    # let the 3rd record be written
(gdb) p count
(gdb) p buffer[0]
(gdb) p buffer[1]
(gdb) p buffer[2]
(gdb) p sizeof(buffer[0])
```

The reads work because the CPU is stopped inside PendSV, a handler. Output (without the source lines):

```
Breakpoint 1, trace_record_switch (from=0 '\000', to=1 '\001') at ker
Breakpoint 1, trace_record_switch (from=1 '\001', to=0 '\000') at ker
Breakpoint 1, trace_record_switch (from=0 '\000', to=1 '\001') at ker
task_switch_context () at kernel/task.c
$1 = 3
$2 = {tick = 5, from = 0 '\000', to = 1 '\001'}
$3 = {tick = 10, from = 1 '\001', to = 0 '\000'}
$4 = {tick = 15, from = 0 '\000', to = 1 '\001'}
$5 = 8
```

- Switches at ticks 5, 10 and 15 (A → B, B → A, A → B): one per quantum of 5 ticks, as in the first run.
- gdb prints a `uint8_t` as a character: `0 '\000'` is the number 0, that is, A.
- A record takes **8** bytes, not 4+1+1 = 6: the compiler adds 2 bytes of **padding** at the end, so that the `tick` of the next record in the array is aligned to 4. The buffer takes 64 × 8 = 512 bytes of kernel RAM.

1.7.20 Writing the trace: reading and printing the records

Two helpers read the ring oldest first; they go right **above** the comment `/* CLASS: Gantt helpers go here */`, which stays for later:

```
/* How many records are still in the buffer. */
static uint32_t stored( void )
{
    return count < TRACE_SIZE ? count : TRACE_SIZE;
}

/* Record number i, counting from the oldest one (i = 0). */
static const struct trace_record *oldest_plus( uint32_t i )
{
    return &buffer[( count - stored() + i ) % TRACE_SIZE];
}
```

`stored()` is the number of records written, at most 64. Until the ring is full (`count` < 64) the oldest record is number 0, in slot 0; once it is full (`count` ≥ 64), the oldest stored record is number `count` -64, in slot (`count` $- 64$) mod 64 $=$ `count` mod 64, the slot the next record will overwrite. `oldest_plus( 0 )` finds it in both cases, and `oldest_plus( i )` is the record i places after it.

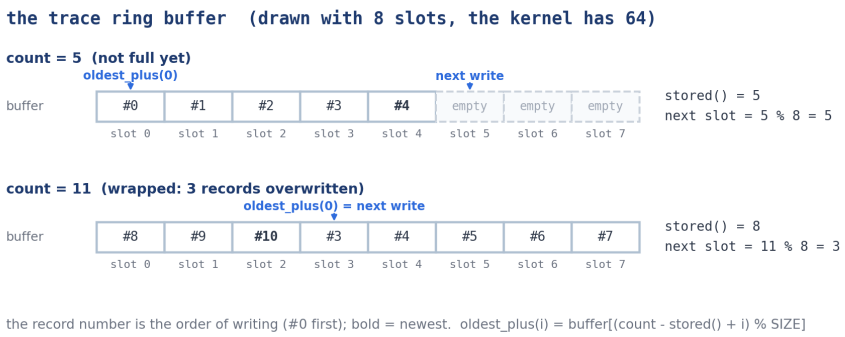


Figure 1.19: The ring buffer, drawn with 8 slots. Before it is full, the oldest record is in slot 0. Once it has wrapped, the oldest record is in the slot that the next write will overwrite, `count % SIZE`: `oldest_plus(0)` finds it in both cases.

In `trace_print`, right **above** the comment `/* CLASS: trace_print body goes here */` (which also stays for later), a first part prints the raw records in the format `schedsim` reads:

```
/* Part 1: the raw records, for the course tool. */
kprintf( "TRACE BEGIN\n" );
for ( uint32_t i = 0; i < stored(); i++ )
{
    const struct trace_record *r = oldest_plus( i );
    kprintf( "T %u %s %s\n", r->tick, tasks[r->from].name,
             tasks[r->to].name );
}
kprintf( "TRACE END %u\n", tick_count );
```

`tasks[]` turns the indexes back into names, which is why the task table is not `static`; `trace_print` runs inside SVCall, privileged, so it uses the kernel's `kprintf`. Now run, save what QEMU prints (`tee` shows the output **and** copies it into a file), quit QEMU with `Ctrl-C` after `all tasks finished`, and draw the result with the course tool:

```
make run | tee run.log
python3 ../../../course-material/tools/schedsim/schedsim.py trace run
```

The path assumes `scratch-os` and `course-material` side by side, as in the course repository; `schedsim trace` reads only the lines between `TRACE BEGIN` and `TRACE END`, ignoring the rest. The end of the run:

```
B: step 15 (tick 44)
TRACE BEGIN
T 5 A B
T 10 B A
T 15 A B
T 20 B A
T 25 A B
T 30 B A
T 32 A B
TRACE END 44
task B exited (tick 44)
all tasks finished
```

and the output of the tool:

```
trace: ticks 5..44 (39 ticks, 7 switches); time 0 = tick 5
time   0    5    10   15   20   25   30   35
    |....|....|....|....|....|....|....|....
A       .....#####.....#####.....##...........
B       #####.....#####.....#####..############
CPU    BBBBBAAAAABBBBBAAAAABBBBBAABBBBBBBBBBBB
schedule=BBBBBAAAAABBBBBAAAAABBBBBAABBBBBBBBBBBB
```

- 7 switches: one every 5 ticks (each `#` block is 5 ticks wide, the quantum), plus `T 32 A B` when A exits in the middle of its slice. Afterwards B's slices end, but B is the only READY task and keeps the CPU, so no record is written. `TRACE END 44` is `tick_count` when B called `trace_dump()`.

- **Does the trace know who ran before the first record? No**: the chart starts at tick 5 (`time 0 = tick 5`), although A ran from tick 0, because the first record is the first information the trace has.

- The switch B → idle is missing: it happens when B exits, **after** B has printed the trace.

1.7.21 Writing the trace: the Gantt chart

Finally the kernel draws the chart itself, one row per task: `/* CLASS: Gantt helpers go here */` (below `oldest_plus`) is **replaced** by a function that says who owned the CPU at a given tick, the width of a chart line, and the drawing function:

```
/* Which task owned the CPU at this tick? It is the `to` of the last
 * record that is not newer than the tick. */
static uint8_t owner_at( uint32_t tick )
{
    uint8_t owner = oldest_plus( 0 )->to;
    for ( uint32_t i = 0; i < stored(); i++ )
    {
        if ( oldest_plus( i )->tick <= tick )
        {
            owner = oldest_plus( i )->to;
        }
    }
    return owner;
}

/* The Gantt chart is cut in lines of at most this many ticks, so it fits
#define GANTT_COLUMNS 64

/* Draw the Gantt chart from tick `first` up to (not including) `now`:
 * one row per task, one character per tick. */
static void draw_gantt( uint32_t first, uint32_t now )
{
    for ( uint32_t start = first; start < now; start += GANTT_COLUMNS )
    {
        /* This line shows ticks start .. end - 1. */
        uint32_t end = start + GANTT_COLUMNS;
        if ( end > now )
        {
            end = now;
        }

        /* The ruler: the last digit of every tick number. */
        kprintf( "\n tick " );
        for ( uint32_t t = start; t < end; t++ )
        {
            kprintf( "%u", t % 10 );
        }
        kprintf( "\n" );

        /* One row per task: '#' while it runs, '.' otherwise. */
        for ( unsigned i = 0; i < NUM_TASKS; i++ )
        {
```

```
            /* The name, then spaces up to 6 characters, so the rows
             * line up (kprintf has no field width). */
            unsigned len = 0;
            while ( tasks[i].name[len] != '\0' )
            {
                len++;
            }
            kprintf( " %s", tasks[i].name );
            for ( ; len < 5; len++ )
            {
                kprintf( " " );
            }
            for ( uint32_t t = start; t < end; t++ )
            {
                kprintf( "%c", owner_at( t ) == i ? '#' : '.' );
            }
            kprintf( "\n" );
        }
    }
}
```

`owner_at` scans the records from the oldest and keeps the `to` of the last one whose tick is not after the given tick: of two records with the same tick, the later one wins. The outer loop of `draw_gantt` cuts a long chart into lines of at most 64 ticks; the ruler shows only the last digit of each tick number, so that every tick takes one character; each row is the task name, padded with spaces by hand (`kprintf` has no field width), then one character per tick: `#` if the task owned the CPU, `.` otherwise. The chart closes the dump: in `trace_print`, the comment `/* CLASS: trace_print body goes here */` (below the `TRACE END` line) becomes:

```
    /* Part 2: the chart, from the first recorded switch to now. */
    if ( stored() > 0 )
    {
        draw_gantt( oldest_plus( 0 )->tick, tick_count );
    }
```

Now `kernel/trace.c` is the same as `week-05/solution/kernel/trace.c` (if the build fails, compare the two files). With `make run`, the end of the output is:

```
T 30 B A
T 32 A B
TRACE END 44

 tick 56789012345678901234567890123456789012345678901234567890123
 A    .....#####.....#####.....##..........#####.....#####.....#####
 B    #####.....#####.....#####..##########.....#####.....#####.....
 idle ..........................................................
task B exited (tick 44)
all tasks finished
```

- The ruler shows the last digit of each tick, from tick 5 (the first record) to 43: the same chart as `schedsim`'s, one row per task, with the idle task as its own

row; each # block of A and B is 5 ticks wide, the quantum, until A exits at tick 32.

- **Why does `trace_print` check `stored() > 0`?** With no record at all, `oldest_plus( 0 )` would point at a slot never written, and `owner_at` would draw a task that never ran. With no switch, there is no chart to draw.

1.7.22 Try it: the trace of the other runs

The trace can check the quantum and the idle task. Each build has its own folder, so both are rebuilt with the new `kernel/trace.c` (quit QEMU with `Ctrl-C` after `all tasks finished`):

```
make QUANTUM=2 run
make DEMO=idle run
```

For `DEMO=idle`, the output can also be saved with `| tee idle.log` and drawn with `schedsim.py trace idle.log`. The charts:

```
(QUANTUM=2)
tick 23456789012345678901234567890123456789012345678901234567890123456789
A    ..##..##..##..##..##..##..##..##..##..##..##..##..##..##..##..##..##..
B    ##..##..##..##..##..##..##..##..##..##..##..##..##..##..##..##..##..##
idle ................................................................................

(DEMO=idle)
T 1 A B
T 1 B idle
...
tick 12345678901234567890123456789012345678901234567890123456789012345678901
A    ..........#..........#..........#..........#..........#..........#.....
B    ......#.....#.....#.....#.....#.....#.....#.....#.....#.....#.....#.....
idle ######.###.#.#####.##.##.#####.#####.###.#.#####.##.##.#####.#####.###.#
```

- **With a quantum of 2, every # block is 2 ticks wide**: the chart matches the configured quantum. It starts at tick 2, the first switch.

- **With `DEMO=idle`, the idle task runs whenever A and B are both BLOCKED**, which is most of the time: idle owns most ticks. A runs at 11 and 22, B every 6 ticks (7, 13, 19, 25, 31).

- At tick 1 two records share the same tick: A blocks (A → B), and B blocks at once in its first `task_delay_until` (B → idle); the chart shows the last one, idle. The same happens at tick 40 with a quantum of 2 (B → A, then A exits at once, A → B), which is why A's last turn does not show.

- The `DEMO=idle` chart ends at tick 31, because B prints it right after its last step: A's steps at 33 and 44 come later.

1.8 Unit 2: summary

- A context switch saves the old task's registers on its own stack and keeps only its stack pointer, in the first field of its TCB (`struct tcb`, `sp` at offset 0): the CPU pushes `r0`–`r3`, `r12`, `lr`, `pc`, `xPSR`; the kernel pushes `r4`–`r11`.
- PendSV does every switch, pended through `ICSR.PENDSVSET` by SysTick and by the system calls; with the least urgent priority (`0xE0`, shared with SysTick) it runs, tail-chained, only when no other handler runs, and always interrupts a task.
- A task that never ran gets a fake frame (`task_init_stack`), so its first switch is like every other one.
- `task_switch_context` picks the next READY task in a circle (round robin), falls back to the idle task, and moves MPU region 1 to the new task's partition: week 4's isolation holds after every switch.
- SysTick counts `tick_count`, wakes BLOCKED tasks and ends the slice after `QUANTUM` ticks; `QUANTUM = 0` is cooperative, and a task that never yields starves the others.
- `task_yield`, `task_delay` (drifts) and `task_delay_until` (no drift) give the CPU away; exit makes a task FINISHED; with nothing else to run, the idle task sleeps with `wfi`.
- The trace records `{tick, from, to}` in a ring buffer at each switch and prints it with a Gantt chart, which `schedsim trace` also draws.

1.9 Unità 3 - Real-time basics

From this lecture to the end of next week the topic is **real-time scheduling**: a step beyond the general scheduling of previous OS courses, somewhat more complex but very important for embedded systems, where many applications have real-time constraints.

1.9.1 Real-time systems

An ordinary computer gives *no guarantee* that a response arrives within a given time: a program slow to load or a video that lags a little is annoying, nothing more. In a **real-time system** the execution time is a strong **constraint**: a program must run *within a given time*.

- Most programs only have to be **correct**: for an input x arriving at time t, the right output y may come at any later time, as long as it comes.
- A real-time system must be **correct and on time**: reacting to an event from the outside world, it must deliver a correct y no later than $t + d$, with d, the **deadline**, fixed in advance.
- The right result too late is as bad as a wrong result, or no result: a correct result after the deadline is considered **incorrect**.

Example: braking. In modern cars a computer controls braking: when the driver presses the pedal, braking must start within a certain time, or a perfectly correct command arrives when the car has already crashed into a wall, another car or a pedestrian. The brake controller reads the wheel speed and must open or close the brake valve within a few milliseconds: a perfect command 100 ms later is useless. Most embedded systems are real-time, at least in part, because they control something physical (a motor, a valve, a display, a radio) that does not wait.

1.9.2 Hard, firm and soft deadlines

Not every late result is equally bad: deadlines (and real-time systems) are usually classified, not very strictly, into three categories, with constraints more relaxed from hard to soft:

deadline	if it is missed	examples
hard	catastrophic consequences on the system or its environment	airbag trigger, brake control, pacemaker pulse, fly-by-wire
firm	the result is useless and is thrown away, but no serious damage	one frame of a live video call, one position fix of a car navigator
soft	the result is still useful, only worse; meeting the deadline is desirable	keyboard echo, a web page loading, the clock on a smartwatch screen

Examples from class: **hard**, the control system of an airplane, which may crash if a deadline is missed; **firm** (the constraints still matter a lot, but a late result is simply **thrown away**), a video played at a given frame rate, which stops processing a late frame and moves to the next one, or a phone compressing a video in real time while recording, which discards a frame that is too late, with a slightly lower but acceptable quality; **soft** (meeting the deadline is **desirable**, a late result is **still useful**, only worse), a smartwatch refreshing the time every second, where a slightly late refresh is not good, but acceptable.

A **hard real-time system** has at least one hard task, and its designers must **prove**, before it runs, that every hard deadline is met: testing is not enough, since it shows that deadlines were met only in the cases tried. One system often mixes all three classes: a car has hard brake control, firm sensor fusion and a soft infotainment screen. The course focuses on **hard real-time embedded systems**: a "real-time task" has a hard deadline unless stated otherwise. **Desktop OSs are not real-time**: they give no timing guarantee and fit none of the three categories. Linux has a real-time version of its kernel (real-time Linux), but it is still not a *hard* real-time OS: that requires a specific **real-time OS** (RTOS), generally very small and imposing many constraints, because being strictly real-time means accepting limitations.

1.9.3 Exercise: hard, firm or soft?

Classify each deadline as hard, firm or soft, and say why in one line:

1. Fire the airbag within 30 ms of a crash being detected.

2. Decode and show the next frame of a live video stream within 40 ms.
3. Update the step counter on a fitness band's display after each step.
4. Adjust a drone's motor speeds within 2 ms of a new gyroscope reading.
5. Read one sample of an audio stream before the next sample arrives; a late sample is skipped.
6. Underline a misspelled word in a text editor within half a second.

Solution.

task	deadline	why
1. airbag	hard	too late, the passenger has already hit the wheel: injury
2. video frame	firm	a late frame is dropped; the video jumps, nobody is hurt
3. step counter	soft	a late update is still correct and still useful, just slower
4. drone motors	hard	late corrections make the drone unstable: it can crash
5. audio sample	firm	a skipped sample is lost (a small click), no damage
6. spell check	soft	the underline is still useful a little later

More context: an airbag's specification requires a reaction within a few milliseconds, and to be certified the system must guarantee its response time, so it needs real-time software; missing one video frame is acceptable, but the missed frame must be discarded; a step counter (e.g. a smartwatch's) may be updated a little late, if the accelerometer data are processed late; the software controlling a drone's stability and motors is hard real-time, since if it does not respond in time the drone will probably crash; wireless headphones contain a small microcontroller that processes the audio, and a few lost samples are probably not even noticed; a spell checker (e.g. while typing on a phone) should respond while typing, but a little lag is not a problem. The class depends on the **consequence of a miss**, not on how short the deadline is: 30 ms (airbag) is hard, 40 ms (video) is firm. Some answers depend on the system: the video of a remote surgery robot is much more than firm.

1.9.4 Real-time is not the same as fast

Real-time and fast are completely different things, and real-time systems are most of the time *slower* than general-purpose ones. A **fast** system optimises (minimises or maximises) an **average** metric over a set of tasks: throughput, computation time, Unit 1's average turnaround or waiting time. A **real-time** system **guarantees the timing of each task**: every job must meet its own deadline, even in the worst case, which sometimes requires *slowing down* some task. A low average says nothing about one task (one late job can hide behind many early ones), and a faster CPU helps but guarantees nothing: what matters is which task runs when, the **scheduling**. A real-time system values **predictability**, computing in advance when each task finishes: think of it not as a system that goes fast, but as one where

everything that runs has been studied and every task can be **proved** to complete within a given time; if the analysis shows that this is impossible with the available processor, a different (maybe faster) one is needed.

Example. Three jobs arrive at time 0: A ($C = 4$) must finish by 5, B and C ($C = 1$ each) by 10. They are scheduled with preemptive priority scheduling (a typical real-time algorithm), with two choices of priority.

Fast: short jobs first (B 3, C 2, A 1), as the theory suggests to maximise speed; B and C have the same duration, so the identifier breaks the tie and B gets the higher priority. Schedule: `BCAAAA`.

```
time  0     6
      |.....|
A     ..####      <- finishes at 6 > 5: deadline miss
B     #.....
C     .#....
CPU   BCAAAA
```

Turnaround: B = 1 (from 0 to 1), C = 2, A = 6; average $(1+2+6)/3 = 3$. Fast on average, but A finishes at $6 > 5$: **deadline missed**, and in a hard real-time system the car has crashed.

On time: urgent job first (A 3, B 2, C 1): A has the earliest deadline (5), B and C both 10, and B again goes before C. Schedule: `AAAABC`.

```
time  0     6
      |.....|
A     ####..
B     ....#.
C     .....#
CPU   AAAABC
```

Turnaround: A = 4, B = 5, C = 6; average $(4+5+6)/3 = 5$: slower on average, but A finishes at $4 \leq 5$ and B and C by 10: **all deadlines are met**, and the car does not crash. When we care about real time, we do not really care about (average) performance.

1.9.5 Periodic, sporadic and aperiodic tasks

Unit 1's two task models (endless loop, run to completion) become really important in real-time systems, refined into three types. A real-time task is an endless sequence of **jobs** (**instances**), each one execution of the task's work, like one turn of the loop of Unit 1's `task_b`.

```
periodic    ^         ^         ^         ^         ^      T = 25 ms
            0         25        50        75        100
sporadic    ^      ^               ^    ^            ^      gap >= 10 ms
aperiodic   ^^  ^         ^ ^           ^                   any gap
                                                  ^ = a job is released
```

- **Periodic** (**time-driven**): a job is released at a constant rate, every **period** T (e.g. every 25 ms), and each iteration of the task's loop must complete before the next activation (the deadline could be shorter than the period, but in the course exercises it is always the **end of the period**). Very important, because it defines a problem that can be **solved mathematically**: with particular algorithms, one can compute on paper whether a task set can be scheduled without missing any deadline. The activations follow a **pattern** and can be predicted, so these tasks can be put in an embedded system with a **proof** that, if the design was done properly, all deadlines will be met whatever happens.

- **Sporadic**: activated many times, but at times **not known**, by events. Without a known pattern nothing can be proved on paper: the algorithms can only try to **minimise the probability** of a deadline miss, without an assurance. Two consecutive jobs are, however, always at least a **minimum inter-arrival time** apart.[3]

- **Aperiodic**: released by events at irregular times, with **no known minimum gap**: in theory two can arrive at the same instant. This is the only difference from sporadic tasks, whose activations are guaranteed to be far enough apart.

Each category needs different scheduling algorithms. **Design consequence**: in, e.g., the landing system of an airplane, tasks that must *never* miss a deadline should be modelled as **periodic** whenever possible, so that calculations can prove that the system always reacts in time; if the tasks really cannot be activated periodically, scheduling enters a more **probabilistic** realm, where real-time behaviour can be maximised but not guaranteed 100%.

Examples from an engine controller. **Periodic**: measure the intake air temperature every 25 ms, a job released by a timer whatever the engine does. Periodic tasks fit very well the acquisition of sensor measurements by **polling**: we decide that reading a temperature sensor every 25 ms, and reacting, is fine. **Sporadic**: compute the combustion data once per crankshaft turn, a job released by the crankshaft sensor, so the gap depends on the engine speed:

engine speed	**time of one turn = gap between two jobs**
1000 RPM	$60\,s/1000 = 60\,ms$
4000 RPM	$60\,s/4000 = 15\,ms$
6000 RPM (the maximum)	$60\,s/6000 = 10\,ms$: the minimum inter-arrival time

Aperiodic: react to the driver pressing a dashboard button: nobody knows when, or how often. Event-triggered tasks are a design choice: **polling** for an event periodically turns the task into a periodic one, but is not computationally efficient; **interrupts** are a much better mechanism, at the price of **unpredictability**, which makes the system less safe from the real-time point of view.

[3] From the slides: in the worst case, a sporadic task behaves like a periodic task with T equal to that minimum inter-arrival time. This is what makes sporadic tasks a useful middle ground: event-driven like aperiodic tasks, but with a worst case that can be planned for, like periodic tasks.

1.9.6 The task model

To compute in advance whether every deadline is met, we assume a simple model:

- **a fixed number of tasks, all known in advance**, chosen once and never changed, exactly as in our scratch-OS (unlike general-purpose OSs, where a task can be started at any time from a terminal);
- **every task is periodic**, with a known period T: the simplest situation, the focus of this lecture and of part of next week;
- **the tasks are independent**: no shared resources, no waiting for each other (no precedence), i.e. executing a task never requires waiting for another task to complete a computation. A very strong constraint, which in reality must sometimes be relaxed;
- **each task has a hard deadline equal to its period**: a job released at the beginning of its period must finish by its end, before the next job of the same task is released ($D = T$, always, in this course: the simplest of several ways to assign deadlines);
- **each task has a known, fixed worst-case execution time** C: very important, since all the calculations rely on knowing how long each task needs, in the worst case, for one execution;
- **all the kernel's overheads** (e.g. the context switch) are **negligible**, i.e. zero: the tasks must be at least an order of magnitude longer than the overhead (if a switch takes a few microseconds, tasks with a period of a few microseconds cannot be scheduled);
- **one CPU**, with time in whole units, as in Unit 1. Embedded real-time systems do use multi-core CPUs nowadays, but in a "simple" way: every core is an **independent CPU**, with its own OS instance scheduling its own tasks, *statically* assigned to it (e.g. 10 tasks on a dual-core CPU: five per core, or any other split). Moving tasks between cores, as Linux does on our PCs, is too unpredictable for real-time systems.

The algorithms of this unit work *if* all these constraints hold, and no longer work if any of them is removed. Real systems break some of these rules: the model is the starting point for a guarantee, not a description of every system.

1.9.7 Timing parameters and notation

In the figures, an **up arrow** marks the **release** (activation) of a job and a **down arrow** its **deadline**; with $D = T$, every new release coincides with the deadline of the previous job of the same task (very important). For a task A, with jobs A_1, A_2, ... (A_k is job k):

- **release** (arrival, activation) time a: the job becomes READY. Job k is released at

$$a = \Phi + (k - 1) \times T$$

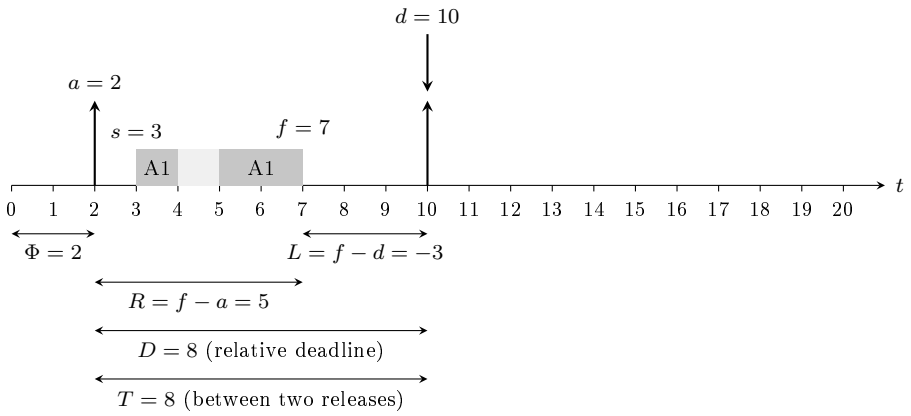


Figure 1.20: Task A: phase $\Phi = 2$, period $T = 8$, $C = 3$ $(1 + 2)$. Its first job is released at 2, is preempted once (light grey: another task runs), and finishes at 7, three units before its deadline 10. The next job is released at 10, then at 18, and so on.

- **phase** Φ: the release time of the first job; usually all tasks start at 0 ($\Phi = 0$ if not given), but some systems must start a task a bit later;
- **computation time** C: the CPU time the job actually uses, taken in its worst case, the **WCET** (in the figure, A runs 1 unit and then 2, because another task runs in between: $C = 3$);
- **period** T, the time between two releases; **relative deadline** D, the time the job has after its release ($D = T$ here); **absolute deadline** $d = a + D$, the instant by which the job must be finished (with $D = T$, the next release);
- **start time** s and **finish time** f; careful: in general $f \neq s + C$, because the job can be preempted (in the figure, $s + C = 6$ but $f = 7$);
- **response time** $R = f - a$, from release to end (in the figure, $7 - 2 = 5$); a job meets its deadline when $R \leq D$;
- **lateness** $L = f - d$: negative = early, 0 = just in time, positive = deadline missed (in the figure, $7 - 10 = -3$). In a hard real-time system every job must always have $L \leq 0$.

Unit 1 called $f - a$ the *turnaround time* and $s - a$ the *response time*; real-time systems call $f - a$ the response time, and from here on "response time" means $f - a$.

1.9.8 Utilisation

Periodic task sets can be analysed mathematically, and the first metric needed is the **utilisation factor**. A task needs C units of CPU in each period T, so it uses the fraction C/T of the CPU, its execution time normalised to its period (between 0 and 1); for a task set:

$$U = \frac{C_1}{T_1} + \frac{C_2}{T_2} + \cdots + \frac{C_n}{T_n} = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

- $U > 1$: the tasks need more CPU time than exists, and no algorithm can meet all deadlines on one CPU: the task set cannot be scheduled.
- $U \leq 1$ is **necessary but not sufficient**: the set can potentially be scheduled, but an algorithm may still miss deadlines (examples with timeline scheduling, below). It would be sufficient only for a *perfect* algorithm, able to use the CPU at 100%: one exists, and will be studied, but it is complex, so in practice simpler schedulers are often preferred, paying for simplicity with non-optimality: they may fail even with $U \leq 1$, **under-utilising** the CPU.

In practice, check U first, even without knowing the algorithm: with $U > 1$, drawing a Gantt chart or applying an algorithm is a waste of time. At the exam, if $U > 1$, the following questions about the schedule need no answer: the problem cannot be solved.

In real life: what if $U > 1$? The customer's product must schedule tasks with known execution times, periods and deadlines, and $U > 1$. In each ratio C/T, the **period** comes from the requirements and cannot change (e.g. the airbag must react within 1 ms); the **computation time** can be reduced with better (more efficient) software, but only up to a limit, beyond which even the best possible designer cannot make the code faster. The usual solution is to **buy a faster CPU**: cost is important in embedded systems, so the design starts with a cheap CPU (e.g. an Arduino-class microcontroller costing a few euros) and, if the tasks do not fit, moves to a slightly more powerful CPU that costs more but fits them.

1.9.9 Hyperperiod

Each periodic task repeats with its own period, and the activations of all the tasks together repeat too, with period the **hyperperiod** H, the **least common multiple** (LCM) of the periods:

$$H = \text{LCM}(T_1, T_2, \ldots, T_n)$$

With all phases 0, every task releases a job at 0 and again at H, so it is enough to draw the schedule from 0 to H, in which task i has exactly H/T_i jobs. **Example**: A ($C = 2, T = 5$), B ($C = 2, T = 10$), C ($C = 2, T = 20$):

$$U = \frac{2}{5} + \frac{2}{10} + \frac{2}{20} = \frac{8}{20} + \frac{4}{20} + \frac{2}{20} = \frac{14}{20} = \frac{7}{10} = 0.700$$

The CPU is busy 70% of the time.

$$H = \text{LCM}(5, 10, 20) = 20$$

Whatever the schedule of the three tasks, it repeats every 20 time units: in each hyperperiod, A has 4 jobs, B has 2, C has 1.

Two ways to check schedulability. **Mathematically**, for the algorithms of this course that come with a mathematical foundation (in practice, a formula to apply); or **empirically**, the only option for algorithms without a mathematical test: apply the algorithm and draw the Gantt chart on paper for one hyperperiod, since an algorithm that meets all deadlines in $[0, H)$ will always meet them.

1.9.10 Why the worst-case execution time?

```
void task_control( void )                    /* one job */
{
    read_sensors();                          /* always:
40 us */
    if ( new_target_received() )             /* only sometimes:
*/
        recompute_trajectory();              /*
300 us */
    if ( error_detected() )                  /* rarely:
*/
        log_error_to_flash();                /*
900 us */
    write_actuators();                       /* always:
20 us */
}
/* usually 60 us; worst case 40 + 300 + 900 + 20 = 1260 us */
```

A task has several execution paths, chosen by its input, each with its own time, and caches add more variation. A guarantee must hold for every job, so it uses the **WCET**, the longest time over all inputs and states: planning with the average (say $100\,\mu s$), the rare job that takes $1260\,\mu s$ would miss its deadline exactly when an error occurs.

Estimating the WCET is hard. Some researchers have worked on it for 30 years: which side of a branch is taken depends on the values of variables, and therefore on the program's inputs, so for a complex program (say, a million assembly instructions) finding the path with the longest execution time **cannot be solved exactly**. In general, the WCET comes from measuring the longest run plus a margin, or from analysing the code: safe but pessimistic. The practical solution is **statistical**: run the program many times with different inputs, measure each execution time, build the distribution (the probability of each execution time) and estimate the WCET from it; but the inputs are usually random, and it is not known whether they are representative of the real application, so much research aims at better estimates. In this course the WCET is always **given**, but for an engineer designing a car's braking system computing it is a real problem, and when safety really matters it is **overestimated**: a measured execution time around 10 becomes 30 or 40 (three or four times larger), to be sure no deadline is missed. The price of overestimating is **wasted CPU time**.

1.9.11 Exercise: notation

A typical exercise gives a set of tasks with their periods and execution times; if nothing is said, all tasks are activated at time 0 ($\Phi = 0$). Three periodic tasks, all with phase 0 and $D = T$:

task	C	T
A	2	4
B	1	8
C	1	12

1. Compute U, as a fraction and with three decimals.
2. Compute the hyperperiod H. How many jobs of each task are released in $[0, H)$?
3. Job B_2 starts at 10 and finishes at 11. Compute its a, d, R and L.
4. Job C_2 starts at 14 and finishes at 15. Compute its a, d, R and L.
5. If C had phase $\Phi = 3$ instead, when would C_2 be released, and what would its absolute deadline be?

Solution.

1. $U = \frac{2}{4} + \frac{1}{8} + \frac{1}{12} = \frac{12}{24} + \frac{3}{24} + \frac{2}{24} = \frac{17}{24} = 0.708$
2. $H = \text{LCM}(4, 8, 12) = 24$. Jobs in $[0, 24)$: A $24/4 = 6$, B $24/8 = 3$, C $24/12 = 2$.
3. B_2: $a = 0+1\times 8 = 8$, $d = 8+8 = 16$, $R = 11-8 = 3$, $L = 11-16 = -5$.
4. C_2: $a = 0 + 1 \times 12 = 12$, $d = 12 + 12 = 24$, $R = 15 - 12 = 3$, $L = 15 - 24 = -9$.
5. With $\Phi = 3$: C_2 is released at $a = 3 + 1 \times 12 = 15$, and its absolute deadline is $d = 15 + 12 = 27$.

The start and finish times come from this schedule, built with the timeline method (next section). Both jobs finish early: negative lateness.

```
time  0          10         20
      |.........|.........|...
A     ##..##..##..##..##..##..
B     ..#.......#.......#.....
C     ...#.........#.........
CPU   AABCAA..AAB.AAC.AAB.AA..
```

1.10 Timeline scheduling

The first real-time algorithm we study, **timeline scheduling** (or **cyclic executive**), is the worst in terms of efficiency: definitely not optimal, and without a mathematical foundation to check whether a task set can be scheduled (the only way is to try to build the Gantt chart). But it is very simple to implement, and used in many real products.

1.10.1 The idea and the rules

It suits simple embedded applications, a few periodic tasks that do not interact (e.g. reading sensors through one shared ADC and updating outputs), which do not need a full operating system.

- The whole schedule is computed **offline**, before the system runs: the algorithm is applied on paper to decide the order and the activation time of every job, and the result is encoded as a fixed table. At run time no scheduler decides anything: the program follows the table ("this task starts at 0 and finishes here, then the next one runs, then the next one").
- The time axis is divided into slots of equal length, the **minor cycles** (or **frames**): minor cycle m = the **greatest common divisor** (GCD) of the periods, so that every period is a whole number of frames and every release falls on a frame boundary. A timer interrupt marks the start of every frame, and each frame runs a fixed list of jobs, one after the other, without preemption.
- The **major cycle**, the sequence of frames that then repeats identically, is M = the LCM of the periods = the **hyperperiod**: it is enough to fill the M/m frames of one major cycle, and if all tasks fit, the schedule always repeats in the same way, starting again from frame 0.

The course rules to build the table:

- Go through the frames in order. At the start of each frame, take the jobs already released and not yet run, and place them **shortest period first** (the most urgent task first; equal periods: lowest identifier first), each right after the previous one.
- A job is placed only if it **fits completely** in the time left in the frame: a job **cannot cross a frame boundary**. If it does not fit, it waits for a later frame, and the next jobs in the list are still tried.
- A placed job runs to the end without preemption; the rest of the frame is idle. Check per frame: the sum of the C of its jobs is at most m.
- The table is **feasible** if every job is placed in a frame that ends by its deadline; a job that finds no such frame is a **deadline miss**.

The algorithm does not say *where* in a frame a job goes (at the beginning, at the end, ...): the course convention, used by `schedsim` and at the exam, is always the **first available slot**, so that every exercise has a single solution.

1.10.2 Timeline: worked example

The tasks of the hyperperiod example, all with the same execution time: A ($C = 2, T = 5$), B ($C = 2, T = 10$), C ($C = 2, T = 20$).

$$U = \frac{7}{10} = 0.700, \qquad m = \mathrm{GCD}(5, 10, 20) = 5, \qquad M = \mathrm{LCM}(5, 10, 20) = 20$$

Frames of 5 units; the hyperperiod is 20, so 4 frames are enough.

frame	jobs	used	
frame 0 $[0, 5)$	A_1 B_1	4/5	C_1 does not fit (1 left): waits
frame 1 $[5, 10)$	A_2 C_1	4/5	
frame 2 $[10, 15)$	A_3 B_2	4/5	
frame 3 $[15, 20)$	A_4	2/5	

```
time  0    5    10   15   20
      |....|....|....|....|
A     ##...##...##...##...
B     ..##........##......
C     .......##...........
CPU   AABB.AACC.AABB.AA...
```

Intuitively, the most urgent task, A (period 5, equal to the minor cycle), gets one job in every frame, B one every two frames, and C one per major cycle, in the first frame where it fits without crossing a boundary. With the course rules: in frame 0 $[0, 5)$, A_1, B_1 and C_1 are all released; shortest period first, A_1 (2 units, 3 left) and B_1 (2 units, 1 left) are placed, but C_1 needs 2 and would cross the frame boundary, so it waits. In frame 1 $[5, 10)$, A_2 (released at 5, shorter period) goes before the waiting C_1: 4 units used. Frame 2 holds A_3 and B_2, released at 10; frame 3 only A_4. Every job is in a frame that ends by its deadline: the table is **feasible**, and C_1 (deadline 20) simply runs one frame after its release.

Why it is not optimal. (From a question in class: what if the tasks had $C = 4$?) Longer execution times may give $U > 1$, which is impossible, but also $U < 1$ with a task set that still cannot be scheduled: the rule that forbids crossing a frame boundary creates **holes** where the CPU is idle, and that idle time is wasted and can never be recovered. Timeline scheduling is a perfect example of a non-optimal algorithm, for which $U \leq 1$ is not enough (examples later in this section).

1.10.3 How a cyclic executive is implemented

The implementation is super simple: the scheduler is basically an infinite loop, with one function per minor cycle (four here) that calls the tasks of its frame in the order of the table:

```
/* A (C=2, T=5), B (C=2, T=10), C (C=2, T=20): each task is a function th
static void frame_0( void ) { task_a(); task_b(); }
/* one function per minor cycle */
static void frame_1( void ) { task_a(); task_c(); }
static void frame_2( void ) { task_a(); task_b(); }
static void frame_3( void ) { task_a(); }

static volatile int frame_start;              /* set by the interrupt: vol

void SysTick_Handler( void ) { frame_start = 1; }
/* timer: every minor cycle (5 ms) */

static void wait_frame( void )                /* idle until the next frame
```

```
{
    while ( frame_start == 0 ) { }
    frame_start = 0;
}

int main( void )
{
    timer_start_ms( 5 );                    /* program the timer to the m
    for ( ;; )                              /* one turn of this loop = or
    {
        wait_frame(); frame_0();
        wait_frame(); frame_1();
        wait_frame(); frame_2();
        wait_frame(); frame_3();
    }
}
```

- Each task is a plain function that runs one job and returns; the major cycle is an endless loop that calls the frame functions in order.

- The timer fires once per minor cycle, and `wait_frame` waits for it: when the jobs of a frame finish early, the loop waits for the beginning of the next frame, so each frame starts exactly on its boundary. `frame_start` is `volatile` because the interrupt changes it: without `volatile`, the compiler could read it once and loop forever on the old value.

- No context switch, no TCB, no ready queue: all tasks run on one stack, inside one loop.

- If a frame overruns (its jobs take longer than m), the flag is already set when `wait_frame` is called: a robust executive checks this and reports the error.

- Changing a task's C or T means recomputing the table and rewriting the frame functions.

With the order fixed on paper and always repeated, this scheduler is **extremely predictable**: this is why it is used in many safety-critical real-time applications, where certification needs a fixed, proven schedule, with periods chosen to fit it. In avionics basically every product uses it: aircraft fly with this algorithm, wasting CPU power but being super predictable. (The professor works on research contracts with automotive and avionics companies that are only now planning to introduce more advanced algorithms, in products that will reach the market in about ten years.)

1.10.4 Timeline scheduling: pros and cons

pros	cons
very simple: a loop, a timer, a table; no kernel needed	fragile: a new task or a changed C or T can change the whole table
very predictable: each job starts at a known time in its frame, with almost no variation	a job longer than a frame never fits: it must be split by hand
no preemption: no shared-data conflicts, no context-switch cost	aperiodic events must wait for a frame that checks for them (polling)
the schedule is checked offline, once, before the system runs	one job that overruns delays every following frame
	non-harmonic periods give short frames and huge tables

Everything is checked offline, on paper, where the wasted CPU time is visible as idle slots: it is accepted in exchange for full predictability. **Fragility**: every control unit (ECU) in a car performs many activities, and a new task added by a firmware upgrade of the engine ECU can completely break the schedule. **Aperiodic events** are hard to fit efficiently in this mechanism. **Overruns**: the WCET is known in theory, but in practice there is always a small chance that a task runs longer than expected, and then the whole schedule is broken.

1.10.5 Non-harmonic periods

Everything looks perfect on paper when the periods are **harmonic**, i.e. each period divides every longer one, as in the example (5, 10, 20): then the GCD is the shortest period and the LCM the longest one. Periods that are not harmonic (e.g. chosen at random) cause **short frames**, because the GCD can be much smaller than every period (with $T = 5, 7, 11$ it is 1: a frame lasts one unit, and no job with $C \geq 2$ fits in any frame), and **huge tables**, because the LCM grows fast (with $T = 5, 7, 11$, $5 \times 7 \times 11 = 385$ frames to store, for three tasks: the system is still predictable, but the pattern repeats only after a long time). Two ways out:

- **Split** a long task into parts that each fit in a frame (e.g. D with $C = 4$ becomes D with $C = 2$ and E with $C = 2$, same period). The parts must still run in order: give the first part the lower identifier, and check the order in the chart.
- **Shorten some periods** until they are harmonic: running a task more often than needed is always allowed, but U grows. E.g. periods 2, 3 and 8 are not harmonic, but running the second task every 2 instead of every 3 gives 2, 2, 8, which are. This restricts the deadline of the second task: a CPU that could handle the original task set may no longer be enough, and a faster one is needed.

So timeline scheduling is simple, but far from perfect: CPU time is wasted both in the idle holes and in artificially shortened periods.

Example. A ($C = 1, T = 4$) and B ($C = 3, T = 6$): $U = 1/4 + 3/6 = 0.750$; GCD = 2, LCM = 12. **As given**, frames of 2 units: schedule `A...A...A...`, because B needs 3 units but a frame has 2, so B **never fits**, and B_1 misses its deadline at 6:

not feasible, even though U is only 0.75. **With B's period shortened to 4**, the periods are harmonic, GCD = LCM = 4: one frame, `ABBB`, **feasible**; B now runs every 4 instead of every 6 units, and $U = 1/4 + 3/4 = 1$: the CPU is never idle.

1.10.6 Exercise: timeline scheduling

Four periodic tasks, phase 0, $D = T$: A ($C = 1, T = 4$), B ($C = 2, T = 8$), C ($C = 1, T = 8$), D ($C = 2, T = 16$).

1. Compute U, the minor cycle and the major cycle.
2. Fill the frames with the course rules; in each frame, write the jobs and the time used.
3. Draw the Gantt chart and write the schedule string. Is the timeline feasible?
4. Compute R and L of job D_1.

Solution.

$$U = \frac{1}{4} + \frac{2}{8} + \frac{1}{8} + \frac{2}{16} = \frac{4}{16} + \frac{4}{16} + \frac{2}{16} + \frac{2}{16} = \frac{12}{16} = \frac{3}{4} = 0.750$$

Minor cycle = GCD(4, 8, 8, 16) = 4; major cycle = LCM(4, 8, 8, 16) = 16: 4 frames.

frame 0 $[0, 4)$	A_1 B_1 C_1	4/4	D_1 does not fit (0 left): waits
frame 1 $[4, 8)$	A_2 D_1	3/4	
frame 2 $[8, 12)$	A_3 B_2 C_2	4/4	
frame 3 $[12, 16)$	A_4	1/4	

```
time  0         10
    |.........|.....
A     #...#...#...#...
B     .##......##.....
C     ...#.......#....
D     .....##.........
CPU   ABBCADD.ABBCA...
```

Feasible. In frame 0 all four jobs are released; B and C have the same period, so B goes first (lowest identifier): A_1, B_1, C_1 fill the 4 units, and D_1 waits. In frame 1, A_2 (released at 4) goes before D_1 (shorter period), and D_1 runs from 5 to 7. B_2 and C_2 are released at 8, in frame 2. D_1: $a = 0$, $s = 5$, $f = 7$, $d = 16$; $R = 7 - 0 = 7$, $L = 7 - 16 = -9$.

1.10.7 Exercise: a frame too small

Four periodic tasks, phase 0, $D = T$: A ($C = 2, T = 6$), B ($C = 3, T = 12$), C ($C = 2, T = 12$), D ($C = 4, T = 24$).

1. Compute U, the minor cycle and the major cycle. Does U allow a feasible schedule?

2. Fill the frames, draw the Gantt chart and write the schedule string. Is the timeline feasible? If not, which job misses, and when?

3. Split D into two parts, D ($C = 2$) and E ($C = 2$), both with $T = 24$. Build the new table. Is it feasible, and do D and E run in the right order?

Solution (1, 2).

$$U = \frac{2}{6} + \frac{3}{12} + \frac{2}{12} + \frac{4}{24} = \frac{8}{24} + \frac{6}{24} + \frac{4}{24} + \frac{4}{24} = \frac{22}{24} = \frac{11}{12} = 0.917 \ (\leq 1)$$

Minor cycle = GCD(6, 12, 12, 24) = 6; major cycle = LCM(6, 12, 12, 24) = 24: 4 frames.

frame 0 $[0, 6)$	A_1 B_1	5/6	C_1 (2) and D_1 (4) do not fit in 1
frame 1 $[6, 12)$	A_2 C_1	4/6	D_1 (4) does not fit in 2
frame 2 $[12, 18)$	A_3 B_2	5/6	C_2 and D_1 do not fit in 1
frame 3 $[18, 24)$	A_4 C_2	4/6	D_1 (4) does not fit in 2: no frame left before 24

```
time  0         10        20
     |.........|.........|...
A      ##....##....##....##....
B      ..###.........###.......
C      ........##..........##..
D      ........................   <- D1 misses its deadline at 24
CPU    AABBB.AACC..AABBB.AACC..
```

Not feasible: D_1 misses its deadline at 24. There are 6 idle units in total, but never 4 in one frame.

Solution (3). Splitting does not change U: $U = 2/6+3/12+2/12+2/24+2/24 = 22/24 = 11/12 = 0.917$.

frame 0 $[0, 6)$	A_1 B_1	5/6	
frame 1 $[6, 12)$	A_2 C_1 D_1	6/6	D_1 (2) now fits in the 2 units left
frame 2 $[12, 18)$	A_3 B_2	5/6	
frame 3 $[18, 24)$	A_4 C_2 E_1	6/6	E_1 finishes at 24 = its deadline: $L = 0$

```
time  0         10        20
     |.........|.........|...
A      ##....##....##....##....
B      ..###.........###.......
C      ........##..........##..
D      ..........##............
E      ......................##
CPU    AABBB.AACCDDAABBB.AACCEE
```

Feasible. D (first part) runs at 10–12, E (second part) at 22–24: the order is right. In frame 0, D_1 and E_1 do not fit in the 1 unit left; in frame 2, C_2 and E_1 do not fit either; so E_1 runs only in frame 3. The CPU is idle only 2 units in 24.

1.11 A tool to check your schedules: schedsim

The way to learn the many scheduling algorithms is to do many exercises, possibly inventing new ones; and since everything in scheduling is an algorithm, the solution of any exercise can be automated. The course provides **schedsim** (in `course-material/tools/schedsim/`), a small Python program that implements all the scheduling algorithms of the course and follows exactly the rules and tie rules of this week: use it after solving on paper, not instead. It is plain Python 3: it runs with `python3`, nothing to install. The **input** is a small text file with the task set, in the slide notation: the algorithm to apply, followed by the data that algorithm needs (different for each algorithm); the folder `examples/` contains one example file per algorithm.

- `explain <file>` solves the exercise and explains every single step, as the solution should be written at the exam (U, the frames, the chart, each job's times, every decision); `solve <file>` gives only the official answers, in the answer format.
- `check <file> <answers>` marks an answer file and, for a wrong schedule, says the first wrong slot and the rule that decides it.
- `run <file>` draws only the Gantt chart; `generate timeline -seed N` creates a new practice exercise (the same seed always gives the same one).

1.11.1 Try it: explain, solve and check a timeline

The task-set file `examples/timeline.txt`, and its explanation:

```
algorithm: timeline
task A C=2 T=5
task B C=2 T=10
task C C=2 T=20

cd course-material/tools/schedsim
python3 schedsim.py explain examples/timeline.txt
```

Output (excerpt; the full output also lists every job and every decision):

```
== Timeline analysis ==
U = sum C/T = 2/5 + 2/10 + 2/20 = 7/10 = 0.700
minor cycle = GCD of periods = 5, major cycle = LCM of periods = 20
frame 0 [0,5): A1 B1 (4/5 used)
frame 1 [5,10): A2 C1 (4/5 used)
frame 2 [10,15): A3 B2 (4/5 used)
frame 3 [15,20): A4 (2/5 used)
every job fits in a frame: the timeline is feasible
== Schedule ==
time  0    5    10   15   20
    |....|....|....|....|
A     ##...##...##...##...
B     ..##........##......
```

```
C     .......##..........
CPU   AABB.AACC.AABB.AA...
schedule=AABB.AACC.AABB.AA...
job  release  start  finish  deadline  response  lateness
C1   0        7      9       20        9         -11
```

`#` = the task runs in that slot; response is $R = f - a$, lateness is $L = f - d$. Then `solve`, and `check` on answers whose schedule contains one mistake:

```
python3 schedsim.py solve examples/timeline.txt
printf 'schedule=AABB.AACC.AABB.AAC..\nU=7/10\nfeasible=yes\nfirst_m
| python3 schedsim.py check examples/timeline.txt -
```

The `\` continues the command on the next line; the `-` tells `check` to read the answers from its standard input (normally they are written in a file, e.g. `my.txt`, checked with `check examples/timeline.txt my.txt`).

```
$ python3 schedsim.py solve examples/timeline.txt
schedule=AABB.AACC.AABB.AA...
U=0.700
feasible=yes
first_miss=none

$ printf '...' | python3 schedsim.py check examples/timeline.txt -
schedule: WRONG
first difference at slot 17: expected ., given C
why: t=17: nothing planned in this part of the frame, idle
U: correct
feasible: correct
first_miss: correct
score: 3/4
```

Slot 17 is in frame 3, which holds only A_4: C has one job per major cycle, already run in frame 1. `U=7/10` is marked correct although the solution says `0.700`: both forms are accepted.

1.11.2 The answer format

The same format is used by the tool and in the **written exam** (rules in `Exam.md`, section "Scheduling exercises: conventions and answer format"):

- One answer per line, as `key=value`, no spaces.
- `schedule=` then one character per time unit: the identifier of the task that runs, or `.` when the CPU is idle; a 20-unit schedule has exactly 20 characters.
- `U=` as a fraction (`7/10`) or a decimal rounded to three places (`0.700`; 17/24 → `0.708`). Any fractional value, not only `U`, may be written either way: for the first FCFS worked example, `avgW` is `11/3` or `3.667` (the slides round it to 3.67 to fit the table).

- `feasible=yes` or `feasible=no`; `first_miss=` the first deadline missed, as `<task>@<time of the deadline>` (e.g. `B@6`, or `D@24` in the "frame too small" exercise), or `none`.
- `R_<id>` and `L_<id>` for the response time and lateness of a job when an exercise asks for them, e.g. `R_D1=7`, `L_D1=-9`. For a timeline, `check` marks only `schedule`, `U`, `feasible` and `first_miss`.
- For the algorithms of Unit 1 (FCFS, round robin, priority), the keys are `W_<id>` (waiting time), `TA_<id>` (turnaround time), `avgW` and `avgTA`.
- The scheduling conventions (time in whole units, $D = T$, the tie rules, higher number = higher priority) are those of `Exam.md`: the same in the slides, the tool and the exam.

For example, `solve` on the FCFS exercise of Unit 1 (the file `course-material/images/week-05/w5-f` prints:

```
schedule=AAAABBBC.DD
W_A=0
W_B=2
W_C=4
W_D=0
TA_A=4
TA_B=5
TA_C=5
TA_D=2
avgW=1.500
avgTA=4
```

Why use the tool. It is a good way to practise, since you can invent as many exercises as you want and get their solutions, and it makes you familiar with the **answer format**. At the exam the scheduling exercises are a pre-formatted text file where the answers must be filled in, and the professor grades them by running `schedsim check` on them. The format must be respected: if it is broken, the tool marks the exercise as wrong even if the solution is correct. In that case the exercise can be checked manually, on request by email, but with about 360 students at the exam, respecting the format is what makes automatic grading possible.

1.12 Unit 3: summary

- A real-time system must be correct and on time: the right result after the deadline is a failure. Real-time is not fast: it guarantees each task, not the average.
- Deadlines are hard (a miss is catastrophic), firm (a late result is useless) or soft (a late result is still useful).
- Tasks are periodic (every T), sporadic (a minimum gap between jobs) or aperiodic (no known gap).

- Notation: release a, phase Φ, WCET C, period T, relative deadline $D = T$, absolute deadline $d = a + D$, response time $R = f - a$, lateness $L = f - d$.
- Utilisation $U = \sum C/T$ must be at most 1; the hyperperiod is the LCM of the periods. We plan with the WCET, never the average.
- Timeline scheduling: minor cycle = GCD, major cycle = LCM, jobs placed shortest period first into frames, no preemption; simple and predictable but fragile, and non-harmonic periods make it fail or explode.
- `schedsim` (`explain`, `solve`, `check`) checks paper solutions, with the answer format and conventions of the exam.

1.13 A prova di coglione

Fino alla settimana scorsa lo scratch-OS eseguiva i task *uno dopo l'altro*. Questo capitolo risponde a una domanda: come si fa a far girare più task "insieme" su una sola CPU e, quando serve, a garantire che ognuno finisca *in tempo*? Il percorso è diviso in tre unità.

- **Unità 1: le idee (solo teoria).** Che cos'è un task e in quali stati si trova (pronto, in esecuzione, bloccato, terminato); che cosa il kernel deve ricordare di ognuno (il TCB); quando entra in gioco lo scheduler. Poi tre politiche per decidere chi usa la CPU: *first-come first-served*, *round robin* e *priorità*. Si confrontano con semplici misure (tempi di attesa, di risposta, di completamento) su esercizi svolti a mano, con le regole che valgono anche all'esame.
- **Unità 2: lo scratch-OS diventa multitasking.** Si studia il codice che realizza il round robin: il *context switch* (salvare un task e riprenderne un altro esattamente da dove si era fermato), fatto con l'eccezione PendSV; il tick di sistema che scandisce i turni; le chiamate che cedono la CPU (yield, delay); il task *idle*, che gira quando nessun altro può. Alla fine si scrive un piccolo *trace* che mostra chi ha usato la CPU e quando. È la base del progetto del corso.
- **Unità 3: il tempo reale.** Nei sistemi real-time conta arrivare *in tempo*, non essere veloci in media. Si classificano le scadenze (hard, firm, soft) e i task (periodici, sporadici, aperiodici), si introducono le grandezze per ragionarci (utilizzo della CPU, iperperiodo, tempo di esecuzione nel caso peggiore) e un primo metodo per pianificare tutto in anticipo, il *timeline scheduling*. Lo strumento `schedsim` permette di controllare le soluzioni nel formato richiesto all'esame.

In breve: prima *che cosa* significa schedulare, poi *come* lo fa davvero un sistema operativo, infine *quando* non basta essere veloci ma bisogna essere puntuali.

1.13.1 Unità 1

L'idea. Una CPU esegue un'istruzione alla volta: il kernel **alterna** i task (A, poi B, poi A) e lo **scheduler** decide a chi tocca. Bisogna sapere *chi* può girare (gli stati), *che cosa* il kernel ricorda di ogni task (il TCB), *quando* si decide e *come*.

Task, context switch e scheduler

Un **programma** è codice fermo; un **processo** è un programma in esecuzione (codice, dati, stack, registri). Un **task** embedded ha stack e registri suoi, ma vive in un *unico* eseguibile con il kernel, senza spazio di indirizzamento privato. Passare da A a B è un **context switch**: si salva tutto lo stato della CPU di A e si carica quello di B, e A non se ne accorge. La **politica** (l'algoritmo) è solo la regola di scelta, poche righe di C; il context switch, invece, dipende dall'hardware ed è la parte difficile (Unità 2).

Stati e transizioni

- **RUNNING**: ha la CPU; al massimo uno per core.
- **READY**: gli manca solo la CPU.
- **BLOCKED**: aspetta un evento (delay, dati, I/O).
- **FINISHED**: ha chiamato `exit`.

Si passa da READY a RUNNING quando lo scheduler lo sceglie. Da RUNNING si torna a READY per preemption o yield, si va in BLOCKED quando si aspetta, in FINISHED con `exit`. **Regola d'oro**: da BLOCKED si va solo in READY, *mai* direttamente in RUNNING.

Forma dei task e TCB

Un task embedded è **run to completion** (setup, lavoro, `exit`) oppure un **ciclo infinito** (setup, poi per sempre: aspetta un evento, lavora). Il ciclo infinito è il caso più comune, e mentre il task è BLOCKED lascia la CPU agli altri.

Il **TCB** è la scheda del task. Contiene lo **SP salvato** (dove sono i registri, sul suo stack), lo **stato**, l'**identità** (nome, funzione, limiti di stack e partizione), le **info di scheduling** (priorità, tick di risveglio) e il puntatore al **TCB successivo**, per costruire le code. I TCB stanno in un array (fisso) o in una lista (dinamica).

Quando gira lo scheduler

Non è un task: è una funzione del kernel, chiamata al boot e in quattro punti:

1. il **tick**, l'interrupt periodico: conta il quanto, può togliere la CPU e sveglia i delay scaduti;
2. una **chiamata bloccante**;
3. uno **yield**;
4. un **exit**.

Attenzione: svegliare un task *non* chiama lo scheduler. Il task diventa READY e aspetta il prossimo punto di attivazione.

Preemptive o cooperativo. Un kernel **preemptive** toglie la CPU anche senza il consenso del task: è reattivo, ma i dati condivisi vanno protetti. Uno **cooperativo** aspetta che il task blocchi, ceda o termini: è semplice, ma un solo loop infinito congela tutto. Avere interrupt non significa avere preemption.

Esercizio "segui due task". A ogni evento chiediti:

- che cosa succede a chi gira;
- se qualcuno si sveglia (e allora diventa solo READY);
- chi sceglie lo scheduler.

Le due trappole: la fine del quanto non toglie la CPU se nessun altro è READY, e READY non vuol dire RUNNING.

Metriche

Un task arriva in a, gli servono C unità di CPU, parte in s e finisce in f.

metrica	**formula**	**obiettivo**	**significato**
utilizzo	occupata / totale	max	quanto lavora la CPU
throughput	task finiti / tempo	max	lavoro prodotto in tutto
turnaround	$f - a$	min	dalla richiesta alla fine
waiting	$f - a - C$	min	tempo passato READY
response	$s - a$	min	attesa prima di partire (il "lag")

La **fairness** chiede che la CPU sia divisa in modo equo e che nessuno aspetti per sempre. Le metriche sono in conflitto, quindi si confrontano le **medie**. Nel real-time, invece, si chiama "response time" la quantità $f - a$.

Regole degli esercizi (anche all'esame)

- Lo schedule si scrive come stringa, con . per la CPU ferma.
- Lo slot t va da t a $t + 1$: chi gira negli slot 0–5 finisce a **6**.
- Il costo dello switch si ignora.
- **Parità**: chi gira tiene la CPU; altrimenti vince la lettera più bassa.
- **Priorità**: numero più alto vuol dire priorità più alta.
- **Round robin**: chi arriva nell'istante in cui scade il quanto entra in coda prima del task interrotto.

Gli algoritmi

FCFS. Coda in ordine di arrivo; ogni task gira fino alla fine (non-preemptive). Il difetto è l'**effetto convoglio**: i task corti aspettano dietro quello lungo. Con gli stessi tre task, l'attesa media è 3.67 se il lungo arriva per primo e 0.33 se arriva per ultimo. Qui waiting e response coincidono.

Round Robin. Coda FIFO. Il task in testa gira al massimo per un quanto q; se il quanto scade, torna in fondo alla coda. Se finisce prima, parte subito il successivo. L'attesa è limitata: al massimo $(n-1)q$.

- q piccolo: buona response, ma molti switch.
- q grande: pochi switch, ma attese lunghe; con q più lungo di ogni task diventa FCFS.

Lo switch è puro spreco: l'overhead è $s/(q+s)$. Con $s = 10\,\mu s$ e $q = 1\,ms$ si perde circa l'1%; con $q = 10\,\mu s$ il 50%.

Priorità. Gira il task READY più importante; il kernel tiene una lista per ogni livello. Solo un task con priorità *strettamente* maggiore toglie la CPU a chi gira. Le priorità sono statiche o dinamiche. FreeRTOS usa priorità più round robin tra i pari. Il rischio è la **starvation**: i task poco importanti possono non girare mai. Il rimedio è l'**aging**: la priorità cresce con l'attesa.

	preemptive	**pro**	**contro**
FCFS	no	semplicissimo	effetto convoglio
Round Robin	sì	attesa limitata	scelta del quanto
Priorità	sì	i task importanti subito	starvation

Metodo per gli esercizi.

1. A ogni istante guarda chi arriva, chi finisce e se scade il quanto.
2. Applica l'algoritmo e le regole di parità, e scrivi la stringa.
3. Ricava s e f (f = ultimo slot +1) e calcola metriche e medie.

Nel mondo reale FCFS resiste nei piccoli dispositivi, le priorità dominano il real-time e i sistemi general-purpose usano il round robin con molte aggiunte.

1.13.2 Unità 2

L'obiettivo. Nella settimana 4 lo scratch-OS eseguiva A fino alla fine e poi B. Ora A e B si **alternano** con un round robin preemptive: ogni task gira per un quanto, poi tocca all'altro. Serve quindi un **context switch** vero: fermare un task in un punto qualsiasi e riprenderlo più tardi esattamente da lì. Questo codice è la **base del progetto**, che cambierà solo il modo in cui un task parte (un programma in FLASH copiato in RAM all'avvio). Nella cartella `base/` lo scheduler è già completo e manca solo il trace; `solution/` contiene anche il trace.

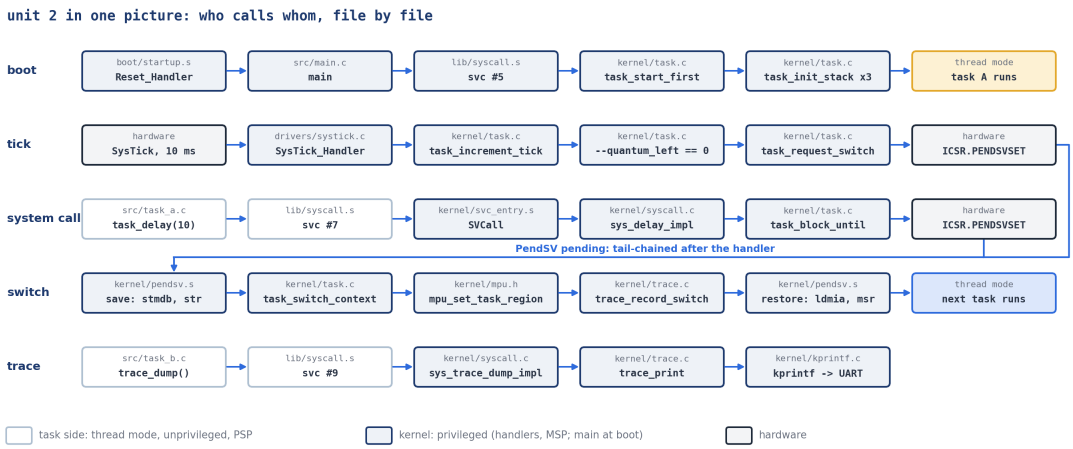


Figure 1.21: L'intera unità in cinque percorsi. All'avvio parte il primo task; il tick o una system call *chiedono* uno switch impostando il bit `PENDSVSET`; PendSV lo *esegue* (salva, sceglie, sposta la finestra MPU, registra nel trace, ripristina); `trace_dump` stampa il trace.

1. Che cosa salvare per mettere in pausa un task

Per riprendere A come se niente fosse servono tre cose: i suoi **registri**, il suo **stack** e i suoi **diritti di memoria**. Lo stack non richiede nulla: ogni task ha il suo, nella propria partizione protetta dall'MPU. Il lavoro di salvataggio dei registri è diviso tra hardware e kernel:

cosa	chi lo salva, e dove	chi lo ripristi
`r0`–`r3`, `r12`, `lr`, `pc`, `xPSR`	la CPU, all'ingresso nell'eccezione, sullo stack del task (PSP)	la CPU, al ritor
`r4`–`r11`	PendSV (`stmdb`), sullo stack, sotto il frame	PendSV (`ldmia`
SP (PSP)	PendSV, nel campo `sp` del TCB	PendSV (`msr p`
diritti di memoria	non si salvano: si ricalcolano	`task_switch_c`

Perché la CPU non salva `r4`–`r11`? Per le regole del C ogni funzione li restituisce intatti al chiamante, quindi per un interrupt normale non serve. Uno switch, però, ritorna in un *altro* task: il kernel deve salvare quelli del vecchio task e caricare quelli del nuovo. Alla fine tutto lo stato del task sta sul suo stack, e nel TCB basta un puntatore.

2. Il TCB e la tabella dei task

La `struct task` della settimana 4 diventa `struct tcb`, con tre campi nuovi:

- `sp`: lo SP salvato;
- `state`: READY, RUNNING, BLOCKED o FINISHED;
- `wake_tick`: il tick di risveglio di un task bloccato.

Restano i campi di prima: nome, funzione d'ingresso, cima dello stack, limiti della partizione e posizione delle variabili.

`sp` **deve essere il primo campo.** L'assembly non conosce le struct: con `sp` all'offset 0, `ldr r0, [r2]` legge lo SP direttamente dal puntatore al TCB. Se fosse primo `name`, PendSV userebbe come stack l'indirizzo in FLASH della stringa e il sistema andrebbe in crash, senza alcun warning in compilazione.

La tabella `tasks[]` contiene A, B e idle, in quest'ordine. `NUM_TASKS` vale 3, `NUM_APP_TASKS` vale 2 e `IDLE_TCB` è l'ultima voce. La tabella non è più `const`, perché `sp`, `state` e `wake_tick` cambiano a run time.

3. Chi fa lo switch: PendSV

Uno switch può servire alla fine di un quanto (nel tick) o a una system call (yield, delay, exit). Copiare il codice dello switch in ogni handler sarebbe un errore. La soluzione: ogni handler si limita a **chiedere** lo switch, e un solo handler lo **esegue**.

- **Chiedere**: `task_request_switch` scrive 1 nel bit `PENDSVSET` (bit 28) dell'ICSR (`0xE000ED04`). Basta una store semplice, perché scrivere 0 negli altri bit non ha effetto. Due richieste insieme costano un solo switch.
- **Priorità**: fault `0x00`, SVCall `0x80`, UART `0xC0`, SysTick e PendSV `0xE0` (la meno urgente). Quindi PendSV non interrompe mai un handler: parte **in tail-chaining**, appena l'ultimo handler finisce, subito prima del ritorno al task.
- **Conseguenze**: PendSV interrompe sempre un *task*, quindi PSP è proprio lo stack di quel task. Inoltre un tick non può spezzare a metà uno switch, perché SysTick ha la stessa priorità. Fare lo switch dentro SysTick sarebbe pericoloso: se il tick interrompe un handler di un dispositivo, quell'handler tornerebbe nel task sbagliato.

4. Lo switch, passo per passo (13 istruzioni)

I numeri sono quelli del primo switch, al tick 5:

1. **Ingresso**: il frame hardware di A è già sul suo stack, spinto dalla CPU quando è entrato SysTick. PSP = `0x200003B8`.
2. **Salva A**: `mrs r0, psp`, poi `stmdb r0!, {r4-r11}` (r0 = `0x20000398`), poi `str r0, [tcb_current]`, cioè `A.sp = r0`.
3. **Scegli**: `bl task_switch_context`. A diventa READY, viene scelto B, `tcb_current = B` e l'MPU region 1 si sposta sulla partizione di B.
4. **Ripristina B**: rilegge `tcb_current`, carica `B.sp` (`0x200013C0`), `ldmia r0!, {r4-r11}`, poi `msr psp, r0`: ora PSP = `0x200013E0`, il frame di B.
5. **Ritorna**: `bx 0xFFFFFFFD` (thread mode, PSP). La CPU estrae il frame di B e B riparte dall'istruzione in cui si era fermato.

Alcuni dettagli da sapere:

- **Chiamare il C dall'handler è sicuro**: `r0`–`r3` sono già nel frame, `r4`–`r11` sono appena stati salvati, e MSP resta allineato a 8 byte. In generale, però, quando l'assembly chiama il C bisogna controllare attentamente quali registri vengono distrutti.

- **L'`lr` di ingresso non si salva**: per tornare a un task serve sempre `0xFFFFFFFD`.
- `tcb_current` **si legge due volte**, perché la funzione C lo cambia.
- `CONTROL` **non si tocca**: nPRIV resta impostato, quindi tutti i task restano unprivileged.
- **L'UART può interrompere PendSV senza danni**, perché non tocca né PSP né i TCB.

5. Il primo avvio: il frame finto

B non ha mai girato, ma PendSV lo tratta come qualunque altro task: carica `sp`, estrae 8 word, imposta PSP e ritorna. Per questo al boot `task_init_stack` prepara su *ogni* stack 16 word, come se il task fosse stato fermato appena prima della sua prima istruzione:

- il frame hardware: `pc` = `entry & ~1` (indirizzo reale dell'istruzione), `lr` = `task_exit`, `xPSR` = `0x01000000` (bit T obbligatorio), gli altri registri a 0;
- sotto il frame, 8 zeri per `r4`–`r11` (vanno bene, perché il C non li legge prima di averli scritti);
- `sp` = cima − 64 byte: per B, `0x20001400` − `0x40` = `0x200013C0`.

Così il primo switch verso un task è identico a tutti gli altri. Solo A, il primissimo task, entra in un altro modo: `main` esegue `svc #5`, e `task_start_first` imposta PSP = `sp` + 8 word (salta gli zeri), imposta nPRIV e ritorna con `0xFFFFFFFD`. Quando una funzione task ritorna, finisce in `task_exit`, cioè fa exit.

6. Scegliere il prossimo task: `task_switch_context`

Non c'è una coda: la tabella stessa è il cerchio. La funzione:

- rimette in READY il vecchio task, se era RUNNING;
- scorre `(rr_last + k) % NUM_APP_TASKS` per $k = 1, 2$ e prende il primo task READY (i BLOCKED e i FINISHED vengono saltati);
- se non trova nessuno, sceglie idle, che non entra mai nel cerchio, e `rr_last` non cambia;
- dà al task scelto un quanto nuovo (`quantum_left = QUANTUM`), aggiorna `tcb_current` e sposta la finestra MPU;
- se il task è davvero cambiato, registra lo switch nel trace.

Un caso da notare: se il quanto di A finisce mentre B è BLOCKED, viene rieletto A, ma il costo dello switch (salvataggio e ripristino) è già stato pagato. Con `malloc` si potrebbe usare una lista di READY, ma questo sistema operativo non ha memoria dinamica.

7. Il tick: `task_increment_tick`

SysTick scatta ogni 10 ms e il quanto si conta in tick, quindi deve essere un multiplo del tick. A ogni tick:

1. `tick_count++`. Finché nessun task è partito, la funzione si ferma qui.
2. Ogni task BLOCKED il cui `wake_tick` è arrivato diventa READY. Il confronto è `(int32_t)(tick_count - wake_tick) >= 0`, che resta corretto anche quando il contatore ricomincia da 0 (dopo 2^{32} tick, circa 497 giorni).
3. Se gira idle, si chiede uno switch *solo* se qualche task si è svegliato.
4. Altrimenti si decrementa `quantum_left`; quando arriva a 0, si chiama `task_request_switch()`.

Con `QUANTUM = 0`, `#if` elimina tutto il codice del quanto e lo scheduler diventa **cooperativo**. Un task svegliato non parte subito: aspetta la fine del quanto di chi sta girando. Il quanto non segue una griglia globale: parte nel momento in cui il task ottiene la CPU.

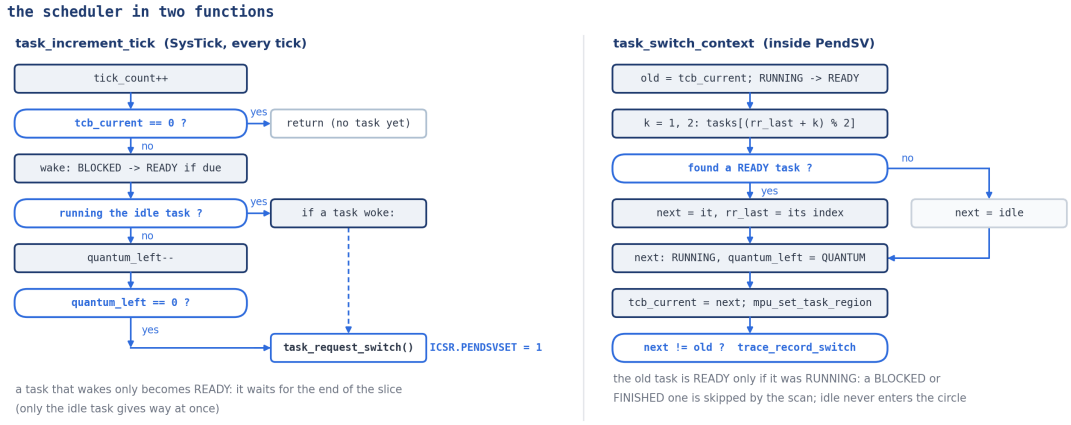


Figure 1.22: Tutto lo scheduler in due funzioni. `task_increment_tick` conta il tempo, sveglia i task e decide se *chiedere* uno switch; `task_switch_context`, dentro PendSV, *sceglie* il task successivo nel cerchio oppure idle, poi sposta l'MPU e registra lo switch nel trace.

8. Cedere la CPU: le system call

Ogni system call è un `svc`; il suo servizio in `kernel/syscall.c` chiede uno switch se serve, e PendSV parte in tail-chaining appena SVCall finisce.

- `task_yield()` (`svc #6`): il task resta READY e torna in fondo al cerchio.
- `task_delay(n)` (`svc #7`): il task è BLOCKED fino a "adesso $+n$"; `delay(0)` equivale a uno yield. Il tempo del lavoro si somma all'attesa, quindi un task periodico **deriva**.
- `task_delay_until(&last, T)` (`svc #8`): il task si sveglia a `last + T` e `last` avanza di T, quindi il periodo resta esatto, **senza deriva**. Il kernel *scrive*

`*last`, perciò prima controlla che l'indirizzo stia nella partizione del chiamante e sia allineato. Se il task è in ritardo non si blocca e recupera subito.

- **exit**: `task_finish_current` rende il task FINISHED e chiede uno switch. Quando resta solo idle stampa `all tasks finished`; `main` non riprende mai il controllo.

Entrambi i delay passano per `task_block_until`, che imposta `wake_tick`, mette lo stato a BLOCKED e chiede lo switch. Quando il task viene scelto di nuovo, riprende subito dopo l'`svc` e lo stub ritorna al chiamante.

Ogni transizione di stato corrisponde a una riga di codice:

- READY ↔ RUNNING in `task_switch_context`;
- → BLOCKED in `task_block_until`;
- BLOCKED → READY in `task_increment_tick`;
- → FINISHED in `task_finish_current`.

Non esiste BLOCKED → RUNNING.

9. Il task idle

Quando nessun task applicativo è READY gira `idle_task`, un ciclo infinito di `wfi`: la CPU dorme fino al prossimo interrupt e risparmia energia.

- È un task come gli altri: unprivileged, con la sua partizione di 4 KB (`0x20002000`). Switch, MPU e stack non hanno casi speciali.
- Non ha quanto e non entra nel cerchio: lo si lascia solo quando qualcuno si sveglia.
- Perché non un ciclo dentro il kernel? Un handler che cicla bloccherebbe SysTick e PendSV: niente più tick, risvegli né switch.

10. Le prove

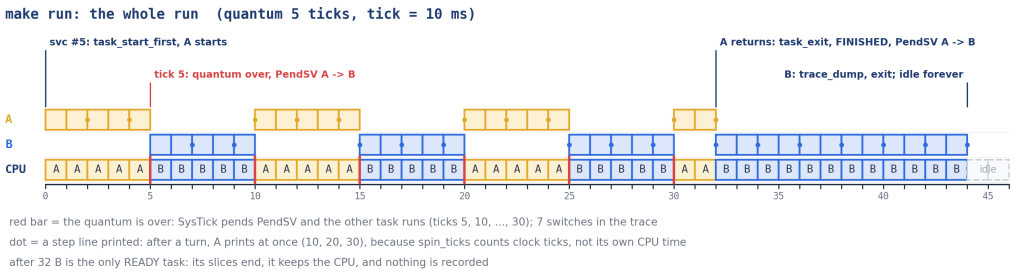


Figure 1.23: `make run` dall'inizio alla fine: A e B si alternano ogni 5 tick, A termina al tick 32, B stampa il trace e termina al 44, poi gira idle per sempre.

- `make run`: dopo ogni turno, A stampa il passo *subito* (ai tick 10, 20, 30), perché `spin_ticks` conta i tick dell'orologio e non il proprio tempo di CPU.

- `QUANTUM=2`: un passo per turno, A termina al 40 e B al 50, con 21 switch invece di 7. È l'overhead $s/(q+s)$ dell'Unità 1 visto dal vivo.

- `DEMO=rogue`: dopo uno switch, B scrive su `uptime_ms` nella RAM del kernel e scatta MemManage (`CFSR=0x82`, indirizzo `0x20003094`). L'isolamento della settimana 4 regge anche dopo gli switch.

- `DEMO=coop`: con `QUANTUM=0` e nessuno yield, B non gira mai (starvation). Serve uno `task_yield()` nei cicli di *entrambi* i task.

- `DEMO=idle`: A (`delay(10)`) gira ai tick 0, 11, 22, 33, 44 e deriva; B (`delay_until`, periodo 6) gira ai tick 7, 13, 19, 25, 31; idle riempie i buchi.

- **gdb**: le variabili del kernel si leggono solo da fermi dentro un handler. In thread mode anche le letture di gdb passano dall'MPU con i diritti del task.

11. Il trace

Un `printf` dentro lo scheduler è lentissimo e cambierebbe proprio lo schedule da osservare. Perciò si **registra in RAM** e si **stampa dopo**.

- **Record**: `{tick, from, to}` occupa 8 byte (6 più 2 di padding). Si scrive in `task_switch_context` solo se `next != old`; l'indice di un task è `old - tasks`, una differenza tra puntatori che conta elementi.

- **Ring buffer** di 64 record: `count` conta tutti i record scritti, e il prossimo va nello slot `count % 64`. `stored()` vale al massimo 64; `oldest_plus(i)` = `buffer[(count - stored() + i) % 64]`. Quando il buffer è pieno, il record più vecchio sta proprio nello slot che verrà sovrascritto.

- **Stampa**: `trace_dump()` (`svc #9`) chiama `trace_print`, che usa `kprintf`. Escono `TRACE BEGIN`, le righe `T tick from to`, `TRACE END`, e poi il Gantt: `owner_at(t)` è il `to` dell'ultimo record con tick $\leq t$, le righe sono da 64 tick, e il controllo `stored() > 0` evita di disegnare dati mai scritti. `schedsim trace` legge le stesse righe.

- **Nessun lock**: PendSV, che scrive, e SVCall, che legge, non si interrompono mai a metà.

- **Limiti**: il trace non sa chi girava prima del primo record. Lo switch B $\rightarrow$ idle avviene dopo la stampa. Con due record allo stesso tick, il Gantt mostra l'ultimo.

La lezione. Un sistema operativo è fatto di tante parti piccole e semplici che lavorano insieme; il difficile è collegarle bene e tenere sotto controllo ogni evento possibile. I bug grandi fanno crashare subito il sistema e si correggono; il vero incubo sono quelli piccoli, che emergono solo in rari casi limite.

1.13.3 Unità 3

L'idea. Finora uno scheduler doveva essere "buono in media". Da qui in poi conta altro: **ogni** task deve finire **entro una scadenza**. È lo scheduling **real-time**, fondamentale nei sistemi embedded, perché controllano qualcosa di fisico (freni, motori, valvole) che non aspetta.

1. Corretto *e* puntuale

Un programma normale deve solo dare il risultato giusto, prima o poi. Un sistema real-time deve dare il risultato giusto **entro** $t + d$, dove d è la **deadline** fissata in anticipo. Un risultato giusto ma in ritardo vale quanto un risultato sbagliato. Esempio: il comando dei freni deve agire entro pochi millisecondi; un comando perfetto che arriva 100 ms dopo è inutile.

Hard, firm, soft. La classe dipende dalla **conseguenza** di una deadline mancata, non da quanto è corta (l'airbag a 30 ms è hard, un frame video a 40 ms è firm):

- **hard**: la conseguenza è catastrofica (airbag, freni, pacemaker, motori di un drone);
- **firm**: il risultato in ritardo è inutile e si **butta**, ma non fa danni (un frame video, un campione audio);
- **soft**: il risultato in ritardo è **ancora utile**, solo peggiore (contapassi, correttore ortografico, orologio dello smartwatch).

In un sistema hard bisogna **dimostrare** prima dell'esecuzione che ogni deadline sarà rispettata: i test mostrano solo i casi provati. Un sistema reale spesso mescola le tre classi (un'auto ha freni hard, fusione dei sensori firm, infotainment soft). Nel corso, "task real-time" significa hard. I sistemi desktop non sono real-time; per l'hard real-time serve un **RTOS**, piccolo e pieno di vincoli.

2. Real-time non vuol dire veloce

Un sistema **veloce** ottimizza una **media** (throughput, turnaround medio). Un sistema **real-time** garantisce **ogni singolo task**, anche nel caso peggiore, a costo di rallentarne qualcuno. Conta la **prevedibilità**: sapere in anticipo quando finirà ogni task. Una CPU più veloce aiuta, ma non garantisce niente: decide lo scheduling.

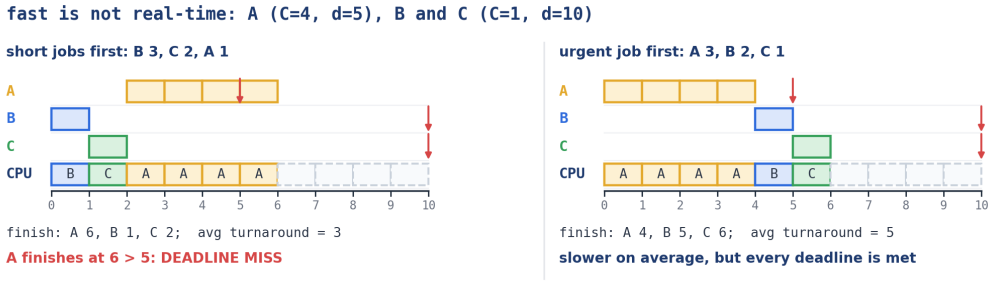


Figure 1.24: Gli stessi tre job con due scelte di priorità. A sinistra si privilegiano i job corti: la media è migliore (3), ma A finisce a 6 e manca la deadline 5. A destra si privilegia il job urgente: la media è peggiore (5), ma tutte le deadline sono rispettate.

3. Tre tipi di task

Un task real-time è una sequenza infinita di **job**: ogni job è un giro del ciclo infinito dell'Unità 1.

- **Periodico**: un job ogni T (es. leggere la temperatura ogni 25 ms, in polling). Le attivazioni sono prevedibili, quindi si può **dimostrare a priori** che tutte le deadline saranno rispettate. Nel corso la deadline coincide sempre con la fine del periodo.
- **Sporadico**: attivato da eventi in istanti ignoti, ma con un **intervallo minimo** tra due job. Esempio: un calcolo a ogni giro dell'albero motore; a 6000 RPM un giro dura $60\,s/6000 = 10$ ms. Nel caso peggiore si comporta come un periodico con T uguale a quell'intervallo minimo.
- **Aperiodico**: eventi senza alcun intervallo minimo, anche due nello stesso istante (es. un pulsante sul cruscotto). Non si può garantire nulla, si può solo ridurre la probabilità di mancare le deadline.

Conseguenza di progetto: ciò che non deve *mai* mancare una deadline va reso periodico quando possibile. Il polling trasforma un evento in un task periodico, ma spreca CPU; gli interrupt sono efficienti, ma meno prevedibili.

4. Il modello e la notazione

Le garanzie valgono solo se il modello è rispettato:

- i task sono in numero **fisso** e noti in anticipo;
- sono tutti **periodici**;
- sono **indipendenti**: niente risorse condivise, nessuno aspetta un altro;
- la deadline relativa è uguale al periodo: $D = T$;
- ogni task ha un **WCET** C noto;
- l'overhead del kernel (es. il context switch) è trascurabile;

- c'è **una sola CPU** (nei multi-core embedded ogni core è una CPU indipendente, con task assegnati staticamente).

Per il job k di un task:

rilascio	$a = \Phi + (k-1)\,T$	(fase Φ, di solito 0)
deadline assoluta	$d = a + D = a + T$	(coincide con il rilascio successivo)
inizio e fine	s, f	(in generale $f \neq s + C$, perché il job può essere interrotto)
tempo di risposta	$R = f - a$	(rispetta la deadline se $R \leq D$)
lateness	$L = f - d$	(negativa: in anticipo; 0: giusto in tempo; positiva: deadline mancata)

Attenzione: qui "response time" è $f - a$, cioè il *turnaround* dell'Unità 1.

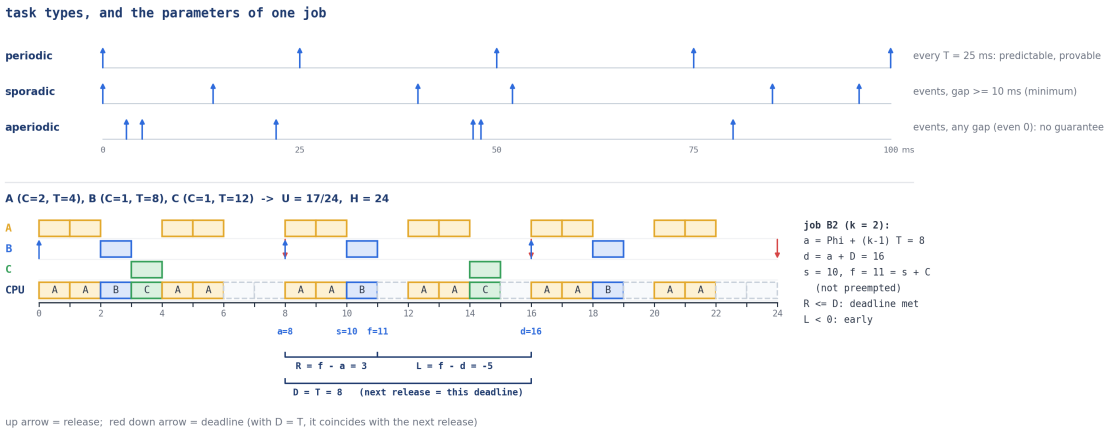


Figure 1.25: In alto, i tre tipi di task: rilasci regolari (periodico), con un intervallo minimo (sporadico) o senza alcun vincolo (aperiodico). In basso, il job B_2 dell'esercizio sulla notazione: $a = 8$, $s = 10$, $f = 11$, $d = 16$, quindi $R = 3$ e $L = -5$.

5. Utilizzazione, iperperiodo, WCET

Utilizzazione. Un task occupa la frazione C/T della CPU; l'intero insieme di task ha $U = \sum C_i/T_i$.

- Se $U > 1$, nessun algoritmo può rispettare tutte le deadline su una CPU: ci si **ferma**, senza disegnare nulla (all'esame le domande successive non richiedono risposta).
- $U \leq 1$ è **necessaria ma non sufficiente**: gli algoritmi semplici possono fallire anche con $U < 1$, sotto-utilizzando la CPU.
- Nella realtà, se $U > 1$: il periodo è fissato dai requisiti e non si cambia; C si riduce con codice migliore, ma fino a un limite. La soluzione tipica è una CPU un po' più potente (e più costosa).

Iperperiodo. $H = \mathrm{LCM}(T_1, \ldots, T_n)$: con fasi nulle lo schedule si ripete identico ogni H, quindi basta disegnarlo in $[0, H)$, dove il task i ha H/T_i job. Esempio: con periodi 5, 10, 20 si ha $H = 20$, cioè 4, 2 e 1 job. La schedulabilità si verifica **con una formula** (per gli algoritmi che ne hanno una) oppure **empiricamente**, disegnando un iperperiodo.

WCET. Un task ha più percorsi di esecuzione, e la garanzia deve valere per tutti: si pianifica sul caso peggiore, mai sulla media. Con la media, proprio il raro job lento (es. quando si registra un errore) mancherebbe la deadline. Calcolare il WCET esattamente è impossibile per programmi complessi: lo si stima misurando o con la statistica, e quando la sicurezza conta lo si **sovrastima** (anche di 3–4 volte), pagando con CPU sprecata. Nel corso il WCET è sempre dato.

6. Timeline scheduling (cyclic executive)

È l'algoritmo meno efficiente e senza una formula di schedulabilità, ma è semplicissimo e **prevedibilissimo**: lo usano quasi tutti i prodotti avionici.

- Lo schedule si calcola **offline**, una volta sola, e diventa una tabella fissa: a run time nessuno decide niente.
- Il **minor cycle** (frame) è $m = \mathrm{GCD}(T)$: ogni rilascio cade sul bordo di un frame. Il **major cycle** è $M = \mathrm{LCM}(T) = H$: si riempiono M/m frame e poi tutto si ripete.
- **Regole del corso**, frame per frame:
  - si considerano i job già rilasciati e non ancora eseguiti, **periodo più corto prima** (a parità di periodo, l'identificatore più basso);
  - un job si piazza solo se **entra tutto**, perché non attraversa mai il bordo di un frame; se non entra, aspetta un frame successivo, e si provano comunque i job seguenti;
  - si usa il primo slot libero e non c'è preemption; il resto del frame resta idle.
- Lo schedule è **fattibile** se ogni job sta in un frame che finisce entro la sua deadline; altrimenti c'è una deadline mancata.

Esempio. A (2, 5), B (2, 10), C (2, 20): $U = 0.7$, $m = 5$, $M = 20$, schedule `AABB.AACC.AABB.AA...`. Nel frame 0 C_1 non entra (resta 1 unità): va nel frame 1, dopo A_2, che ha il periodo più corto.

Implementazione. Bastano un timer che segnala l'inizio di ogni frame e un ciclo infinito che chiama, frame dopo frame, una funzione con i job di quel frame:

- `wait_frame` aspetta il bordo del frame successivo; il flag è `volatile`, perché lo modifica l'interrupt e il compilatore non deve tenerne una copia;
- non servono context switch, TCB o code: tutti i task usano un unico stack;
- se il flag è già alzato quando si arriva a `wait_frame`, c'è un **overrun** da segnalare.

Pro e contro.

- **Pro**: è semplicissimo, tutto è verificato offline, non ci sono preemption (quindi nessun conflitto sui dati condivisi) né costi di switch.
- **Fragile**: un nuovo task, o un C o un T cambiato, può stravolgere tutta la tabella.
- **Job troppo lunghi**: un job più lungo di un frame va spezzato a mano.
- **Eventi aperiodici**: si gestiscono solo con il polling.
- **Overrun**: un job che dura più del previsto ritarda tutti i frame seguenti.

Periodi non armonici. I periodi sono armonici se ognuno divide quelli più lunghi (5, 10, 20). Se non lo sono, il GCD diventa piccolo (con 5, 7, 11 è 1, e nessun job con $C \geq 2$ entra in un frame) e l'LCM enorme (385 frame). Due rimedi:

- **spezzare** un task lungo in parti, dando alla prima l'identificatore più basso, così le parti girano nell'ordine giusto;
- **accorciare** qualche periodo fino a renderlo armonico: è sempre lecito, ma U cresce.

Esempio: A (1, 4) e B (3, 6) danno frame da 2 unità, in cui B non entra mai: B_1 manca la deadline a 6, pur con $U = 0.75$. Con il periodo di B ridotto a 4 lo schedule diventa `ABBB`, fattibile, con $U = 1$.

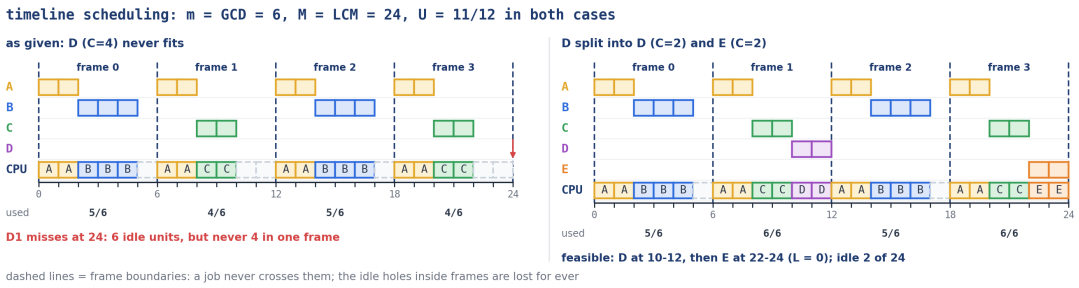


Figure 1.26: L'esercizio "a frame too small". Con $U = 11/12$, D ($C = 4$) non entra in nessun frame e D_1 manca la deadline a 24: ci sono 6 unità idle in tutto, ma mai 4 nello stesso frame. Spezzando D in D ed E (2 unità ciascuno) lo schedule diventa fattibile, e le due parti girano nell'ordine giusto.

7. Il metodo completo e `schedsim`

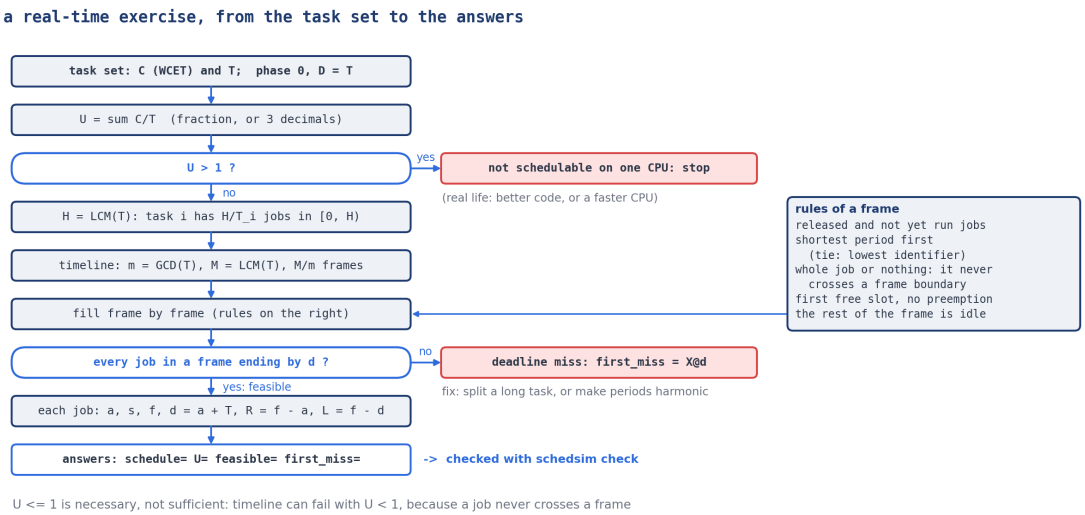


Figure 1.27: Come si risolve un esercizio real-time: prima U (se è maggiore di 1 ci si ferma), poi H, i frame riempiti con le regole del corso, la fattibilità, i parametri dei job, e infine le risposte nel formato d'esame.

`schedsim` (`course-material/tools/schedsim/`, Python 3, nessuna installazione) applica gli algoritmi del corso con le stesse regole di parità. Va usato *dopo* aver risolto l'esercizio su carta:

- `explain` mostra ogni passo, come andrebbe scritto all'esame;
- `solve` dà solo le risposte;
- `check` corregge un file di risposte e indica il primo slot sbagliato e la regola che lo decide;
- `run` disegna solo il Gantt; `generate timeline -seed N` crea un esercizio nuovo per allenarsi.

Formato delle risposte (anche all'esame). Una risposta per riga, `chiave=valore`, senza spazi:

- `schedule=` con un carattere per unità di tempo e `.` quando la CPU è idle;
- `U=` come frazione (`7/10`) oppure con tre decimali (`0.700`);
- `feasible=yes` oppure `feasible=no`;
- `first_miss=` nella forma `D@24`, oppure `none`;
- `R_D1=7` e `L_D1=-9` quando l'esercizio chiede i parametri di un job;
- per FCFS, round robin e priorità: `W_<id>`, `TA_<id>`, `avgW`, `avgTA`.

All'esame il file viene corretto automaticamente con `schedsim check`: se il formato è sbagliato, l'esercizio risulta sbagliato anche con la soluzione giusta.